



INSTITUT SUPERIEUR DE TECHNOLOGIE D'ANTANANARIVO

--oOo--

*Premier Établissement Public d'Enseignement Supérieur
et de Recherche Habilité et Accrédité à Madagascar par le MESUPRES*



ECOLE DU GENIE INDUSTRIEL

Mention : **GENIE ENERGETIQUE ET AUTOMATISME**

MEMOIRE DE FIN D'ETUDES EN VUE DE L'OBTENTION DU DIPLOME DE
TECHNICIEN SUPERIEUR SPECIALISE DE GRADE LICENCE

Parcours : **GENIE DES SYSTEMES AUTOMATISES**

ÉTUDE ET DEVELOPPEMENT DE DRIVE INSIGHT, UNE APPLICATION WEB POUR LA GESTION DE LOCATION DE VEHICULES.

Présenté par **Sehenonirina Elisa RANDRIAMASINORO** le Mercredi 14 août 2024
à 11h

Membres du jury :

Président : Docteur Pierrot RANJARANIMARO

Examineur pédagogique : Monsieur Andriamihaja RASOANARIVO

Examineur professionnel : Docteur Soloniaina RAKOTOARIJAONA

Rapporteur : Madame Mirantsoa Volana RANDRIAMANDIMBISOA

XVI^{ème} Promotion

REMERCIEMENTS

Je souhaite tout d'abord exprimer ma gratitude envers le Dieu Saint-Esprit pour m'avoir accordé la patience, le courage, la volonté et la motivation nécessaires pour mener ce modeste travail à terme.

Par la présente, je souhaite témoigner de ma profonde gratitude envers toutes les personnes au sein de l'Institut Supérieur de Technologie d'Antananarivo, dont le soutien et la collaboration ont été essentiels à la réalisation de ce mémoire de fin d'études :

- Professeur Luc RAKOTONDRAJONA, Directeur Général de l'Institut Supérieur de Technologie d'Antananarivo ;
- Docteur Pierrot RANJARANIMARO, Directeur de l'École du Génie Industrie et Président du jury ;
- Docteur RAKOTO RAZAFINDRAZATO Guy Marie, Chef de la mention Génie Energétique et Automatisme ;
- Docteur Soloniaina RAKOTOARIJAONA, Examineur professionnel ;
- Monsieur Andriamihaja RASOANARIVO, Examineur pédagogique ;
- Madame Mirantsoa Volana RANDRIAMANDIMBISOA, Rapporteur.

Enfin, je tiens à exprimer ma profonde gratitude envers mes parents qui ont été mes plus grands soutiens, que ce soit sur le plan financier ou émotionnel, tout au long de mon stage et lors de l'élaboration de ce document.

TABLES DES MATIERES

REMERCIEMENTS.....	i
TABLES DES MATIERES	ii
AVANT-PROPOS	iv
GLOSSAIRE.....	v
LISTES DES ACRONYMES	vi
LISTES DES FIGURES	vii
LISTES DES TABLEAUX	ix
INTRODUCTION GENERALE.....	1
PARTIE I MATERIELS ET METHODES	2
Chapitre I.1 Système Automatisé	3
I.1.1 DEFINITIONS	3
I.1.2 ARCHITECTURE D'UN SYSTEME AUTOMATISES	3
Chapitre I.2 Internet des objets	5
I.2.1 DEFINITION.....	5
I.2.2 CYCLE DE DONNEES DE L'IOT.....	5
I.2.3 MICROCONTROLEUR	5
I.2.4 MODULE GPS	8
I.2.5 MODULE GSM/3G/LTE.....	10
I.2.6 DOMAINE D'APPLICATION	11
Chapitre I.3 Technologie numérique	13
I.3.1 PROTOCOLE HTTP	13
I.3.2 PROGRAMMATION.....	13
I.3.3 DEVELOPPEMENT WEB.....	17
I.3.4 LOGICIEL XAMPP	20
I.3.5 MACHINE LEARNING	21
Chapitre I.4 Cadrage des travaux.....	27
I.4.1 METHODOLOGIE DE L'ANALYSE FONCTIONNELLE	27
I.4.2 ANALYSE FONCTIONNELLE DU SYSTEME	28
PARTIE II RESULTATS ET DISCUSSIONS.....	31
Chapitre II.1 Principe de fonctionnement du système.....	32
II.1.1 FONCTIONNEMENT EN BLOC DU SYSTEME	32
II.1.2 ARCHITECTURE DU SYSTEME.....	34
II.1.3 COMMUNICATION DES HARDWARES	36
Chapitre II.2 Intégration des IoT et GPS.....	37
II.2.1 ETUDE DU SYSTEME D'IOT	37

II.2.2	TRAITEMENT DES DONNEES	40
Chapitre II.3	Conception de l'application web.....	42
II.3.1	MODE DE FONCTIONNEMENT DE L'APPLICATION.....	42
II.3.2	ARCHITECTURE GENERALE DE L'APPLICATION	43
II.3.3	MODELISATION DE LA BASE DE DONNEES.....	45
II.3.4	FONCTIONNALITES MODE UTILISATEUR	49
II.3.5	FONCTIONNALITES MODE ADMINISTRATEUR.....	54
Chapitre II.4	Développement et déploiement de l'application web.....	60
II.4.1	DEVELOPPEMENT FRONTEND	60
II.4.2	DEVELOPPEMENT BACKEND	64
II.4.3	DEPLOIEMENT DE L'APPLICATION WEB SUR XAAMP	70
Chapitre II.5	Discussions	71
II.5.1	ANALYSE SWOT	71
II.5.2	COUT DE MISE EN ŒUVRE	72
II.5.3	PERSPECTIVE D'AMELIORATION	72
CONCLUSION GENERALE.....		76
REFERENCES BIBLIOGRAPHIQUES.....		I

AVANT-PROPOS

L'Institut Supérieur de Technologie d'Antananarivo (IST-T), composé de trois Ecoles, à savoir, l'École du Génie Industriel, l'École du Génie Civil et l'École du Génie du Management d'Entreprise et du Commerce, est un établissement d'enseignement supérieur à caractère technologique et professionnel formant des Techniciens et des Ingénieurs à Madagascar.

Le parcours académique inclut un cursus suivi à l'École du Génie Industriel. Cette formation a englobé trois années d'études permettant l'acquisition de connaissances techniques et technologiques, à la fois théoriques et pratiques, grâce à des stages réguliers intégrés à chaque année scolaire.

Le présent document constitue le mémoire de fin d'études, destiné à être présenté devant un jury composé de membres qualifiés. Ce moment marquera la conclusion du parcours dans la mention Génie Energétique et Automatisme. À travers ce mémoire, les travaux effectués au sein de l'IST sont mis en lumière.

La soutenance de ce mémoire représente une étape essentielle pour l'obtention du diplôme de Technicien Supérieur. Une reconnaissance est exprimée envers toutes les personnes ayant apporté leur soutien tout au long de ce parcours.

GLOSSAIRE

Backend : La partie cachée d'un site web ou d'une application qui gère la logique, les bases de données, l'authentification, et la communication avec le serveur.

Frontend : La partie visible d'un site web ou d'une application avec laquelle les utilisateurs interagissent directement.

Serveur : Un serveur est généralement un appareil ou un logiciel qui fournit des services, des ressources ou des données à d'autres appareils, appelés clients.

Protocole : Un protocole est un ensemble de règles et de conventions qui définissent la manière dont les données sont transmises et reçues sur un réseau. Les protocoles courants incluent TCP/IP (utilisé sur Internet) et HTTP (utilisé pour les pages web).

LISTES DES ACRONYMES

API	: Application Programming Interface
CAN	: Controller Area Network
CPU	: Central Processing Unit
E/S	: Entrée/Sortie
Fc	: Fonction Complémentaire
SWOT	: Strengths Weaknesses Opportunities Threats
Fp	: Fonction Principale
GPIO	: General-Purpose Input/Output
HTTP	: Hypertext Transfer Protocol
I2C	: Inter-Integrated Circuit
IDE	: Integrated Development Environment
IoT	: Internet of Things
LED	: Light Emitting Diode
PWM	: Pulse Width Modulation
RAM	: Random Access Memory
ROM	: Read-Only Memory
SPI	: Serial Peripheral Interface
UART	: Universal Asynchronous Receiver/Transmitter
URL	: Uniform Resource Locator
TIC	technologies de l'information et de la communication

LISTES DES FIGURES

Figure I-1 : Architecture d'un système automatisé.....	3
Figure I-2 : Cycle de données de l'IoT.....	5
Figure I-3 : Structure d'une ESP32.....	6
Figure I-4 : ESP32-CAM.....	7
Figure I-5 : Brochage ESP32-cam	8
Figure I-6 : GPS Autonomes	8
Figure I-7 : Puce intégrant un GPS	9
Figure I-8 : ZED-F9P GPS	9
Figure I-9 : LEA-6 series GPS.....	10
Figure I-10 : Module GSM.....	10
Figure I-11 : Module 3G	11
Figure I-12 : Module LTE/4G	11
Figure I-13 : Interface Arduino IDE.....	15
Figure I-14 : Interface Visual Studio Code	16
Figure I-15 : Logo de l'HTML.....	17
Figure I-16 : Logo du CSS.....	17
Figure I-17 : Logo du JavaScript.....	18
Figure I-18 : Logo du React.....	18
Figure I-19 : Logo d'Angular	18
Figure I-20 : Logo du Vue.js	19
Figure I-21 : Architecture d'une application web	20
Figure I-22 : Tableau de bord du logiciel XAMPP	21
Figure I-23 : Machine Learning vs Programmation traditionnelle.....	22
Figure I-24 : Exemple de Modèle Machine Learning	22
Figure I-25 : Optimisation du modèle	22
Figure I-26 : Représentation du modèle de régression linéaire	23
Figure I-27 : Arbre de décision.....	25
Figure I-28 : Code générique Forêt aléatoire	25
Figure I-29 : Diagramme de bête à corne	27
Figure I-30 : Principe du diagramme FAST.....	28
Figure I-31 : Diagramme FAST d'un aspirateur	28
Figure I-32 : Le diagramme de bête à corne du système	29
Figure I-33 : Diagramme FAST du Système	30
Figure II-1 : Schéma bloc du système.....	33
Figure II-2 : Architecture du Système	34
Figure II-3 : Communication des hardwares.....	36
Figure II-4 : Le module GPS Neo6M.....	37
Figure II-5 : Particule Boron LTE	38
Figure II-6 : Le traitement de données du GPS et capteurs.....	40
Figure II-7 : Le code de traitement des capteurs et GPS	41
Figure II-8 : L'algorithme de fonctionnement de l'application	43
Figure II-9 : Architecture du système	44
Figure II-10 : MCD de l'application web.....	45
Figure II-11 : Code SQL pour créer la table Voitures.....	46
Figure II-12 : Code SQL pour créer la table Clients	47
Figure II-13 : Code SQL pour créer la table Capteurs	47
Figure II-14 : Code SQL pour créer la table Administrateurs	48
Figure II-15 : Code SQL pour créer la table Réservations.....	48

Figure II-16 : Code SQL pour créer la table Maintenances	49
Figure II-17 Scénario d'utilisation du mode utilisateur	50
Figure II-18 Page d'accueil	51
Figure II-19 Page Voitures disponibles	52
Figure II-20 Page détails de la voiture	52
Figure II-21 Page Paiement.....	53
Figure II-22 : Scénario d'utilisation du mode administrateur	55
Figure II-23 : Page d'authentification.....	56
Figure II-24 : L'onglet suivi des données du capteur	57
Figure II-25 : L'onglet map	58
Figure II-26 : L'onglet historique maintenance.....	58
Figure II-27 : L'onglet information du locataire	59
Figure II-28 : Fonctionnalité en bloc du code HTML de la page d'accueil.....	60
Figure II-29 : Fonctionnalité du code JavaScript de la page d'accueil.....	60
Figure II-30 : Fonctionnalité du code de la page de disponibilité des voitures.....	61
Figure II-31 : Fonctionnalité en bloc du code frontend de la page détails du véhicule	61
Figure II-32 : Fonctionnalité du code HTML et PHP de la page paiement	62
Figure II-33 : Fonctionnalité du code HTML de la page d'authentification.....	62
Figure II-34 : Fonctionnalité du code HTML et CSS de la page du choix du véhicule encours de location	63
Figure II-35 : Fonctionnalité du code HTML CSS JS PHP de la page détails de la voiture	63
Figure II-36 : Fonctionnalité en bloc du code backend de la page disponibilité des véhicules.....	64
Figure II-37 : Fonctionnalité du code backend de la page détail du véhicule.....	65
Figure II-38 : Code enregistrement des informations du client dans la base de données	66
Figure II-39 : Code PHP de l'authentification	67
Figure II-40 : Fonctionnalité du code backend de la page choix du véhicule	68
Figure II-41 : Fonctionnalité en bloc du code PHP pour la page détails du véhicule ..	69
Figure II-42 : Dossier 'location_voiture' dans htdocs de XAAMP	70
Figure II-43 : Fichiers de l'application web dans le répertoire 'location_voiture'	70
Figure II-45 : Nombre de données pour concevoir le modèle prédictif teste	73
Figure II-46 : Divisions des données en test et en entraînement.....	73
Figure II-47 : Création du modèle forêt aléatoire	73
Figure II-48 : Entraînement du modèle	73
Figure II-49 : Evaluation du modèle	74
Figure II-50 : Matrice de confusion.....	74
Figure II-51 : Rapport de classification de l'évaluation.....	75

LISTES DES TABLEAUX

Tableau I-1 : Les techniques du Machine Learning avec leurs algorithmes d'optimisation selon la catégorie d'apprentissage	24
Tableau II-1 : Rôles et Fonctions des Composants.....	35
Tableau II-2 : Choix de composants.....	38
Tableau II-3 : L'analyse SWOT de l'application web	71
Tableau II-4 : Coût de mise en œuvre.....	72

INTRODUCTION GENERALE

Les technologies jouent un rôle essentiel dans l'amélioration de la vie quotidienne, permettant aux individus de vivre et aux entreprises de fonctionner de manière plus efficace et flexible. Les systèmes technologiques ont évolué de simples outils de support à des machines sophistiquées intégrant des niveaux élevés d'automatisation. La gestion des systèmes nécessitait auparavant une surveillance manuelle et une intervention constante pour prévenir les pannes. À présent, les véhicules modernes, notamment dans le secteur automobile, sont équipés de technologies avancées qui surveillent leur état et leurs performances grâce à divers capteurs intégrés. Ces capteurs recueillent des données sur des paramètres critiques, facilitant la prise de décisions et la maintenance.

Dans la gestion des flottes de véhicules, les systèmes automatisés offrent des opportunités pour améliorer la satisfaction des clients tout en maîtrisant les coûts. En effet, les entreprises doivent gérer les réservations, l'entretien des véhicules et la surveillance de l'état des systèmes, tout en anticipant les problèmes potentiels. Cette gestion devient encore plus complexe lorsque les flottes comprennent un grand nombre de véhicules avec des besoins variés. Pour répondre à ces enjeux, il est important d'explorer comment les technologies automatisées peuvent être utilisées pour améliorer le service client et optimiser la gestion des flottes grâce aux capteurs embarqués des véhicules. La problématique de ce mémoire porte alors sur la gestion des flottes et des systèmes technologiques par l'exploitation des données collectées par les capteurs.

Ce sujet cherche à améliorer l'efficacité dans la gestion des systèmes automatisés, particulièrement bénéfique pour des services comme la location de véhicules. L'objectif est de créer une application web qui optimise ces systèmes en utilisant les données des capteurs. L'étude examine comment les technologies modernes peuvent transformer la gestion des flottes de véhicules, où la précision et l'efficacité sont essentielles. Elle propose une approche innovante pour exploiter les données des capteurs, offrant des solutions pratiques aux défis actuels de la gestion des systèmes.

Ce travail est structuré comme suit. Dans la première partie, les matériels et méthodes seront abordés, en détaillant les bases des systèmes automatisés, les domaines d'études et les méthodologies adoptées. La deuxième partie est consacrée aux résultats et discussions, exposant la conception du système ainsi que l'analyse SWOT et son coût de mise en œuvre. Enfin, la conclusion résume les résultats et propose des perspectives d'amélioration.

PARTIE I MATERIELS ET METHODES

Chapitre I.1 Système Automatisé

I.1.1 DEFINITIONS

Un système est un ensemble organisé d'éléments interagissant entre eux et avec l'extérieur, dans le but de réaliser une fonction définie [1].

Un système automatisé ou automatique est un système qui exécute toujours le même cycle de travail qui est programmé à l'avance, sans l'intervention de l'utilisateur. [1]

L'opérateur est responsable de la programmation, du démarrage et de l'arrêt du système. Un système automatisé peut être constitué de plusieurs sous-systèmes automatisés.

I.1.2 ARCHITECTURE D'UN SYSTEME AUTOMATISES

Un système automatisé est composé de deux parties principales : la partie opérative et la partie commande à lesquelles peuvent s'ajouter une troisième, la partie de dialogue :

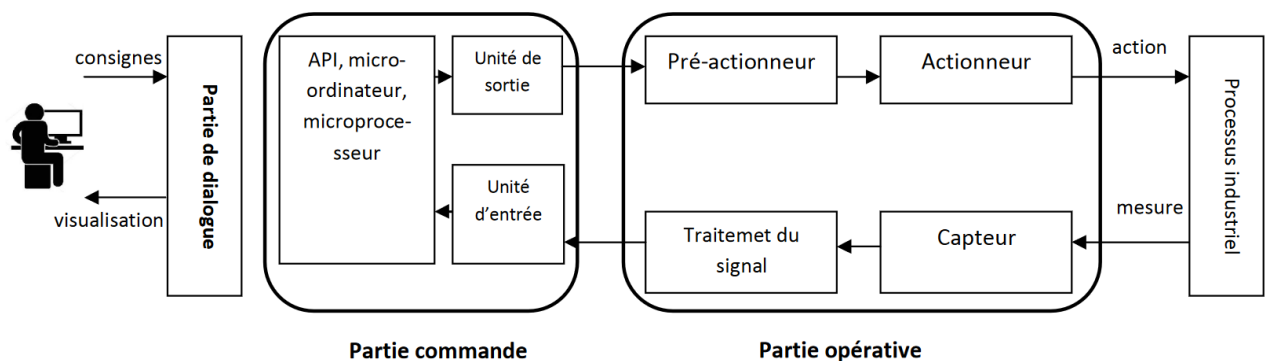


Figure I-1 : Architecture d'un système automatisé

Source : [1]

I.1.2.1. Partie opérative

La partie opérative se compose de plusieurs ensembles : les pré-actionneurs, les actionneurs et les capteurs. Les pré-actionneurs, directement dépendants des actionneurs, sont nécessaires à leur fonctionnement, comme un distributeur pour un vérin. Les actionneurs fournissent à l'unité de production l'énergie mécanique nécessaire à son fonctionnement à partir d'une source d'énergie extérieure, comme dans le cas d'un moteur. Les capteurs, quant à eux, génèrent des informations à partir de grandeurs physiques variées telles que le déplacement, la température ou la pression, informations qui sont ensuite utilisées par la partie commande.

I.1.2.1.1 Pré-actionneurs

Les pré-actionneurs sont des composants qui préparent, dirigent ou modifient les signaux ou les données en amont des actionneurs principaux. Ils jouent un rôle essentiel dans le traitement et la gestion des informations ou des commandes avant qu'elles n'atteignent les composants responsables de l'action physique.

Quelques types courants de pré-actionneurs incluent les électrovannes et les décodeurs. Les électrovannes sont des composants électromagnétiques qui contrôlent le flux de fluide, généralement liquide ou gaz, dans les systèmes pneumatiques et hydrauliques. Les décodeurs, quant à eux, prennent un signal

d'entrée sous forme de code binaire et activent une sortie spécifique en fonction de ce code. Ils sont largement utilisés dans divers systèmes de contrôle et de commutation.

I.1.2.1.2 Actionneurs

Les actionneurs sont des composants qui transforment un signal électrique ou électronique en une action physique, telle que le mouvement, la lumière, le son, etc. Ils sont cruciaux pour mettre en œuvre des fonctions concrètes en réponse aux commandes ou aux informations reçues.

Les moteurs électriques sont des actionneurs courants qui convertissent l'énergie électrique en mouvement rotatif. Ils trouvent des applications variées, allant des véhicules aux machines industrielles. Les diodes électroluminescentes (LED) transforment quant à elles un courant électrique en lumière visible, comme on le voit dans les écrans de téléphones portables.

I.1.2.1.3 Capteurs

Les capteurs sont des dispositifs qui mesurent diverses grandeurs physiques, telles que la température, la pression, la lumière, le déplacement, etc., et les transforment en signaux électriques ou électroniques. Ils sont utilisés pour collecter des données sur l'environnement ou les conditions d'un système.

Les types de capteurs fréquemment utilisés incluent les capteurs de température et les capteurs de pression. Un capteur de température convertit les variations de température en signaux électriques. Les capteurs de pression mesurent la force exercée par un fluide ou un gaz sur une surface.

I.1.2.2. Partie commande

La partie commande est composée de plusieurs groupes. Les interfaces d'entrée transforment les informations des capteurs placés sur la partie opérative ou dans la partie dialogue en informations compatibles avec les caractéristiques technologiques du système, comme un clavier. Les interfaces de sortie convertissent les informations élaborées par l'unité de traitement en données compatibles avec les pré-actionneurs, ainsi que les dispositifs de visualisation et d'avertissement, comme un écran. L'unité de traitement, qui peut inclure des automates programmables industriels (API), des ordinateurs ou des microprocesseurs, élabore les ordres destinés aux actionneurs en fonction des informations reçues des différents capteurs et des opérations à réaliser.

I.1.2.3. La partie dialogue

La section de dialogue se compose de deux ensembles. Les visualisations et avertisseurs transforment les informations fournies par l'automate en données perceptibles par l'homme, telles que des signaux optiques ou sonores. Les capteurs, quant à eux, convertissent les informations fournies par l'homme, comme l'action manuelle sur un bouton poussoir, en données exploitables par l'automate.

Chapitre I.2 Internet des objets

I.2.1 DEFINITION

L'Internet des objets (IoT) est un réseau d'objets et de terminaux connectés équipés de capteurs (et d'autres technologies) leur permettant de transmettre et de recevoir des données entre eux et avec d'autres systèmes. [2]

I.2.2 CYCLE DE DONNEES DE L'IOT

Le cycle de données IoT se compose de plusieurs étapes. D'abord, les dispositifs IoT, équipés de divers capteurs comme ceux de température, de mouvement ou de localisation, capturent des données sur leur environnement ou leur état, comme la localisation via un module GPS. Un microcontrôleur, tel qu'un ESP32-CAM, joue un rôle crucial dans cette étape en traitant les données capturées localement avant de les transmettre. Ensuite, ces données sont partagées à travers des réseaux de communication tels que le Wi-Fi, le Bluetooth ou les réseaux cellulaires, avec le module 3G/LTE facilitant cette communication à travers un réseau cellulaire. Les données capturées sont alors traitées par des systèmes centraux ou des plateformes cloud, ce qui peut inclure des analyses en temps réel, la détection de tendances ou la fusion de données provenant de plusieurs sources (cf figure I-2).

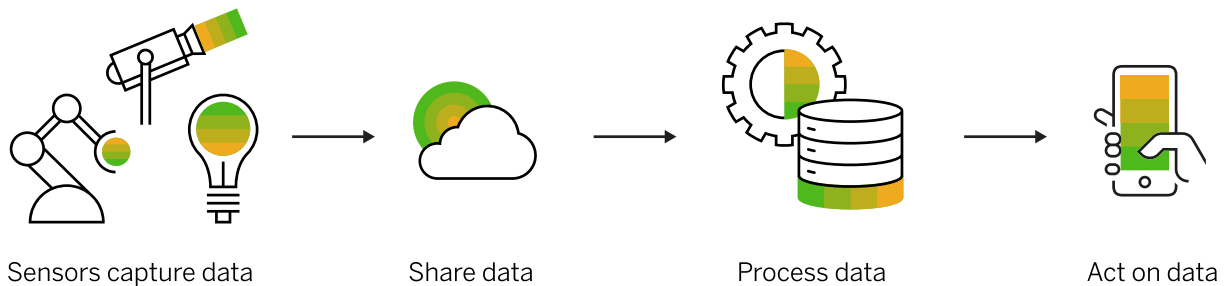


Figure I-2 : Cycle de données de l'IoT

Source : [2]

I.2.3 MICROCONTROLEUR

Un microcontrôleur est un composant électronique polyvalent qui joue un rôle central dans de nombreuses applications électroniques modernes. Il s'agit d'une puce microélectronique qui intègre divers éléments essentiels pour le traitement de l'information et le contrôle de dispositifs. Les microcontrôleurs sont programmables, ce qui signifie qu'ils peuvent être configurés pour exécuter des tâches spécifiques en fonction du code informatique qui leur est téléchargé.

I.2.3.1. Architecture des microcontrôleurs

L'architecture d'un microcontrôleur comprend plusieurs composants fondamentaux, chacun ayant un rôle spécifique. Le processeur central (CPU) est le cerveau du microcontrôleur et est chargé d'exécuter les instructions du programme, en effectuant des calculs, des opérations logiques et en gérant les données. La mémoire, généralement divisée en RAM (Random Access Memory) pour le stockage temporaire des données et en ROM (Read-Only Memory) ou Flash pour le stockage permanent du programme, est également intégrée. Les interfaces d'entrée/sortie (E/S) permettent au microcontrôleur de se connecter à des capteurs, des actionneurs et d'autres périphériques externes, facilitant la collecte de données et le contrôle des

dispositifs. Enfin, les timers et compteurs sont utilisés pour générer des retards, mesurer des intervalles de temps et réaliser des opérations basées sur le temps.

I.2.3.2. ESP32

L'ESP32 est un microcontrôleur populaire et polyvalent conçu pour les applications IoT (Internet des objets) et les systèmes embarqués. La figure I-23 ci-dessous illustre sa structure :

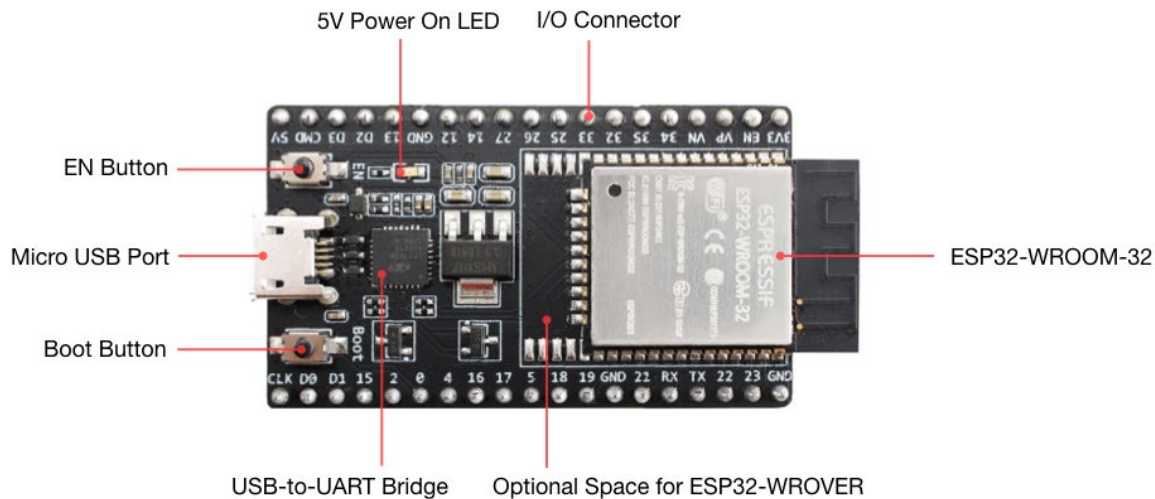


Figure I-3 : Structure d'une ESP32

Source :[3]

I.2.3.3. Architecture de l'ESP32

L'ESP32 est conçu sur l'architecture Xtensa Dual-Core, dotée de deux cœurs de processeur. Chaque cœur fonctionne à une fréquence variant de 80 MHz à 240 MHz, ce qui permet une grande puissance de traitement. L'ESP32 est équipé de 520 Ko à 4 Mo de mémoire RAM, selon le modèle, utilisée pour le stockage temporaire des données en cours d'exécution. En outre, il possède une mémoire flash ROM intégrée pour le stockage des programmes et des données permanentes, avec une capacité généralement comprise entre 4 Mo et 16 Mo selon les modèles.

I.2.3.4. Caractéristiques de l'ESP32

L'ESP32 est un microcontrôleur aux fonctionnalités polyvalentes. Il intègre des connexions sans fil Wi-Fi et Bluetooth, y compris Bluetooth Classique et Bluetooth Low Energy. Il offre une connectivité étendue avec de nombreuses interfaces telles que des ports UART, I2C, SPI, GPIO, ainsi que des convertisseurs analogique-numérique (CAN) pour communiquer avec divers périphériques. Conçu pour une faible consommation d'énergie, l'ESP32 est adapté aux applications nécessitant une alimentation par batterie. Il comprend également des fonctionnalités de sécurité telles que le chiffrement matériel et la prise en charge de TLS/SSL pour sécuriser les communications.

I.2.3.5. ESP32-CAM

L'ESP32-CAM est un module qui intègre un microcontrôleur ESP32 ainsi qu'une caméra OV2640 (2M pixels). Il est conçu en particulier pour les applications de surveillance vidéo, de capture d'images, et d'autres applications qui nécessitent une caméra. En plus, elle est équipée d'un lecteur de cartes microSD qui permet de stocker des images, des vidéos. Voici une image d'un ESP32-CAM :



Figure I-4 : ESP32-CAM

Source :[4]

I.2.3.5.1 Caractéristiques de l'ESP32-cam

L'ESP32-CAM est un module qui combine un microcontrôleur ESP32 et une caméra, offrant diverses fonctionnalités polyvalentes. Le microcontrôleur ESP32, à double cœur, fournit une puissance de calcul élevée et une connectivité sans fil via Wi-Fi et Bluetooth. Le module est équipé d'un connecteur pour appareil photo, facilitant la connexion d'une caméra pour capturer des images ou des vidéos. Certains modèles d'ESP32-CAM intègrent également un microphone pour l'audio. Il dispose de plusieurs broches GPIO (General Purpose Input/Output) pour connecter divers composants et périphériques, ainsi qu'un emplacement pour carte microSD, permettant de stocker localement des images ou des vidéos.

I.2.3.5.2 Brochage de l'ESP32-CAM

Le brochage de l'ESP32-CAM comprend plusieurs éléments clés. Pour l'alimentation, il y a une broche 5V pour la tension d'alimentation et une broche GND pour la masse. Le port USB est représenté par les broches UOT et UOR, qui assurent la communication USB vers UART. Les broches GPIO0 et GPIO2 sont utilisées pour contrôler la communication Wi-Fi et Bluetooth. L'interface GPIO générale offre plusieurs broches pour connecter d'autres périphériques et composants. Enfin, l'interface microSD est composée des broches SD_CLK, SD_CMD, SD_DATA0, SD_DATA1, SD_DATA2, et SD_DATA3, qui sont dédiées à la connexion de la carte microSD. La figure I-25 illustre le brochage de l'ESP32-CAM.

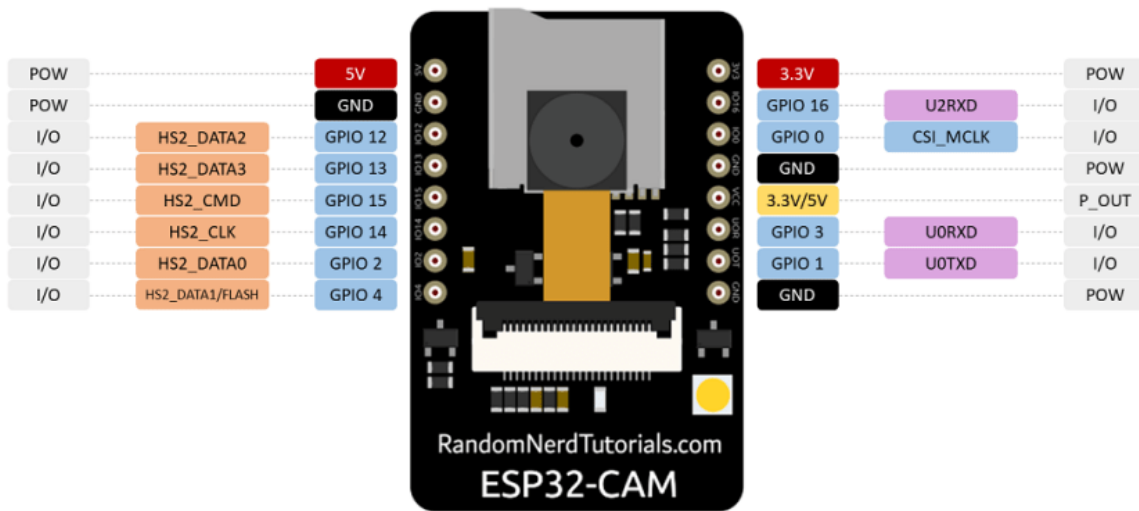


Figure I-5 : Brochage ESP32-cam

Source : [5]

I.2.4 MODULE GPS

Les modules GPS (Global Positioning System) sont des composants électroniques de petite taille qui permettent à un appareil de déterminer sa position géographique avec une grande précision. Ils sont largement utilisés dans une multitude d'applications, des smartphones aux véhicules en passant par les objets connectés.

I.2.4.1. Différents types de module GPS

Il existe une grande variété de modules GPS, chacun ayant ses propres caractéristiques et applications :

- Modules autonomes intègrent toutes les fonctionnalités nécessaires au positionnement, y compris l'antenne GPS. Ils sont faciles à intégrer dans d'autres appareils.



Figure I-6 : GPS Autonomes

Source : [6]

- Modules intégrés sont conçus pour être soudés directement sur une carte électronique. Ils sont souvent utilisés dans les applications où l'espace est limité.

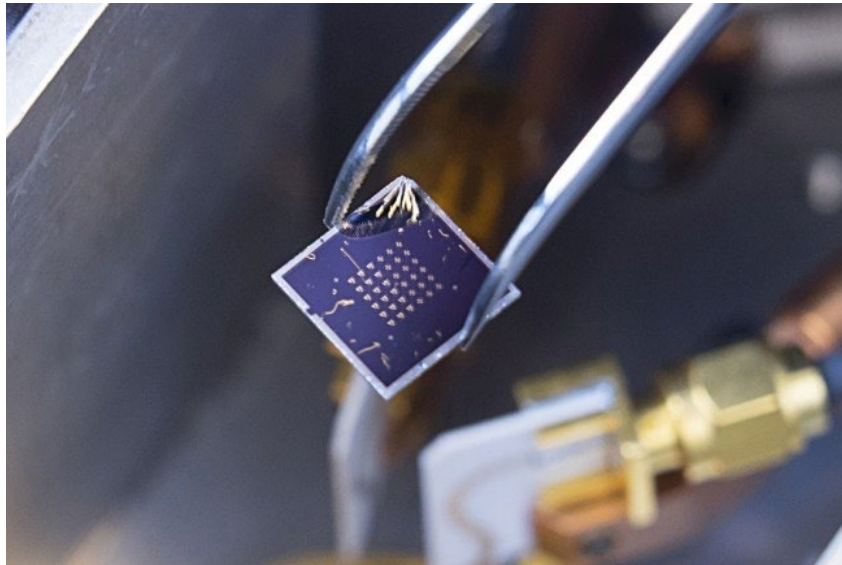


Figure I-7 : Puce intégrant un GPS

Source : [7]

- Modules haute précision offrent une précision de positionnement extrêmement élevée, de l'ordre du centimètre. Ils sont utilisés dans des applications professionnelles comme la géodésie ou l'agriculture de précision.

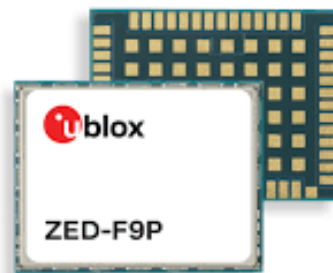


Figure I-8 : ZED-F9P GPS

Source : [8]

- Modules à faible consommation sont optimisés pour une consommation d'énergie minimale. Ils sont idéaux pour les applications alimentées par batterie, comme les trackers GPS pour animaux.



Figure I-9 : LEA-6 series GPS

Source : [9]

I.2.5 MODULE GSM/3G/LTE

Les modules GSM, 3G et LTE sont des composants électroniques qui permettent à un appareil de se connecter à un réseau mobile et ainsi d'échanger des données.

I.2.5.1. Module GSM

GSM (Global System for Mobile communications) est la première génération de téléphonie mobile numérique. Les modules GSM permettent la transmission de voix et de données à faible débit.



Figure I-10 : Module GSM

Source : [10]

I.2.5.2. Module 3G

3G (Third Generation) offre des débits plus élevés que le GSM, permettant ainsi des applications plus riches comme la vidéo en streaming ou le téléchargement de fichiers volumineux.

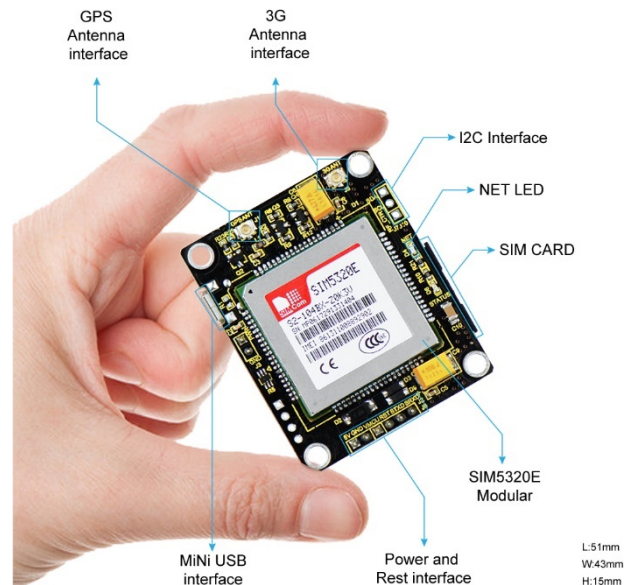


Figure I-11 : Module 3G

Source :[11]

I.2.5.3. Module 4G

LTE (Long Term Evolution) est la technologie 4G, qui offre des débits encore plus élevés et une latence réduite, idéale pour les applications nécessitant une grande bande passante comme le streaming vidéo en haute définition ou les jeux en ligne.



Figure I-12 : Module LTE/4G

Source : [12]

I.2.6 DOMAINE D'APPLICATION

Les réseaux IoT et les données qu'ils génèrent sont utilisés dans divers domaines modernes :

I.2.6.1. Maisons intelligentes

Les systèmes domestiques utilisent des capteurs et des dispositifs intelligents pour contrôler des éléments tels que l'éclairage, la température, et les systèmes de sécurité. Ces dispositifs peuvent être gérés à distance via des applications, permettant aux utilisateurs de contrôler leur maison même lorsqu'ils ne sont pas présents.[2]

I.2.6.2. Voitures connectées

De nombreux véhicules modernes sont équipés de technologies IoT qui facilitent la conduite, la gestion des itinéraires et la sécurité. En plus des fonctionnalités avancées telles que la prévention des collisions et la planification des itinéraires, les véhicules connectés peuvent également bénéficier de la maintenance prédictive et du suivi en temps réel de leur état. Les capteurs embarqués surveillent les performances et l'état des composants du véhicule, permettant de détecter les problèmes potentiels avant qu'ils ne deviennent critiques. Les données collectées sont analysées pour prévoir les besoins de maintenance, réduire les pannes imprévues et améliorer la fiabilité globale du véhicule. [2]

I.2.6.3. Gestion de trafic

Les capteurs et les caméras connectés contribuent à fluidifier le trafic en ajustant les signaux de circulation en temps réel, en optimisant l'éclairage routier et en détectant les accidents pour améliorer la sécurité et l'efficacité du réseau routier.

Chapitre I.3 Technologie numérique

Les technologies numériques constituent un domaine vaste et multidisciplinaire qui englobe l'étude, la conception, le développement, l'exploitation et l'utilisation des systèmes informatiques, des logiciels et du matériel pour le traitement, le stockage, la transmission et la manipulation de l'information de manière efficace et automatisée. Elles sont étroitement liées à la gestion des données, à la résolution de problèmes, à la communication et à la prise de décision à l'aide des ordinateurs et des technologies de l'information et de la communication (TIC).

I.3.1 PROTOCOLE HTTP

I.3.1.1. Généralité [17]

Le protocole HTTP (HyperText Transfer Protocol) est un protocole de communication client-serveur développé pour le Web.

HTTP permet à un client et un serveur HTTP d'échanger des représentations de ressources (documents, images, sons, résultats de requêtes SQL, etc.).

Un protocole est l'ensemble des règles qui gouvernent une conversation entre un client et un serveur.

I.3.1.2. Client-Serveur

Un client HTTP, souvent un navigateur web comme Firefox ou Chrome, établit une connexion avec un serveur HTTP et envoie une ou plusieurs requêtes HTTP. En réponse, le serveur HTTP accepte les demandes de connexion des clients et répond à chaque requête HTTP avec une réponse appropriée.

I.3.2 PROGRAMMATION

La programmation est l'art de créer des instructions ou des algorithmes informatiques en utilisant des langages de programmation spécifiques. Les programmeurs écrivent des lignes de code qui définissent comment un ordinateur doit effectuer des tâches, résoudre des problèmes ou traiter des données.

I.3.2.1. IDE

Un environnement de développement intégré, ou IDE, est un logiciel de création d'applications, qui rassemble des outils de développement fréquemment utilisés dans une seule interface utilisateur graphique (GUI).[13]

Un IDE (Environnement de Développement Intégré) se compose généralement de plusieurs éléments essentiels. L'éditeur de code source est un éditeur de texte spécialisé qui facilite la rédaction du code logiciel, offrant des fonctionnalités telles que la coloration syntaxique, la saisie automatique en fonction du langage et la vérification de bogues en temps réel. Les utilitaires d'automatisation de version locale permettent d'automatiser des tâches telles que la compilation du code source en code binaire, la mise en paquet de ce code binaire, et l'exécution de tests automatisés, rendant le processus de développement plus fluide. Enfin, le débogueur est un programme conçu pour tester d'autres programmes en affichant les emplacements des bogues dans le code d'origine, ce qui aide les développeurs à identifier et corriger les erreurs.

I.3.2.1.1 *Arduino IDE :*

L'Arduino IDE appartient au type d'IDE de développement embarqué. Il est spécifiquement conçu pour la programmation des microcontrôleurs Arduino. A l'ouverture, l'interface visuelle du logiciel ressemble à ceci : Des icônes de commande en haut, une page pour écrire le programme, une bande en bas.



Figure I-13 : Interface Arduino IDE

Source : Auteur

L'IDE Arduino permet d'éditer des programmes, appelés croquis, qui sont écrits en langage C. Une fois le programme écrit, il est compilé pour le convertir en langage machine que l'ESP32-cam peut comprendre. La console de l'IDE affiche des informations sur le processus de compilation ainsi que les messages d'erreur éventuels. Après la compilation, le programme est téléversé dans la mémoire de l'ESP32-cam via le port USB de l'ordinateur. Ce processus, appelé téléversement ou upload, permet de transférer le programme dans la mémoire de l'ESP32-cam, où il devient un microgiciel. La console continue de fournir des informations sur le déroulement du téléversement et les erreurs potentielles. En outre, il est possible de communiquer avec l'ESP32-cam à l'aide du terminal ou moniteur série, permettant une interaction en temps réel tant que la connexion via câble USB est maintenue.

1.3.2.1.2 Visual Studio Code :

Visual Studio Code, est un IDE léger et extensible qui prend en charge une large gamme de langages de programmation, notamment JavaScript, TypeScript, Python, C++, Java, Go, PHP, Ruby, et plus encore, gratuit développé par Microsoft.

Cette image montre l'interface de l'IDE :

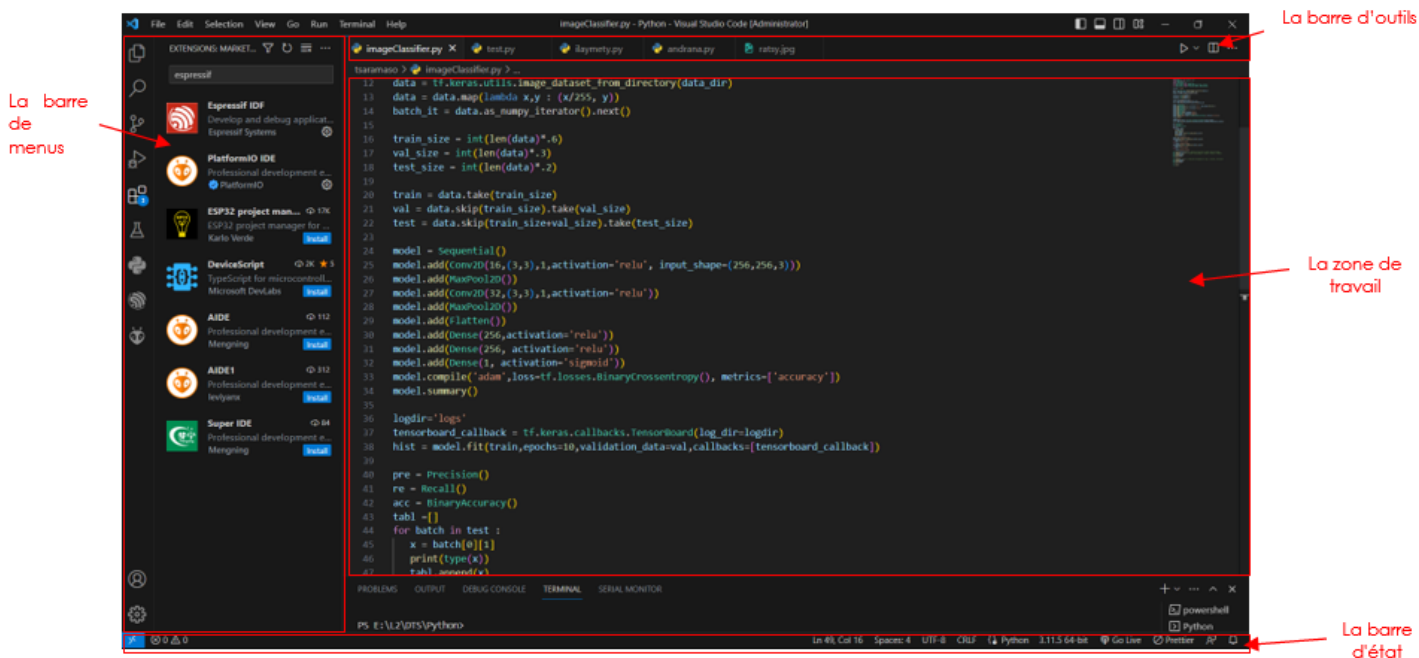


Figure I-14 : Interface Visual Studio Code

Source : Auteur

Dans l'interface de Visual Studio Code, on distingue plusieurs zones fonctionnelles. La barre de menus permet d'accéder aux diverses fonctionnalités de l'éditeur. La barre d'outils contient des boutons pour des actions courantes, telles que l'ouverture de fichiers, l'exécution de programmes ou la recherche dans le code. La zone de travail affiche le code source du fichier ou du dossier actuellement actif. Enfin, la barre d'état fournit des informations sur l'état de l'éditeur, incluant le nombre de lignes et de colonnes du fichier, le langage de programmation utilisé, et la position du curseur.

I.3.3 DEVELOPPEMENT WEB

Le développement web est l'art et la science de créer des sites web et des applications web. Il s'agit de construire des interfaces numériques qui permettent aux utilisateurs d'interagir avec des données et des services en ligne. Pour comprendre ce processus, il est essentiel de distinguer deux éléments clés : le frontend et le backend.[14]

I.3.3.1. Le frontend : l'interface visible de l'utilisateur

Le frontend, ou face avant, est la partie d'une application web que l'utilisateur voit et avec laquelle il interagit directement. C'est l'interface graphique qui donne vie au site web.

Les technologies utilisées pour le frontend sont principalement :

- **HTML (HyperText Markup Language):** C'est le langage de base pour structurer le contenu d'une page web. Il définit les éléments tels que les titres, les paragraphes, les images, les liens, etc.



Figure I-15 : Logo de l'HTML

Source : [15]

- **CSS (Cascading Style Sheets):** Ce langage permet de styliser les éléments HTML, en définissant leur apparence (couleurs, polices, mise en page, etc.).



Figure I-16 : Logo du CSS

Source :[16]

- **JavaScript:** C'est un langage de programmation qui ajoute de l'interactivité aux pages web. Il permet de créer des effets dynamiques, de manipuler le DOM (Document Object Model).



Figure I-17 : Logo du JavaScript

Source : [17]

Les frameworks frontend populaires sont :

- **React:** Un framework JavaScript créé par Facebook, utilisé pour construire des interfaces utilisateur complexes et performantes.

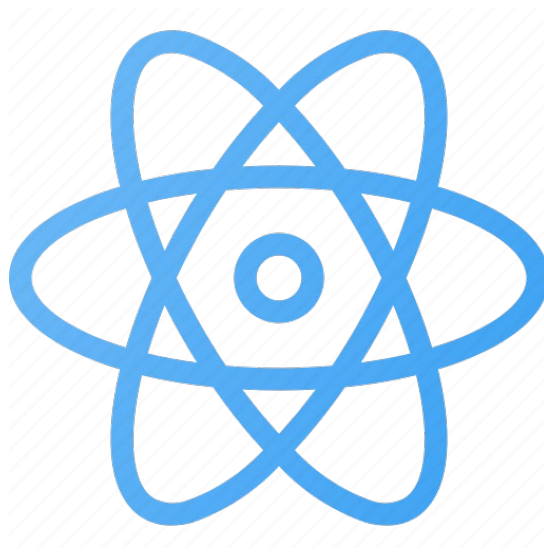


Figure I-18 : Logo du React

Source : [18]

- **Angular:** Un framework complet créé par Google, offrant une structure robuste pour les applications web.



Figure I-19 : Logo d'Angular

Source : [19]

- **Vue.js:** Un framework progressif, flexible et facile à apprendre, idéal pour les projets de toutes tailles.



Figure I-20 : Logo du Vue.js

Source :[20]

I.3.3.2. Le backend

Le backend, ou face arrière, est la partie invisible de l'application web. Il s'occupe de la logique métier, du traitement des données et de la communication avec la base de données.

Les technologies utilisées pour le backend sont :

- Langages de programmation : Node.js (JavaScript), Python (Django, Flask), Ruby on Rails, PHP (Laravel, Symfony), Java (Spring), etc. Ces langages permettent de créer les fonctionnalités de l'application.
- Serveurs web : Nginx, Apache, etc. Ils sont responsables de la gestion des requêtes HTTP et de la réponse aux clients.
- Bases de données : MySQL, PostgreSQL, MongoDB, NoSQL, etc. Elles stockent les données de l'application de manière organisée et permettent de les récupérer rapidement.

I.3.3.3. Cycle de vie d'une requête web :

Ci-a le cycle de vie d'une requête web en général :

- 1- Le client (navigateur) envoie une requête au serveur.
- 2- Le serveur reçoit la requête et la transmet au backend.
- 3- Le backend traite la requête, accède à la base de données si nécessaire, et prépare une réponse.
- 4- Le backend envoie la réponse au serveur.
- 5- Le serveur renvoie la réponse au client.
- 6- Le navigateur affiche le contenu de la réponse dans le frontend.

I.3.3.3.1 Exemple architectural d'un site web :

Cette image illustre l'architecture d'une application web typique en utilisant les technologies Apache, PHP et MySQL.

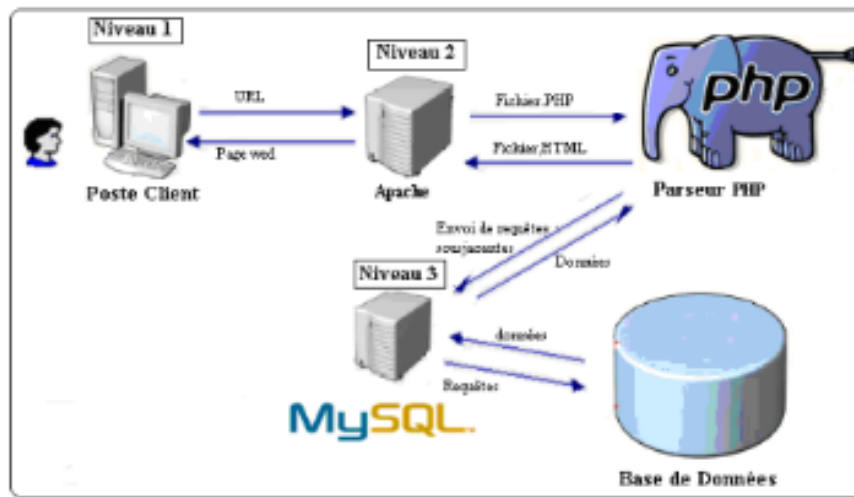


Figure I-21 : Architecture d'une application web

Source : [21]

Voici une explication de chaque niveau :

1- Niveau 1 : Poste Client

L'utilisateur interagit avec l'application web via un navigateur en saisissant une URL pour accéder à une page web. Le navigateur envoie alors une requête au serveur web pour récupérer cette page.

2- Niveau 2 : Serveur Apache

Le serveur web Apache reçoit la requête de l'utilisateur et envoie les fichiers PHP au parseur PHP tout en transmettant les fichiers HTML directement au client. Le parseur PHP exécute le code PHP contenu dans les fichiers, générant ainsi du HTML dynamique. De plus, le code PHP peut envoyer des requêtes SQL à la base de données pour récupérer ou manipuler des données.

3- Niveau 3 : Base de Données

Le serveur de base de données MySQL stocke les données de l'application et répond aux requêtes SQL envoyées par le code PHP en fournissant les données demandées. Les requêtes peuvent inclure des commandes pour récupérer, insérer, mettre à jour ou supprimer des données.

I.3.4 LOGICIEL XAMPP

XAMPP est un acronyme qui signifie Cross-Platform (X), Apache, MySQL, PHP et Perl. Il s'agit d'un ensemble de logiciels libres qui permet de créer un environnement de développement local complet sur un ordinateur.

I.3.4.1. Rôle de XAAMP

XAMPP est utilisé principalement pour le développement web, offrant un environnement isolé permettant aux développeurs de créer et tester des applications sans impacter d'autres applications sur l'ordinateur. Il est également un excellent outil d'apprentissage pour les langages de programmation web comme PHP et Perl, ainsi

que pour se familiariser avec les bases de données MySQL. De plus, XAMPP facilite le prototypage rapide d'applications web, permettant de tester et affiner des idées rapidement.

I.3.4.2. Les composants de XAMPP

Apache est un serveur web open source très populaire, chargé de servir les pages web depuis un ordinateur. MySQL est un système de gestion de bases de données relationnelles utilisé pour stocker et gérer les données des applications web. PHP est un langage de programmation côté serveur, largement employé pour développer des applications web dynamiques. Perl est un langage de programmation interprété, souvent utilisé pour des tâches d'administration système et de traitement de texte. MariaDB est une alternative à MySQL, également incluse dans certaines distributions de XAMPP.

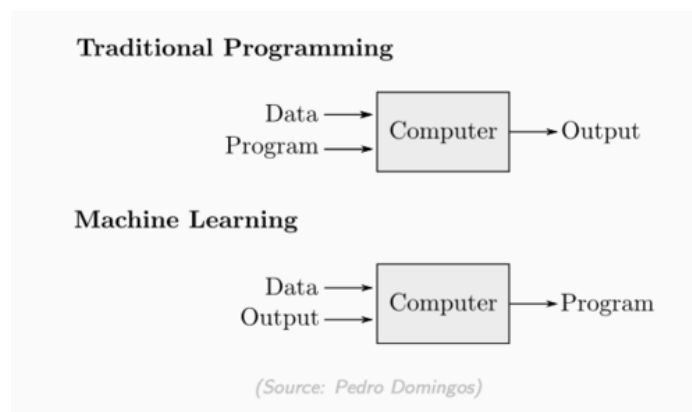


Figure I-22 : Tableau de bord du logiciel XAMPP

Source : Auteur

I.3.5 MACHINE LEARNING

Le Machine Learning est un domaine de l'intelligence artificielle qui consiste à programmer une machine pour que celle-ci apprenne à réaliser des tâches en étudiant des exemples (données) de ces dernières. La figure I-27 illustre la différence entre la Machine Learning et la programmation traditionnelle :



Source : [22]

La machine utilise ces données pour trouver les paramètres d'une technique choisie afin de résoudre un problème spécifique. Par exemple, un modèle d'expression mathématique $f(x) = ax + b$, qui est la technique de régression linéaire est couramment utilisé pour résoudre un problème de régression linéaire, c'est-à-dire pour estimer la valeur dépendante Y à partir des valeurs indépendantes X.

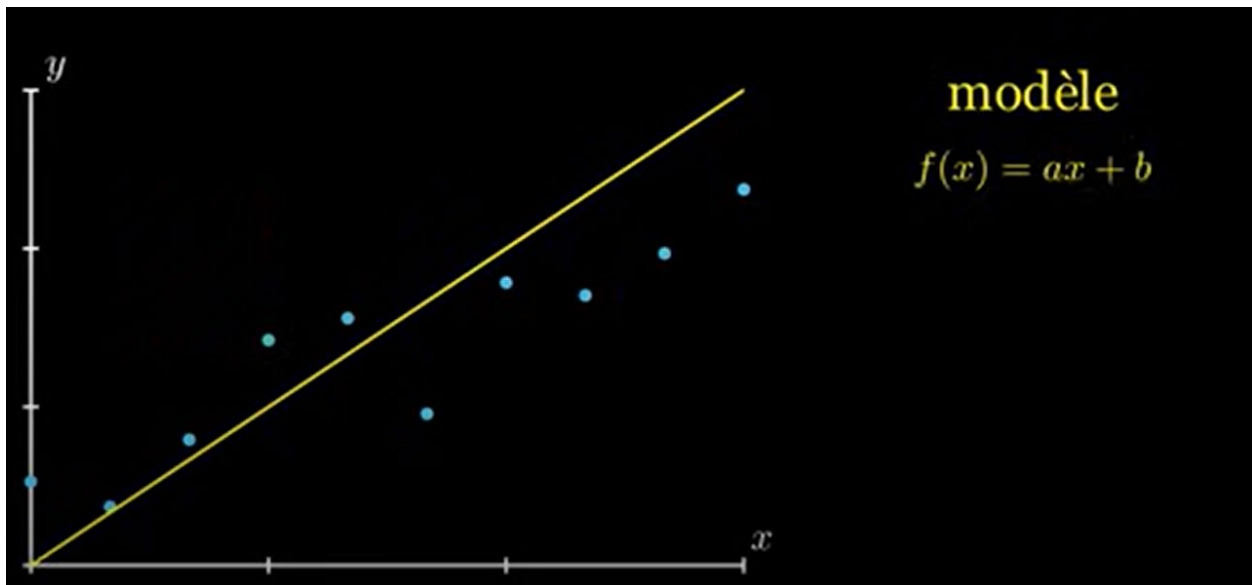


Figure I-24 : Exemple de Modèle Machine Learning

Source : [23]

Le but du Machine Learning est de trouver les paramètres, a et b dans notre cas, de manière à obtenir le meilleur modèle possible, c'est-à-dire le modèle qui s'ajuste le mieux aux données.

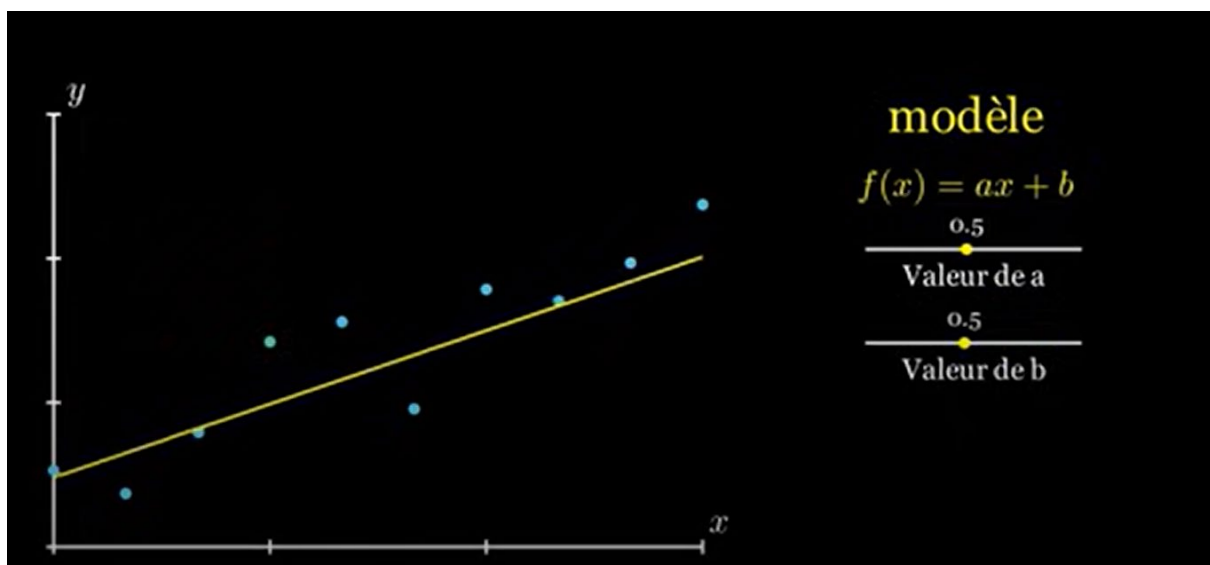


Figure I-25 : Optimisation du modèle

Source : [23]

Pour parvenir à cela, on programme **un algorithme d'optimisation** dans la machine. Cet algorithme teste différentes combinaisons de valeurs pour 'a' et 'b' jusqu'à trouver celle qui minimise la distance entre le modèle et les points de données observés.

Le Machine Learning consiste à élaborer un modèle en utilisant un algorithme d'optimisation pour minimiser les erreurs entre le modèle et les données. L'erreur, généralement appelée '**Loss**' est calculée en fonction de la différence entre la sortie prédite par le modèle et la valeur réelle de sortie.

Ainsi le modèle entraîné, pour prédire la valeur Y d'un point, il suffit de fournir la valeur X en entrée du modèle.

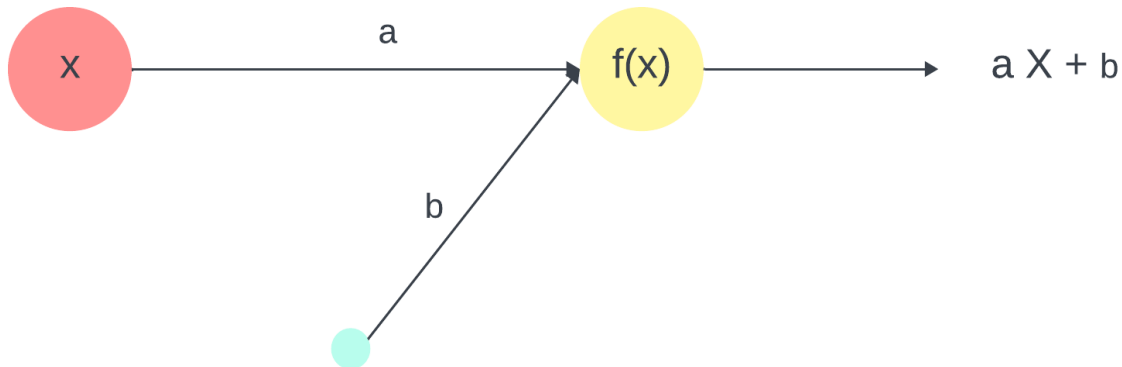


Figure I-26 : Représentation du modèle de régression linéaire

Source : Auteur

I.3.5.1. Catégories d'apprentissage

Le Machine Learning se divise en quatre catégories principales d'apprentissage. L'apprentissage supervisé utilise des données étiquetées pour entraîner des modèles. L'apprentissage non supervisé traite des données non étiquetées pour découvrir des structures sous-jacentes. L'apprentissage par renforcement enseigne à un agent à maximiser les récompenses en prenant des décisions. L'apprentissage profond, ou Deep Learning, utilise des réseaux de neurones pour traiter des données complexes.

Les techniques du Machine Learning sont nombreuses, et elles ont leurs modes d'apprentissage qui leur conviennent le mieux (voir tableau I-1).

Le tableau I-1 présente différentes catégories d'apprentissage machine, les scénarios et techniques associées, ainsi que les méthodes d'optimisation utilisées. Par exemple, l'apprentissage supervisé utilise des modèles comme la régression linéaire et les arbres de décision, optimisés par descente de gradient et Adam.

Catégorie	Scénario	Techniques(modèles)	Optimisation
Apprentissage supervisé	Données étiquetées	Régression linéaire, régression logistique, arbres de décision, Forêt aléatoire	Descente de gradient, Adam
Apprentissage non supervisé	Données non étiquetées	K-means, Analyse en Composantes Principales (ACP)	Descente de gradient, K-means++

Apprentissage semi-supervisé	Données étiquetées et non étiquetées	Label Propagation, Self-training	Sélection de seuil
Apprentissage par renforcement	Apprendre à un agent à prendre des décisions en maximisant les récompenses	Q-learning, Deep Q-Networks, Policy Gradient	Méthode de Monte-carlo, Proxiaml Policy Optimization
Apprentissage Profond ou Deep Learning	Données complexes pour extraire des caractéristiques hautement abstraites	Réseaux de neurones profonds	Descente de gradient, Descente de gradient par Adam, Backpropagation

Tableau I-1 : Les techniques du Machine Learning avec leurs algorithmes d'optimisation selon la catégorie d'apprentissage

Source : [25], [27], [28], [29]

I.3.5.2. Processus générique de machine learning

Le processus générique de machine learning comprend plusieurs étapes. Il commence par le chargement des données, suivi du prétraitement pour nettoyer et préparer les données. Ensuite, les données sont séparées en ensembles d'apprentissage et de test. Un modèle est ensuite créé et entraîné sur l'ensemble de données d'apprentissage. Après l'entraînement, le modèle est évalué pour mesurer sa performance. Enfin, le modèle est utilisé pour effectuer des prédictions sur de nouvelles données.

I.3.5.3. Forêt aléatoire

Le forêt aléatoire ou Radom forest est une méthode d'apprentissage automatique utilisé pour la classification.

Il s'agit d'un ensemble d'arbres de décision qui fonctionne ensemble pour améliorer les performances et la robustesse par rapport à un seul arbre de décision.

I.3.5.3.1 Constituant de la forêt aléatoire

Une forêt aléatoire est un ensemble (ou "forêt") de plusieurs arbres de décision. La méthode combine les résultats de tous ces arbres pour prendre une décision finale.

Un arbre de décision est un modèle qui divise les données en fonction des valeurs de certaines caractéristiques pour prédire la classe d'une instance. Chaque nœud de l'arbre représente une question sur une caractéristique, et chaque branche représente les réponses possibles. Les feuilles de l'arbre correspondent aux classes de sortie.

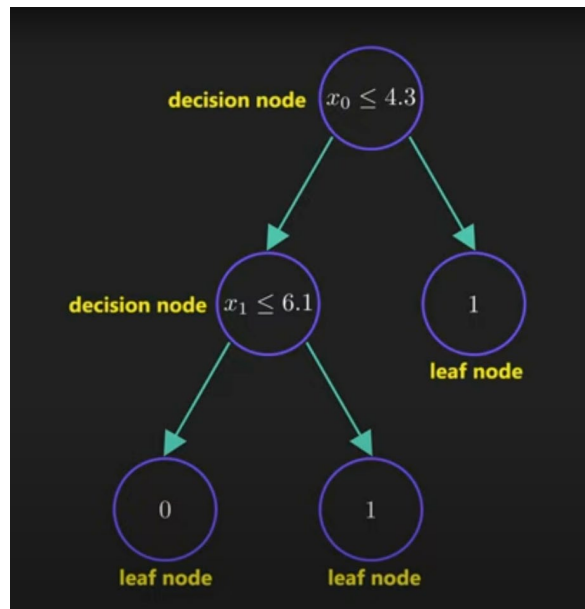


Figure I-27 : Arbre de décision

Source : [30]

1.3.5.3.2 Code générique pour le modèle forêt aléatoire :

Le code générique commence par utiliser un ensemble de données pour former un modèle, en divisant ces données en deux groupes, l'un pour entraîner le modèle et l'autre pour tester sa performance. Un modèle est ensuite créé et apprend à partir du groupe de données d'entraînement, assimilant les caractéristiques pour faire des prédictions. Une fois l'entraînement terminé, le modèle est testé avec le groupe de données réservé et fait des prédictions sur des données nouvelles. Enfin, le code évalue la précision du modèle en comparant les prédictions aux résultats corrects et affiche un pourcentage indiquant la fréquence des prédictions correctes par rapport au nombre total de prédictions.

```

1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.model_selection import train_test_split
3 from sklearn.datasets import load_iris
4 from sklearn.metrics import accuracy_score
5
6 # Charger un jeu de données exemple
7 data = load_iris()
8 X = data.data
9 y = data.target
10
11 # Diviser les données en ensembles d'entraînement et de test
12 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
13
14 # Créer et entraîner le modèle
15 model = RandomForestClassifier(n_estimators=100, random_state=42)
16 model.fit(X_train, y_train)
17
18 # Prédiction
19 y_pred = model.predict(X_test)
20
21 # Évaluer les performances
22 accuracy = accuracy_score(y_test, y_pred)
23 print(f'Accuracy: {accuracy:.2f}')
24

```

Figure I-28 : Code générique Forêt aléatoire

Source : [31]

I.3.5.3.3 *Evaluation de la performance du modèle*

Pour évaluer les performances d'un modèle de classification, il est nécessaire d'utiliser la mesure de précision et le rappel.

- **Précision :**

La précision mesure la proportion des prédictions positives qui sont effectivement correctes.

$$\text{précision} = \frac{\text{vrais positifs}}{\text{vrais positifs} + \text{faux positifs}}$$

- **Rappel (recall)**

Le rappel mesure la proportion des instances positives qui ont été correctement identifiées par le modèle.

$$\text{rappel} = \frac{\text{vrais positifs}}{\text{vrais positifs} + \text{faux négatifs}}$$

La différence clé entre les deux mesures réside dans leur focus. La précision se concentre sur la qualité des prédictions positives, c'est-à-dire le pourcentage de prédictions positives qui sont correctes. Le rappel, quant à lui, évalue la quantité de positives détectées, en mesurant combien des instances positives réelles ont été correctement identifiées.

Chapitre I.4 Cadrage des travaux

I.4.1 METHODOLOGIE DE L'ANALYSE FONCTIONNELLE

I.4.1.1. Diagramme Bête à corne

Le diagramme Bête à corne permet d'exprimer et de valider le besoin, il permet d'aboutir à une expression des fonctions remplies dans un environnement donné. C'est L'analyse fonctionnelle externe est la première approche fonctionnelle d'un produit.

Expression du BESOIN : Diagramme « bête à cornes » :

Pour définir le besoin éprouvé par l'utilisateur pour un produit, il faut répondre à trois (3) questions, celles-ci étant généralement regroupée dans un graphique appelé familièrement « bête à corne » (ou diagramme APTE) :

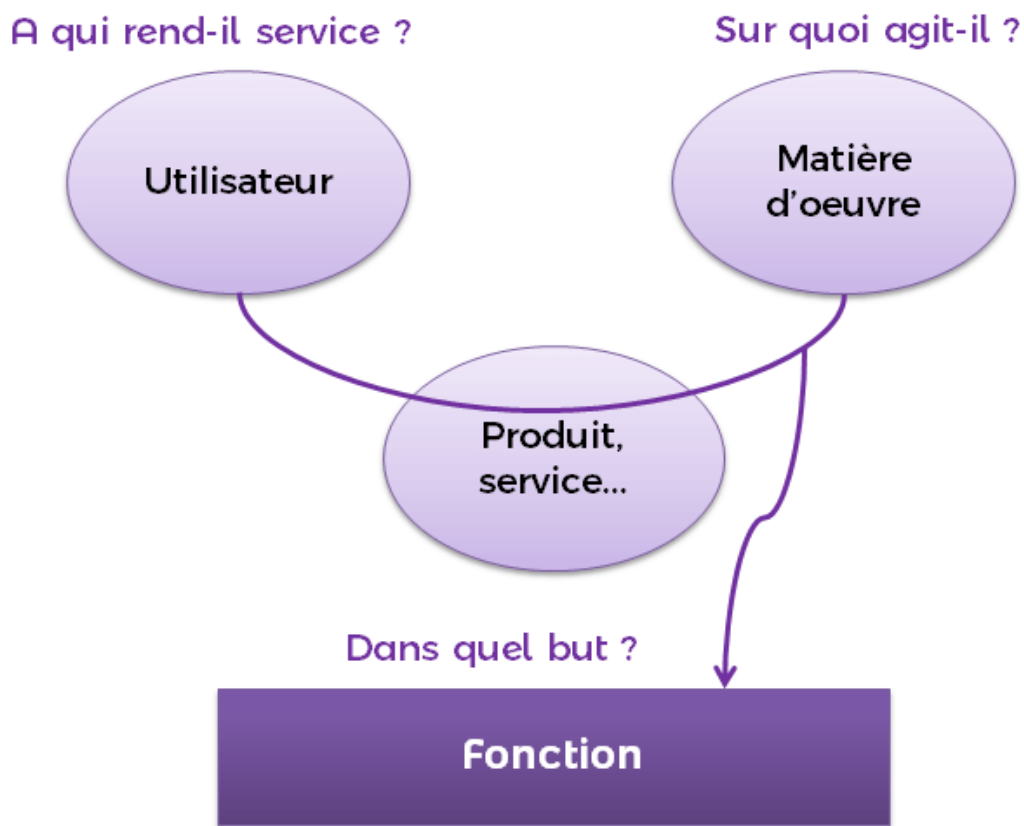


Figure I-29 : Diagramme de bête à corne

Source : [32]

I.4.1.2. Diagramme FAST :

Le diagramme FAST, ou Technique du Système d'Analyse Fonctionnelle, est un schéma construit de gauche à droite. Il illustre les relations logiques entre différentes fonctions afin de répondre aux questions de type comment et pourquoi.[33]

Il a pour objectif de décomposer logiquement une fonction identifiée pour aboutir aux solutions technologiques.

Le principe de représentation graphique est le suivant :

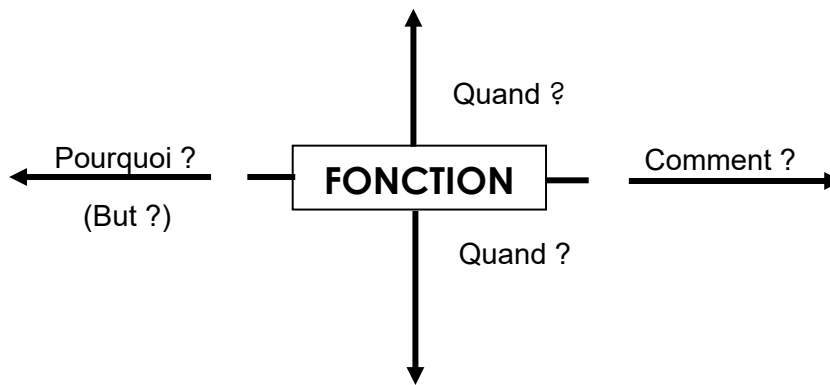


Figure I-30 : Principe du diagramme FAST

La représentation du diagramme FAST d'un aspirateur est la suivante :

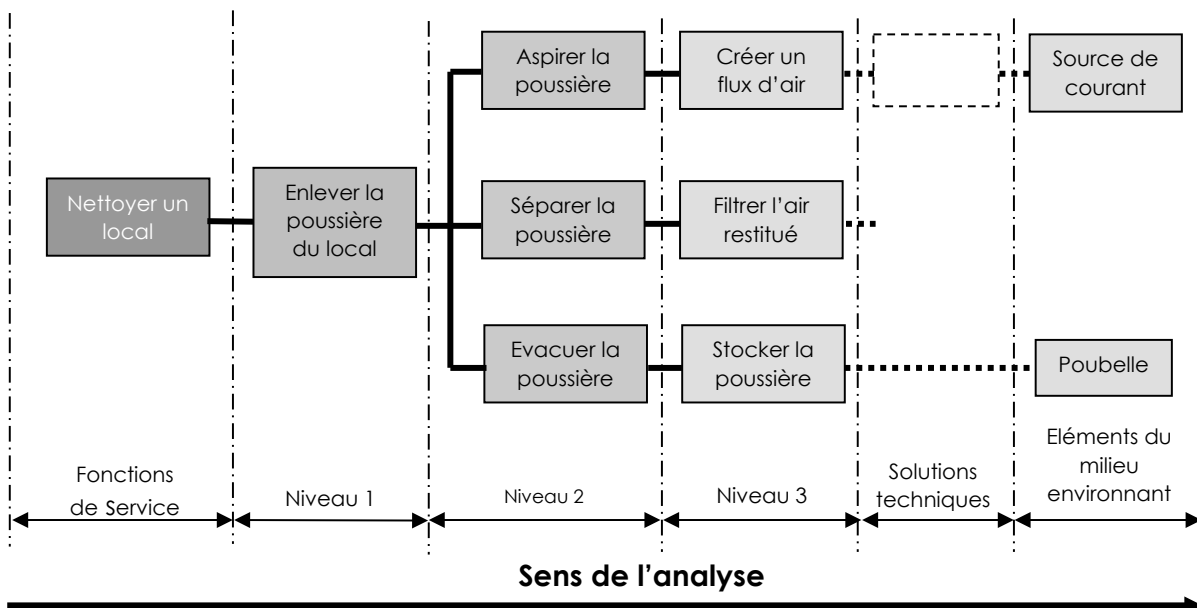


Figure I-31 : Diagramme FAST d'un aspirateur

I.4.2 ANALYSE FONCTIONNELLE DU SYSTEME

Ce chapitre consiste à identifier les besoins et à chercher des solutions pour satisfaire les besoins identifiés.

I.4.2.1. Le Diagramme de Bête à Corne du système

Le diagramme de bête à corne permet d'exprimer et de valider le besoin de notre produit.

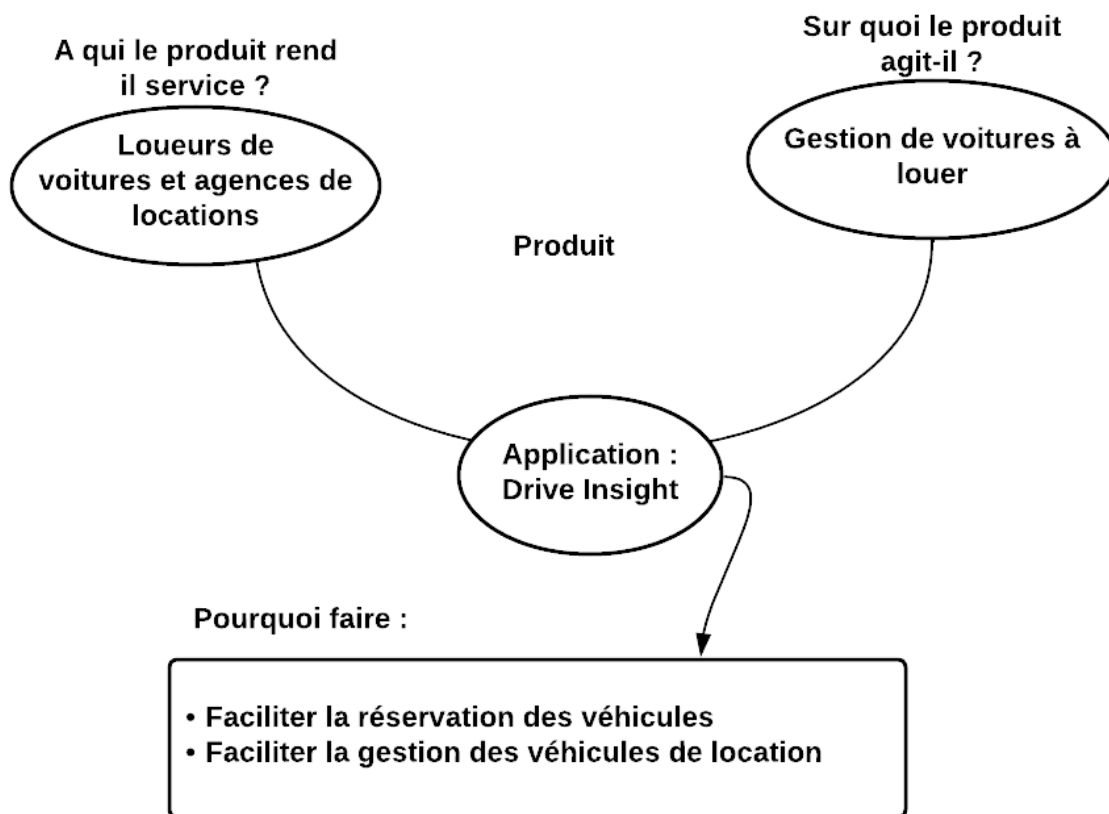


Figure I-32 : Le diagramme de bête à corne du système

Source : Auteur

A qui le produit rend-il service ?

Les principaux utilisateurs de l'application web « Drive Insight » sont les loueurs de voitures et les agences de location.

Sur quoi le produit agit-il ?

L'application web va agir sur la gestion des véhicules à louer.

Les fonctions de notre produit :

- Fp1 : Faciliter la réservation du véhicule
- Fp2 : Faciliter la gestion des véhicules de location
- Fc1 : Collectionner les données des véhicules
- Fc2 : Archiver les données des locations passées
- Fc3 : Conserver l'historique des maintenances des véhicules

I.4.2.2. DIAGRAMME FAST DU système :

La figure II-2 est une illustration du diagramme du FAST qui décompose les deux fonctions principales en sous fonctions et ses solutions technologiques

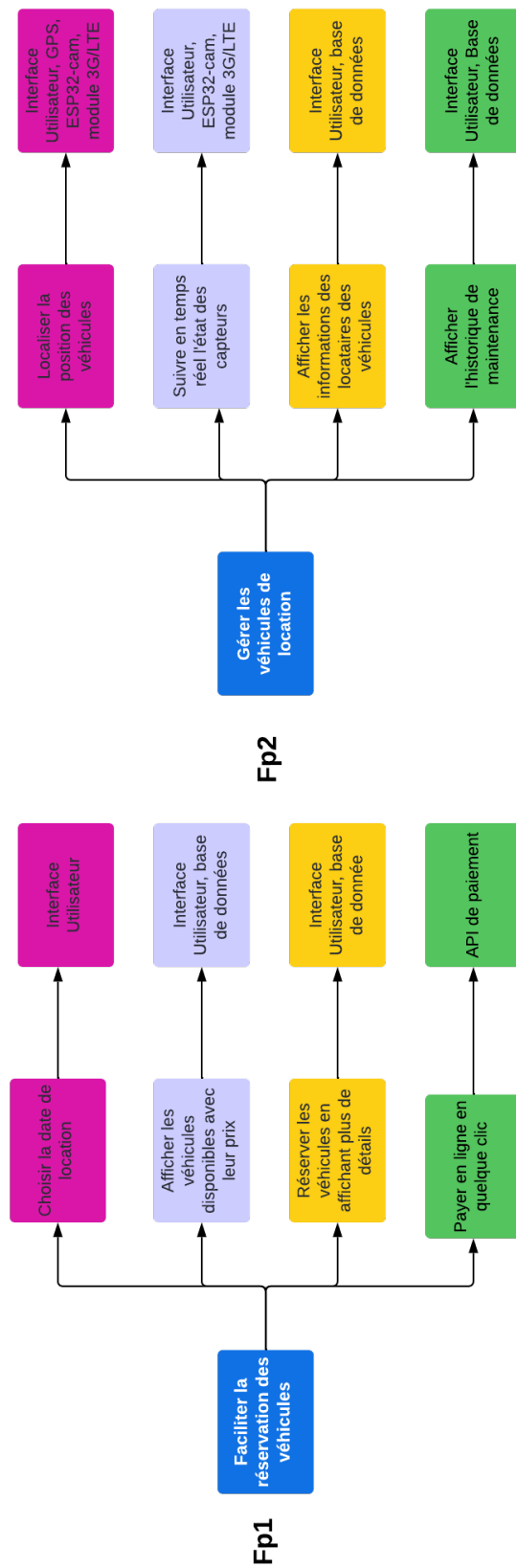


Figure I-33 : Diagramme FAST du Système

Source : Auteur

PARTIE II RESULTATS ET DISCUSSIONS

Chapitre II.1 Principe de fonctionnement du système

II.1.1 FONCTIONNEMENT EN BLOC DU SYSTEME

Le schéma bloc de la figure II-1 illustre l'architecture globale du système. Le premier bloc représente le système IoT avec les véhicules, équipés de capteurs et de GPS. Le GPS fournit des informations de localisation, tandis que les capteurs recueillent des données sur l'état des véhicules. Ces informations sont ensuite transmises à la carte ESP32-CAM, qui intègre les données du GPS et des capteurs. Le module de communication 3G/LTE envoie ensuite ces données au serveur central, où elles sont stockées et traitées. Le second bloc comprend l'application web, qui est constituée du serveur et de l'interface utilisateur, permettant aux utilisateurs de consulter les données en temps réel pour gérer et réserver les véhicules.

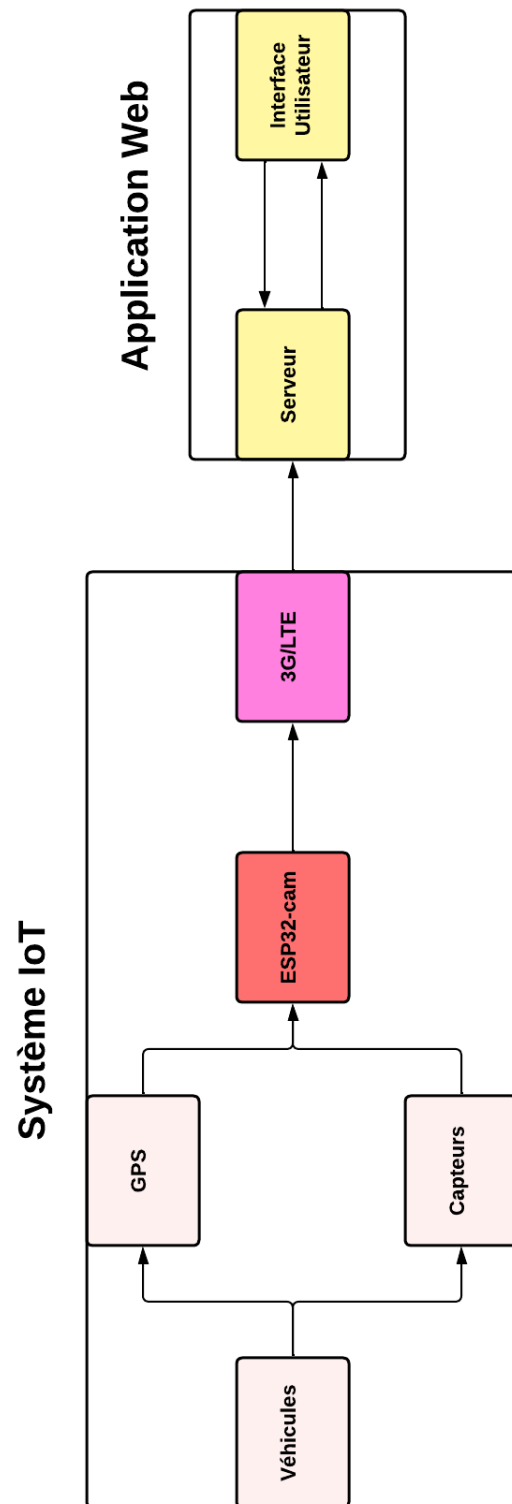


Figure II-1 : Schéma bloc du système

Source : Auteur

II.1.2 ARCHITECTURE DU SYSTEME

Le système se divise en deux parties principales. La partie commande comprend un microcontrôleur ESP32-CAM, chargé de la gestion des données, ainsi qu'un ordinateur portable utilisé pour le suivi du système. La partie opérative est constituée de capteurs qui surveillent les véhicules, d'un module GPS qui fournit les informations de localisation, et d'un module 3G/LTE qui transmet les données (voir figure II-2).

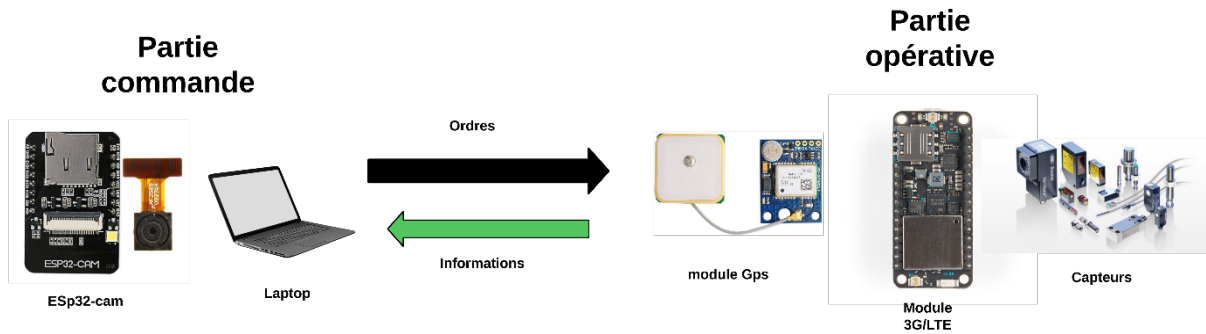


Figure II-2 : Architecture du Système

Source : Auteur

II.1.2.1. Rôles et fonctions des composants :

Chaque composant du système joue un rôle spécifique dans son fonctionnement. Les capteurs mesurent divers paramètres du véhicule, tels que les niveaux d'huile et de carburant. Le module GPS capture les informations de localisation pour permettre un suivi en temps réel. L'ESP32-CAM collecte les données provenant des capteurs et du GPS, et envoie ces informations au serveur à l'aide du module 3G/LTE. Ce dernier assure la transmission des données en utilisant les réseaux mobiles. Enfin, le serveur héberge l'application web, traite les requêtes des utilisateurs, et gère les interactions avec les bases de données pour le bon fonctionnement du système. Le tableau qui suit détaille les rôles et les fonctions de chaque composant.

Tableau II-1 : Rôles et Fonctions des Composants

<i>Composant</i>	<i>Rôle</i>	<i>Fonction</i>
Capteurs	Collecte des données sur le véhicule	Mesure divers paramètres : niveau d'huile, niveau de carburant, niveau d'huile, etc
Module GPS	Fourniture de données de localisation	Capture les informations de localisation du véhicule pour le suivi en temps réel
ESP32-cam	Traitement des données et communication	Collecte les données du module GPS et des capteurs. Envoie les données au serveur via 3G/LTE
Module 3G/LTE	Transmission des données	Permet la communication entre l'ESP32cam et le serveur en utilisant les réseaux de données mobiles
Serveur	Héberge les contenus de l'application web, traite les requêtes des utilisateurs, gère les interactions avec les bases de données	Reçoit les données de l'ESP32-cam et permet le bon fonctionnement de l'application web à l'aide du traitement du requête et l'interaction avec les bases de données

Source : Auteur

II.1.3 COMMUNICATION DES HARDWARES

Les principaux hardware du système sont l'appareil du client (ordinateur ou téléphone), le serveur et la carte ESP32-CAM. Ils communiquent entre eux via le protocole HTTP, comme le montre la figure II-3.

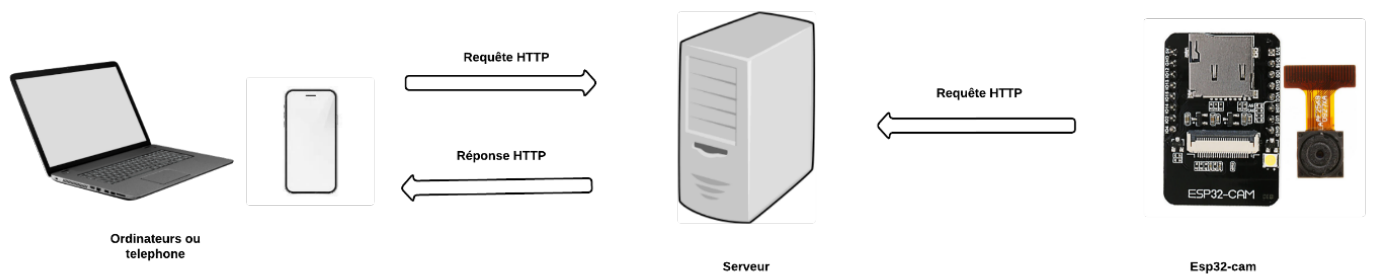


Figure II-3 : Communication des hardware

Source : Auteur

Chapitre II.2 Intégration des IoT et GPS

II.2.1 ETUDE DU SYSTEME D'IOT

Pour surveiller à distance et localiser les véhicules modernes, qui sont déjà équipés de capteurs, en temps réel, les composants électroniques suivants sont utilisés : le microcontrôleur ESP32-CAM, le module GPS Neo6M, et le module Particle Boron.

II.2.1.1. Caractéristique des composants

II.2.1.1.1 Module GPS Neo6M

Le module GPS Neo6M offre plusieurs avantages notables. Il fonctionne avec une alimentation de 3V à 5V et est doté d'une antenne en céramique qui assure un signal de haute qualité. Ce module intègre également une EEPROM pour sauvegarder les paramètres de configuration et un indicateur de signal LED. Le débit en bauds par défaut est de 9600. En termes de dimensions, le module mesure 23 mm sur 30 mm, l'antenne fait 25 mm sur 25 mm, et le câble a une longueur de 50 mm. La figure suivante illustre ce module.

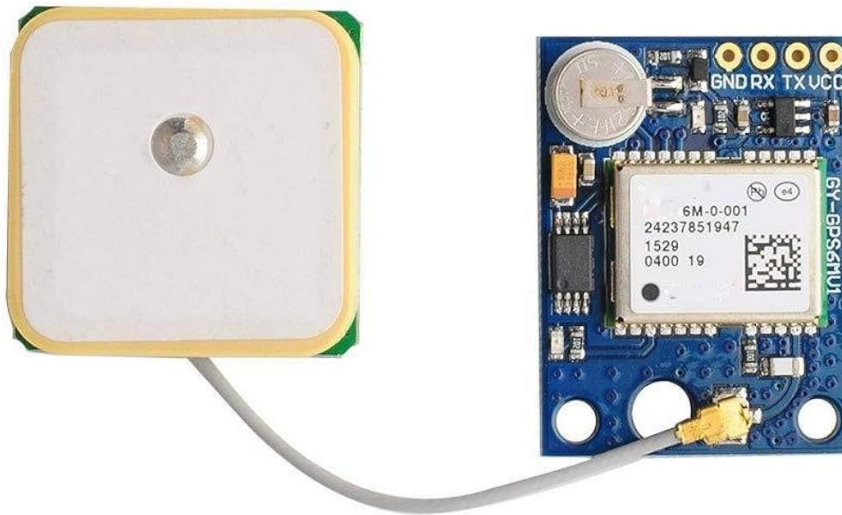


Figure II-4 : Le module GPS Neo6M

Source : [34]

II.2.1.1.2 Module particule Boron LTE :

Le module Particle Boron LTE présente plusieurs caractéristiques importantes. Il fonctionne avec une alimentation de 3,3V et dispose d'un double emplacement SIM pour Nano 4FF et MFF2. Ce module est équipé d'un indicateur LED RGB et d'un port Micro USB 2.0. Il prend en charge les réseaux LTE pour une connectivité améliorée.

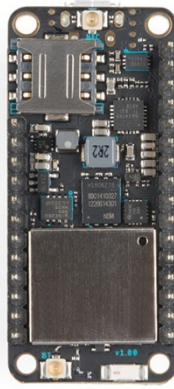


Figure II-5 : Particule Boron LTE

Source : [4]

II.2.1.2. Justificatif des choix des composants

Le tableau II-2 détaille le choix des composants pour le système IoT, chacun ayant un rôle spécifique. Le module GPS Neo6M est utilisé pour fournir des données de localisation et est facilement intégré grâce à un bon support des bibliothèques. L'ESP32-CAM est choisi pour le traitement des données et la communication, agissant comme un microcontrôleur. Le module Particle Boron LTE est sélectionné pour la transmission des données, offrant l'avantage de continuer à fonctionner même en cas de problème de batterie, grâce à sa batterie lithium intégrée.

Composant	Rôle	Justification
Module GPS Neo6M	Donne les données de localisation	Le Neo6M est plus facile à intégrer grâce à son bon support de bibliothèques.
ESP32-CAM	Traitement des données et communication	Microcontrôleur à disposition
Module Particle Boron LTE	Transmission des données	Le Particle Boron, avec sa batterie lithium intégrée, permet encore la transmission de données en cas de dysfonctionnement de la batterie

Tableau II-2 : Choix de composants

Source : Auteur

II.2.1.3. Branchement ESP32-CAM au NEO-6M GPS Module

L'ESP32-CAM est connecté au module GPS NEO-6M pour localiser le véhicule. Le signal de transmission (TX) du module GPS est relié à la broche de réception universelle (UOR) du GPIO 3 de l'ESP32-CAM. Le signal de réception (RX) du module GPS est branché à la broche de transmission universelle (UOT) du GPIO 1 de l'ESP32-CAM. L'alimentation positive du module GPS est connectée à la tension d'alimentation de 3,3 volts de l'ESP32-CAM, et la masse du module GPS est reliée à la masse de l'ESP32-CAM.

II.2.1.4. Branchement ESP32-CAM au Particle Boron

L'ESP32-CAM est connecté au Particle Boron pour envoyer les données du véhicule. Le signal de transmission (TX) du Particle Boron est relié à la broche de réception universelle (UOR) du GPIO 3 de l'ESP32-CAM. Le signal de réception (RX) du Particle Boron est également connecté à la broche de réception universelle (UOT) du GPIO 3 de l'ESP32-CAM. Enfin, la masse du Particle Boron est branchée à la masse de l'ESP32-CAM.

II.2.1.5. Branchement ESP32-CAM avec les capteurs

Les capteurs sont connectés à l'ESP32-CAM pour permettre le traitement des données. Les capteurs recueillent diverses informations sur l'état du véhicule, telles que les niveaux d'huile, de carburant, et d'autres paramètres essentiels. Ces données sont ensuite transmises à l'ESP32-CAM, qui les traite et les prépare pour une utilisation ultérieure. Cette connexion assure une intégration fluide entre les capteurs et le microcontrôleur, facilitant ainsi la gestion et l'analyse des données du véhicule.

II.2.2 TRAITEMENT DES DONNEES

Les données des capteurs et du GPS sont envoyées en temps réel au serveur via le protocole HTTP, en utilisant à la fois l'ESP32-CAM et le module Particle Boron LTE. La figure ci-dessous illustre le traitement simplifié de ces données, montrant comment les informations collectées par les capteurs et le GPS sont transmises et gérées.

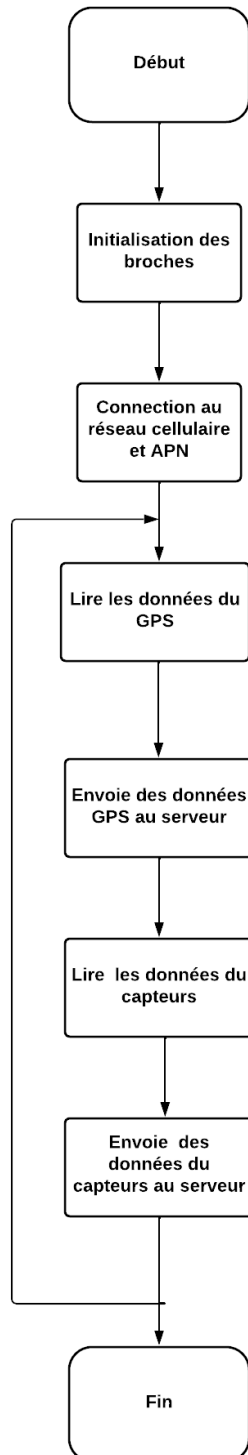


Figure II-6 : Le traitement de données du GPS et capteurs

Source : Auteur

La figure II-7 est la transformation de la figure II-6 en code, prêt à être téléversé dans l'ESP32-CAM pour le traitement.

```
1 #include <Particle.h>
2 #include <HttpClient.h>
3 #include <TinyGPS++.h>
4
5 // Déclaration des objets GPS et Serial
6 TinyGPSPlus gps;
7 const int RX_PIN = 1; // RX du Boron
8 const int TX_PIN = 3; // TX du Boron
9
10 // Broches des capteurs
11 const int SENSOR_PIN_FUEL = 2; // Pin pour le capteur de niveau de carburant
12 const int SENSOR_PIN_OIL = 14; // Pin pour le capteur de niveau d'huile
13 const int SENSOR_PIN_BATTERY = 15; // Pin pour le capteur d'état de batterie
14
15 // URL du script PHP sur le serveur
16 const char* serverName = "192.168.1.101";
17 const char* id_voiture = "1";
18
19 HttpClient http;
20 Http_request_t request;
21 Http_response_t response;
22 http_header_t headers[] = {
23   { "Content-type", "application/x-www-form-urlencoded" },
24   { NULL, NULL } // Fin de la liste des en-têtes
25 };
26
27 // Temporisateurs pour les envois de données
28 unsigned long previousMillisGPS = 0;
29 unsigned long previousMillisSensors = 0;
30 const long intervalsGPS = 10000; // Intervalle pour les données GPS (10 secondes)
31 const long intervalsSensors = 5000; // Intervalle pour les données des capteurs (5 secondes)
32
33 void setup() {
34   // Initialisation des broches
35   pinMode(SENSOR_PIN_FUEL, INPUT);
36   pinMode(SENSOR_PIN_OIL, INPUT);
37   pinMode(SENSOR_PIN_BATTERY, INPUT);
38   Serial.begin(115200);
39   Serial.begin(9600, SERIAL_8N1, RX_PIN, TX_PIN);
40
41   // Connexion au réseau cellulaire
42   Particle.connect();
43   while (!Particle.connected()) {
44     delay(500);
45     Serial.print(".");
46   }
47   Serial.println("Connecté au réseau cellulaire");
48   Particle.cellular.setName("Tolma Internet");
49   Particle.cellular.setAPN("telanet");
50 }
51
52 void loop() {
53   unsigned long currentMillis = millis();
54
55   // Lecture des données GPS
56   while (Serial.available() > 0) {
57     gps.encode(Serial.read());
58   }
59
60   // Envoi des données GPS à intervalles réguliers
61   if (currentMillis - previousMillisGPS >= intervalsGPS) {
62     previousMillisGPS = currentMillis;
63     envoyerDonneesGPS();
64   }
65
66   // Envoi des données des capteurs à intervalles réguliers
67   if (currentMillis - previousMillisSensors >= intervalsSensors) {
68     previousMillisSensors = currentMillis;
69     envoyerDonneesCapteurs();
70   }
71 }
72
73 void envoyerDonneesGPS() {
74   if (gps.location.isUpdated()) {
75     float latitude = gps.location.lat();
76     float longitude = gps.location.lng();
77
78     // Créer l'URL avec les données GPS
79     String gpsPath = "/location_voiture/admin/update_location.php?latitude=" + String(latitude, 6) + "&longitude=" + String(longitude, 6) + "&id_voiture=" + String(id_voiture);
80
81     // Préparer la requête HTTP pour les données GPS
82     request.hostname = serverName;
83     request.port = 80;
84     request.path = gpsPath;
85
86     // Envoyer les données GPS au serveur
87     http.get(request, response, headers);
88
89     if (response.status == 200) {
90       Serial.println("Données GPS envoyées avec succès !");
91     } else {
92       Serial.println("Erreur lors de l'envoi des données GPS");
93     }
94   }
95 }
96
97 void envoyerDonneesCapteurs() {
98   int niveauCarburant = analogRead(SENSOR_PIN_FUEL); // Lire le niveau de carburant
99   int niveauHuile = analogRead(SENSOR_PIN_OIL); // Lire le niveau d'huile
100   int etatBatterie = analogRead(SENSOR_PIN_BATTERY); // Lire l'état de la batterie
101
102   // Créer le chemin avec les données des capteurs
103   String sensorPath = "/location_voiture/admin/save_sensor_data.php";
104   String sensorQuery = "niveau_carburant=" + String(niveauCarburant) +
105     "&niveau_huile=" + String(niveauHuile) +
106     "&etat_batterie=" + String(etatBatterie) +
107     "&id_voiture=" + String(id_voiture);
108
109   // Préparer la requête HTTP pour les données des capteurs
110   request.hostname = serverName;
111   request.port = 80;
112   request.path = sensorPath + sensorQuery;
113
114   // Envoyer les données des capteurs au serveur
115   http.get(request, response, headers);
116
117   if (response.status_code == 200) {
118     Serial.println("Données des capteurs envoyées avec succès !");
119   } else {
120     Serial.println("Erreur lors de l'envoi des données des capteurs");
121   }
122 }
```

Figure II-7 : Le code de traitement des capteurs et GPS

Source : Auteur

Chapitre II.3 Conception de l'application web

II.3.1 MODE DE FONCTIONNEMENT DE L'APPLICATION

La figure suivante présente deux diagrammes de flux distincts, chacun représentant un processus différent dans la gestion et la réservation de véhicules de location. Le diagramme de gauche illustre le processus du point de vue de l'utilisateur, commençant par la sélection de l'intervalle de dates pour la location, suivie du choix du véhicule, puis du paiement en ligne, et se terminant par la confirmation de la réservation. Le diagramme de droite montre le processus du point de vue de l'administrateur ou du gestionnaire, débutant par la sélection du véhicule, suivi de la localisation du véhicule, de la visualisation de l'état des capteurs, de la consultation des historiques de maintenance, et de l'accès aux informations sur le locataire du véhicule, avant de terminer le processus.

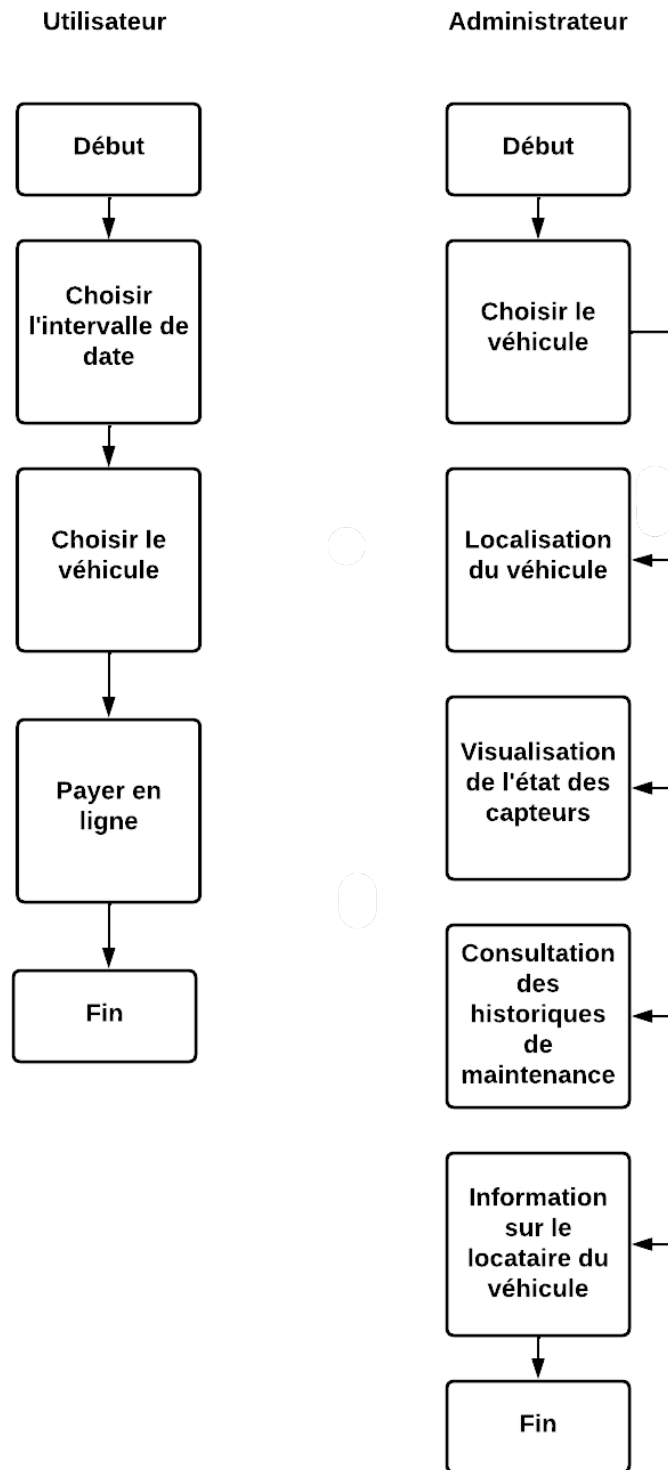


Figure II-8 : L'algorithme de fonctionnement de l'application

Source : Auteur

II.3.2 ARCHITECTURE GÉNÉRALE DE L'APPLICATION

L'architecture de l'application repose sur trois principaux composants : le front end, le backend et la base de données.

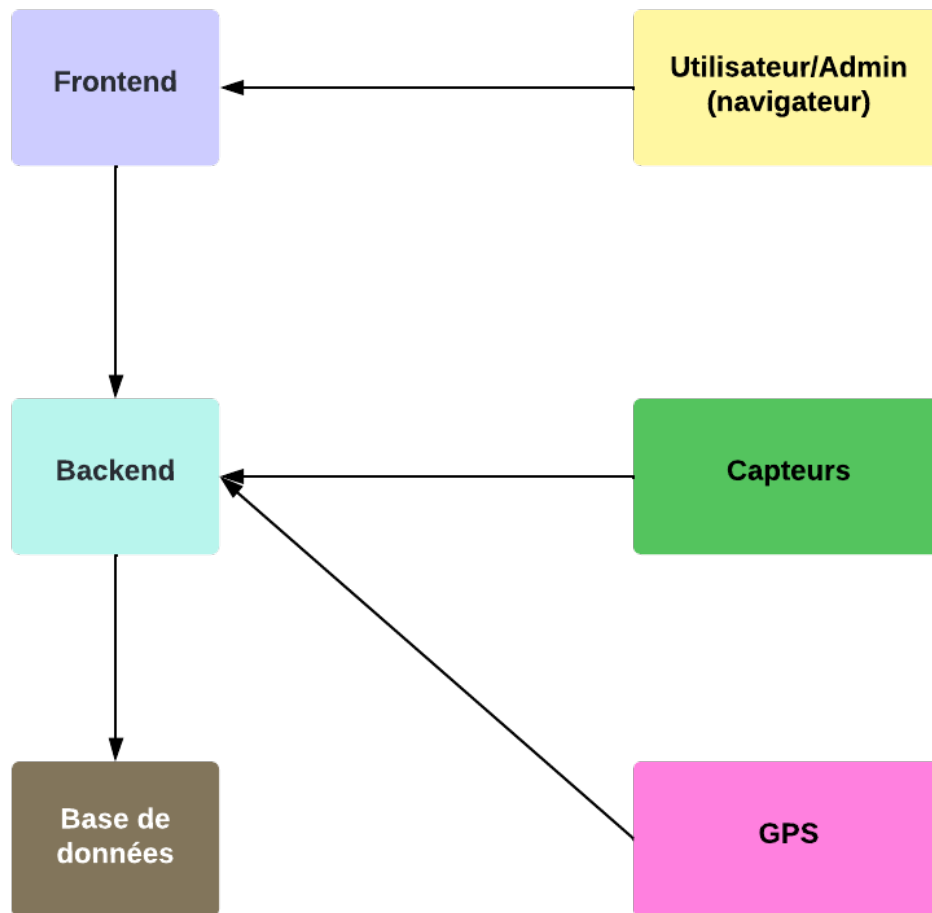


Figure II-9 : Architecture du système

Source : Auteur

II.3.2.1. Frontend

Le frontend représente la partie visible de l'application, développée en HTML, CSS et JavaScript. Il fournit l'interface utilisateur pour les clients et les administrateurs, permettant aux clients de réserver des voitures et aux administrateurs de gérer les véhicules.

II.3.2.2. Backend

Le backend constitue la partie serveur de l'application, développée en PHP. Il gère les requêtes provenant du frontend, exécute des opérations sur la base de données, et renvoie les résultats au frontend. De plus, il communique avec les capteurs et les GPS pour obtenir et traiter les données.

II.3.2.3. Base de données

La base de données conserve toutes les informations concernant les clients, les véhicules, les réservations, les données des capteurs et les maintenances.

II.3.2.4. Capteurs

Les capteurs installés dans les véhicules surveillent le niveau de carburant, le niveau d'huile et l'état de la batterie. Ils transmettent les données en temps réel au backend pour leur traitement et leur analyse.

II.3.2.5. GPS

Le GPS fournit en temps réel les coordonnées géographiques au backend, ce qui permet de suivre en continu la position du véhicule. Cette fonctionnalité est essentielle pour obtenir des informations précises sur l'emplacement du véhicule à tout moment, facilitant ainsi le suivi et la gestion des déplacements.

II.3.2.6. Navigateurs (utilisateurs/ administrateurs)

Les navigateurs web offrent aux utilisateurs et aux administrateurs la possibilité d'accéder à l'application web. Ils permettent ainsi d'interagir avec l'interface utilisateur, de consulter les informations, de gérer les réservations et d'effectuer diverses tâches liées à l'application depuis n'importe quel appareil connecté à Internet.

II.3.3 MODELISATION DE LA BASE DE DONNEES

II.3.3.1. Modélisation au niveau conceptuel

Les administrateurs utilisent des attributs classiques (username, password, etc) pour l'authentification et l'identification afin de gérer le système. Les voitures, qui sont les biens à louer, sont décrites par des attributs détaillant leurs caractéristiques. Les clients, ou locataires, sont identifiés et contactés à l'aide de leurs attributs spécifiques. Les techniciens sont responsables de la maintenance des véhicules, tandis que les capteurs collectent des données sur l'état des véhicules. Le MCD de l'application est illustré comme suit.

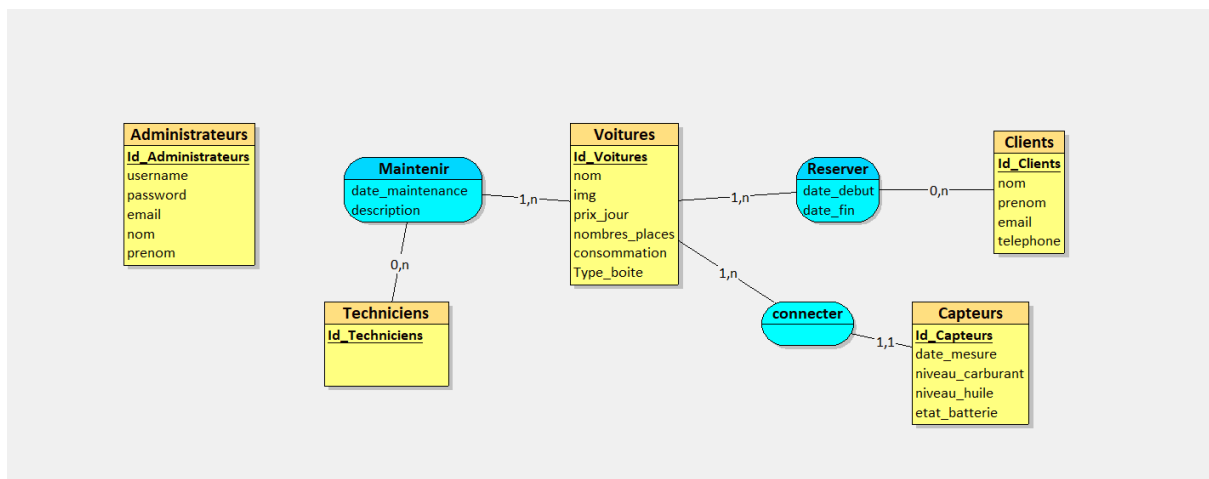


Figure II-10 : MCD de l'application web

Source : Auteur

La relation "maintenir" indique qu'un administrateur a la possibilité de planifier la maintenance pour une ou plusieurs voitures. La relation "réserver" montre qu'un client peut réserver une ou plusieurs voitures pour une période déterminée. Enfin, la relation "connecter" signifie qu'une voiture peut être équipée d'un ou plusieurs capteurs.

II.3.3.2. Modélisation au niveau logique

Après avoir réalisé la modélisation conceptuelle, on passe à la modélisation logique comme suit :

- Voitures = (Id_Voitures COUNTER, nom VARCHAR(50), img VARCHAR(255), prix_jour INT, nombres_places INT, consommation_ DECIMAL(15,2), Type_boite VARCHAR(20));

- Clients = (Id_Clients COUNTER, nom VARCHAR(50), prenom VARCHAR(50), email VARCHAR(100), telephone VARCHAR(20));
- Capteurs = (Id_Capteurs COUNTER, date_mesure DATETIME, niveau_carburant DECIMAL(15,2), niveau_huile DECIMAL(15,2), etat_batterie VARCHAR(20), #Id_Voitures);
- Techniciens = (Id_Techniciens COUNTER);
- Reserver = (#Id_Voitures, #Id_Clients, date_debut VARCHAR(50), date_fin VARCHAR(50));
- Maintenir = (#Id_Voitures, #Id_Techniciens, date_maintenance VARCHAR(50), description TEXT);
- Administrateurs = (Id_Administrateurs COUNTER, username VARCHAR(50), password VARCHAR(50), email VARCHAR(50), nom VARCHAR(50), prenom VARCHAR(50));

II.3.3.2.1 Relations :

Une voiture peut avoir plusieurs réservations au fil du temps, permettant à différents clients de la louer à différentes périodes. De même, un client peut effectuer plusieurs réservations pour diverses voitures ou pour la même voiture à des moments distincts. Chaque voiture peut être équipée de plusieurs capteurs pour surveiller différents aspects de son état, tels que le niveau de carburant, l'huile, et l'état de la batterie.

II.3.3.3. Modélisation au niveau physique

Dans ce projet, le Système de Base de Données MySQL a été choisi. Le Modèle Physique des Données (MPD) se présente sous la forme d'un script SQL qui crée toutes les tables du Modèle Logique des Données (MLD) dans la base de données "location_voiture" sur MySQL. Les illustrations suivantes présentent la création des différentes tables nécessaires pour le bon fonctionnement de l'application.

II.3.3.3.1 Création de la table Voitures :

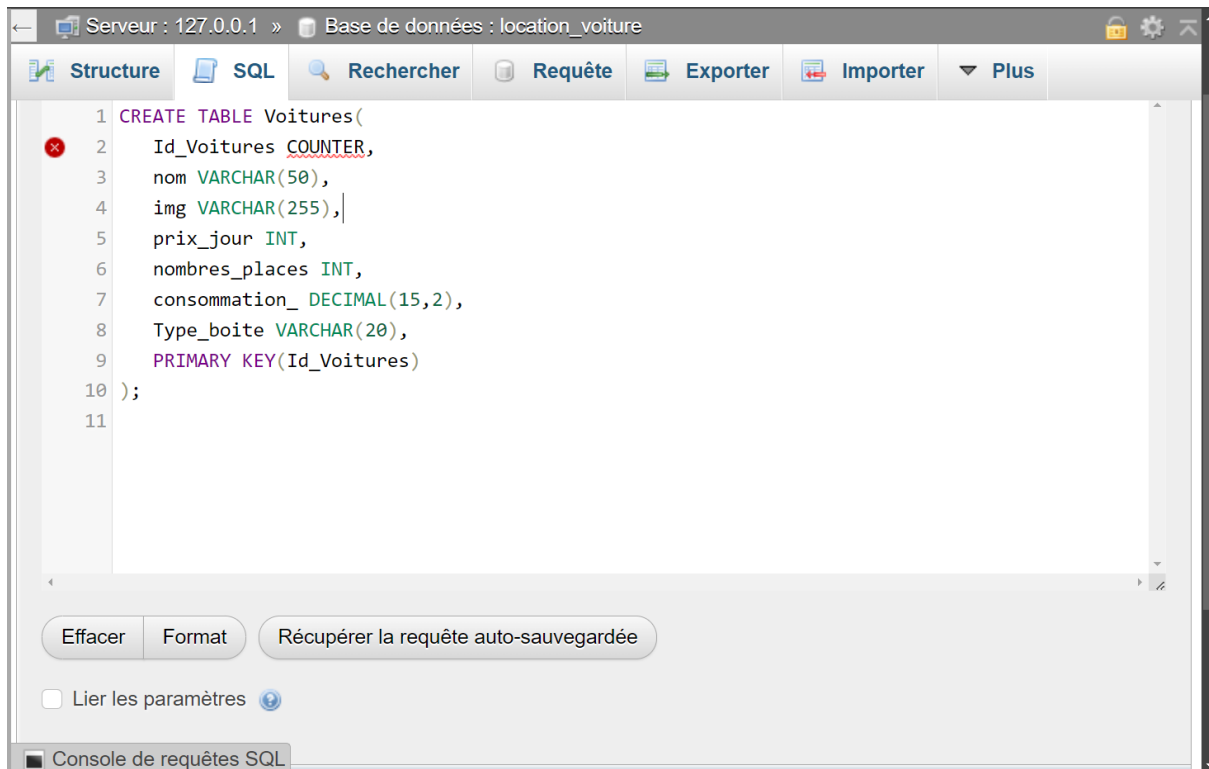


Figure II-11 : Code SQL pour créer la table Voitures

Source : Auteur

Source : Auteur

II.3.3.3.2 Création de la table Clients :

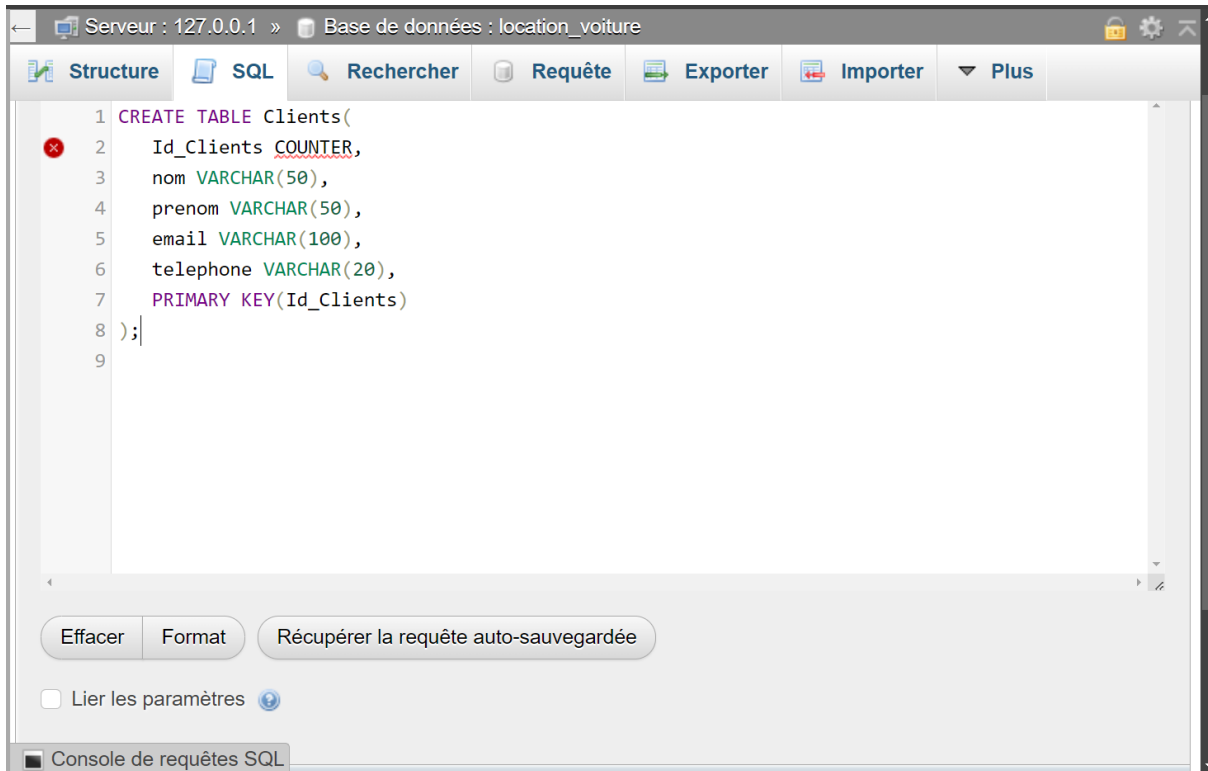


Figure II-12 : Code SQL pour créer la table Clients

Source : Auteur

II.3.3.3.3 Création de la table Capteurs :

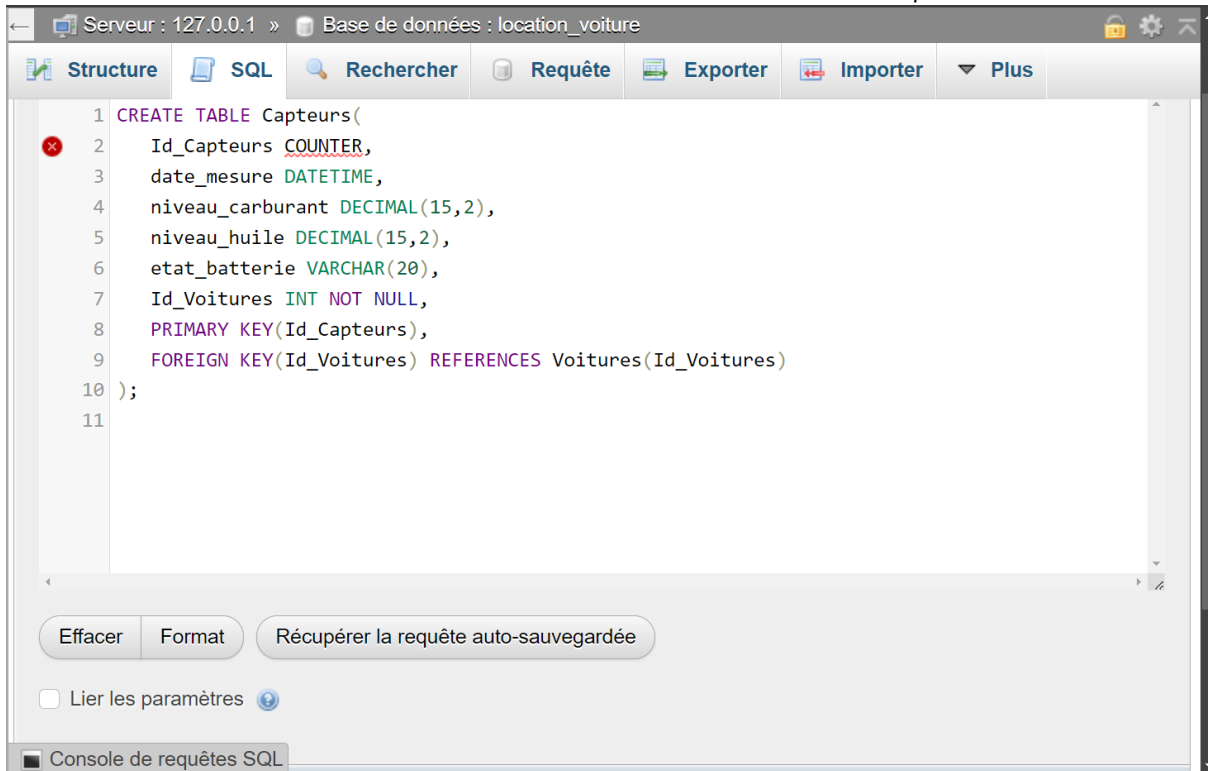


Figure II-13 : Code SQL pour créer la table Capteurs

Source : Auteur

II.3.3.3.4 Création de la table Administrateurs



Figure II-14 : Code SQL pour créer la table Administrateurs

Source : Auteur

II.3.3.3.5 Création de la table Réservations



Figure II-15 : Code SQL pour créer la table Réservations

Source : Auteur

II.3.3.3.6 Création de la table Maintenances :

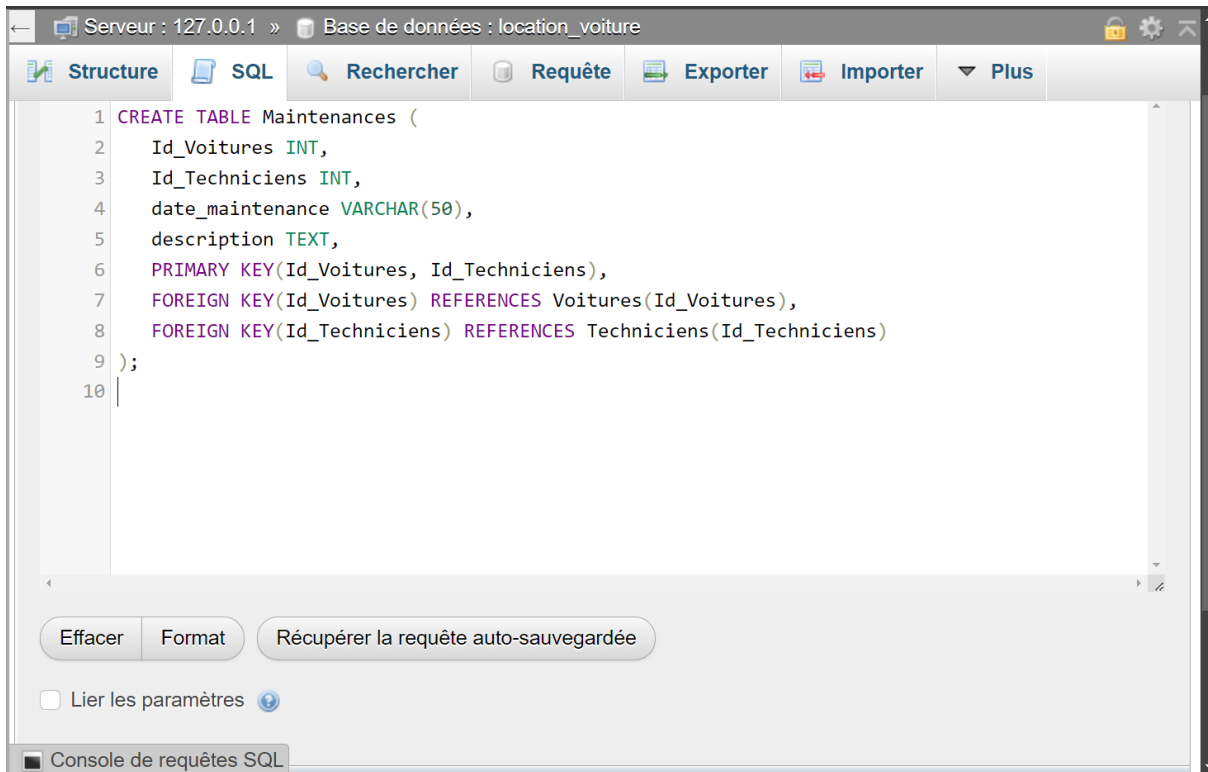


Figure II-16 : Code SQL pour créer la table Maintenances

Source : Auteur

II.3.4 FONCTIONNALITES MODE UTILISATEUR

Le mode utilisateur de l'application web permet aux clients de louer des véhicules de manière intuitive et efficace. Les fonctionnalités offertes incluent l'accueil et le formulaire, la disponibilité des véhicules, le détail du véhicule et la réservation, le paiement et l'enregistrement des données.

L'utilisateur commence par accéder à la page d'accueil de l'application, où il trouve un formulaire de réservation. Il y saisit les informations nécessaires, telles que les dates de début et de fin de la location. Le système affiche alors une liste des véhicules disponibles pour ces dates. L'utilisateur sélectionne le véhicule souhaité parmi les options proposées et confirme sa réservation. Il saisit ensuite ses informations personnelles et de paiement pour finaliser la réservation. Enfin, le système valide les informations fournies et enregistre la réservation dans la base de données (voir figure II-13).

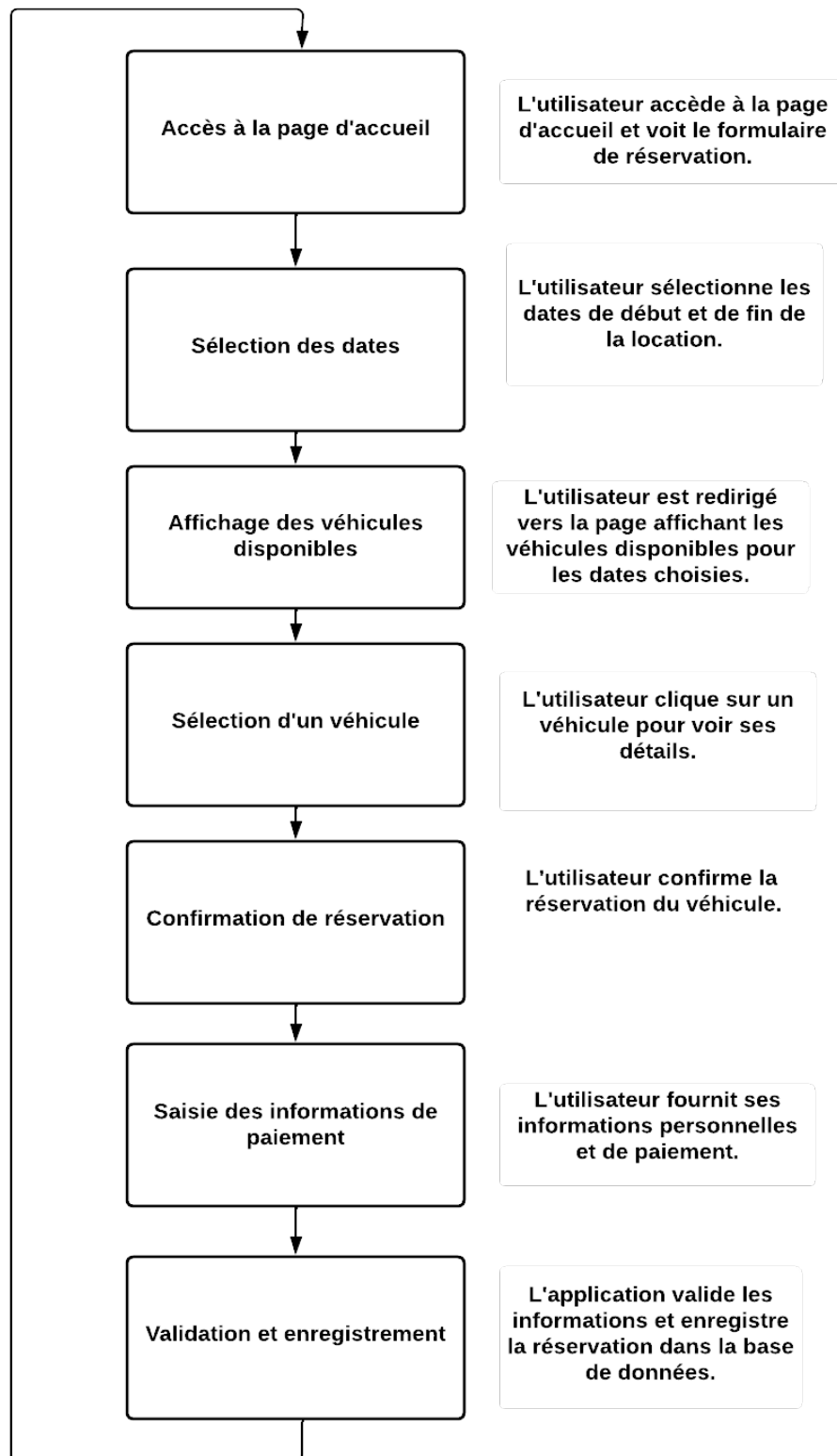


Figure II-17 Scénario d'utilisation du mode utilisateur

Source : Auteur

II.3.4.1. Accueil et formulaire

L'utilisateur est accueilli par une interface affichant le logo de l'application et un formulaire permettant de sélectionner les dates de début et de fin de la location. Une fois les dates saisies, l'application vérifie leur validité, en s'assurant par exemple que la date de début est antérieure à la date de fin.

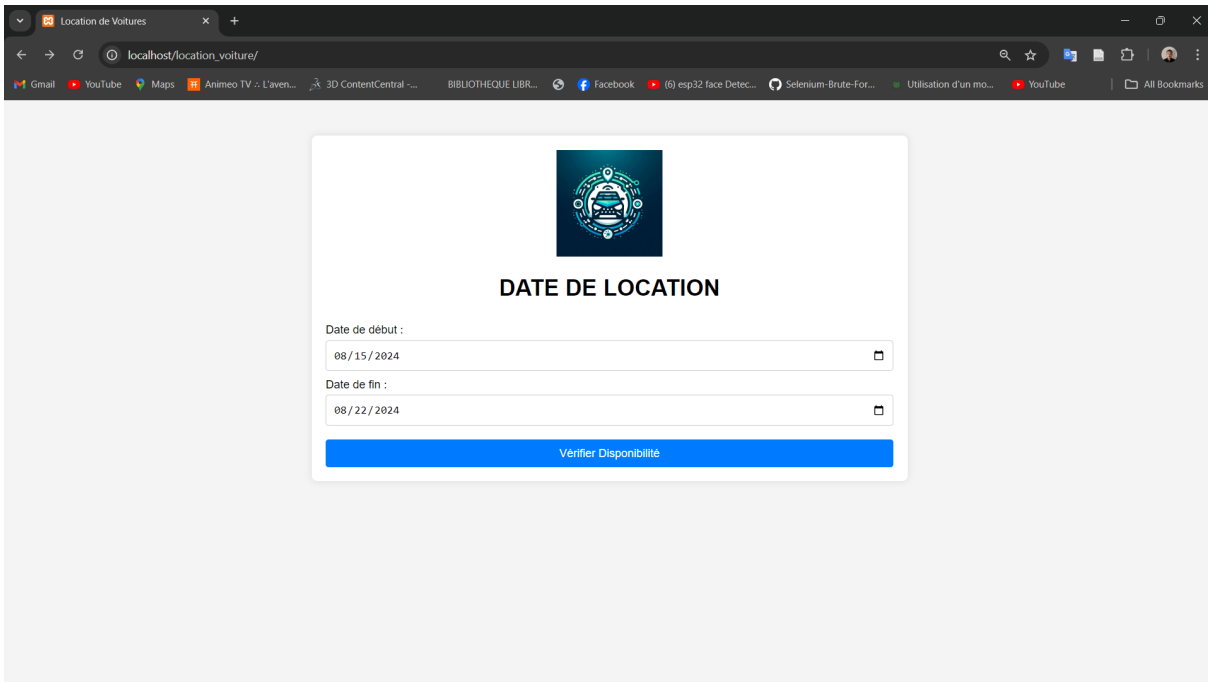


Figure II-18 Page d'accueil

Source : Auteur

II.3.4.2. Disponibilité des véhicules

Après la validation des dates, l'utilisateur est redirigé vers une page affichant la liste des véhicules disponibles pour les périodes sélectionnées. Chaque véhicule est présenté avec une image et son prix journalier.

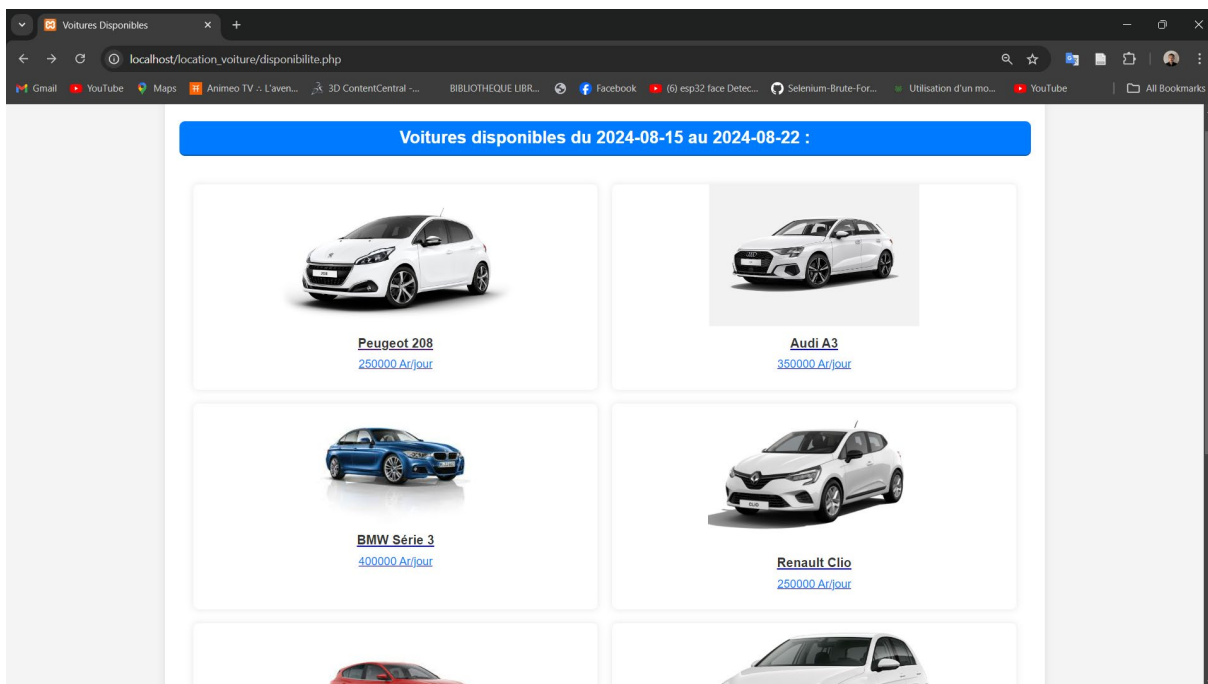


Figure II-19 Page Voitures disponibles

Source : Auteur

II.3.4.3. Détail du véhicule et réservation :

En cliquant sur un véhicule, l'utilisateur est redirigé vers une page présentant les caractéristiques détaillées du véhicule sélectionné. Sur cette page, l'utilisateur a la possibilité de confirmer la réservation du véhicule pour les dates choisies.

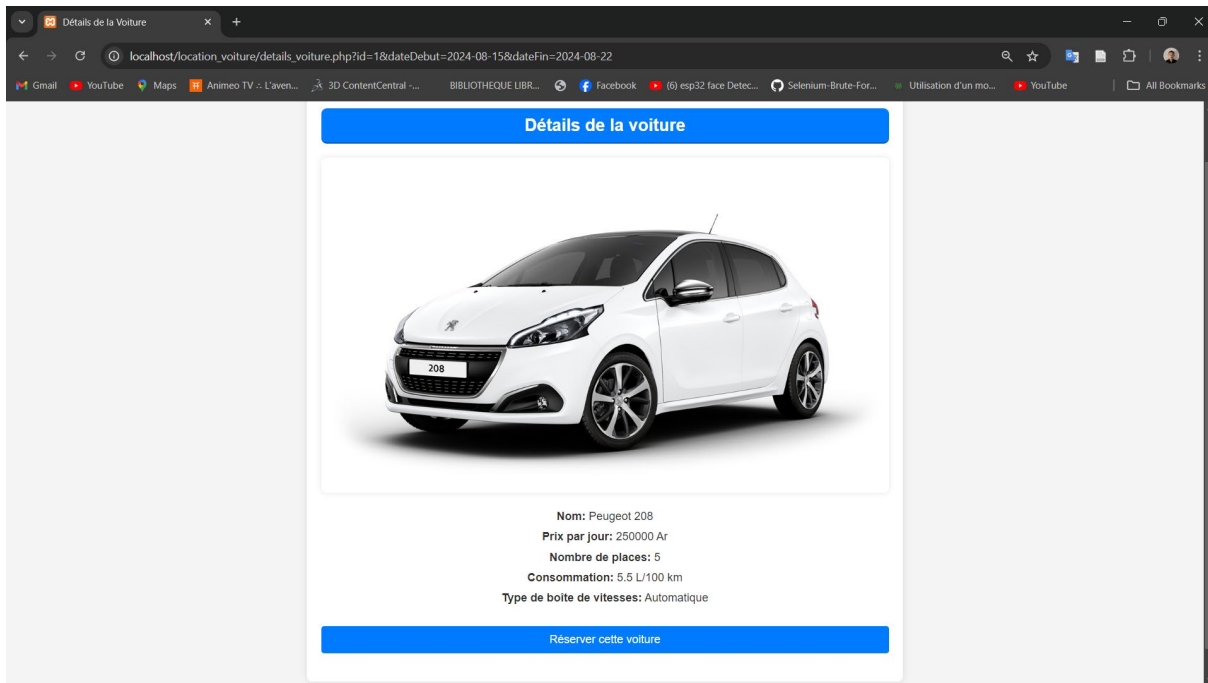


Figure II-20 Page détails de la voiture

Source : Auteur

II.3.4.4. Paiement et Enregistrement des données

Une fois la réservation confirmée, l'utilisateur est dirigé vers la page de paiement où il doit fournir ses informations personnelles (nom, prénom, téléphone, adresse e-mail) ainsi que les informations de sa carte de crédit (numéro, nom du titulaire, date d'expiration, CVV). Après la validation des informations de paiement, l'application enregistre les données de réservation et les informations du client dans la base de données.



Informations de Paiement

Montant total à payer : **300,000.00 €**

Informations Personnelles

Nom :

Prénom :

Téléphone :

Adresse e-mail :

Informations de Carte de Crédit

Numéro de carte Visa :

Nom du titulaire :

Date d'expiration (MM/AAAA) :

CVV :

Valider le Paiement

Figure II-21 Page Paiement

Source : Auteur

II.3.5 FONCTIONNALITES MODE ADMINISTRATEUR

Le mode administrateur de l'application web offre aux administrateurs des outils pour suivre l'état des capteurs et la position des véhicules, afficher l'historique de maintenance, et consulter les informations des locataires des véhicules. Les fonctionnalités disponibles pour les administrateurs comprennent l'authentification, le choix du véhicule en cours de location, le suivi en temps réel de l'état des capteurs, la position du véhicule, l'historique de maintenance, et les informations du locataire.

L'administrateur commence par s'identifier à l'aide de ses identifiants uniques, tels que le nom d'utilisateur et le mot de passe, pour garantir un accès sécurisé aux données sensibles. Après authentification, il peut consulter la liste des véhicules actuellement en location et obtenir des informations détaillées sur chaque véhicule en sélectionnant celui de son choix. L'administrateur a également la possibilité de suivre en temps réel l'état des véhicules, incluant les niveaux de carburant, d'huile, l'état de la batterie, ainsi que la position géographique sur une carte. Il peut consulter l'historique des opérations de maintenance effectuées sur chaque véhicule, ce qui permet de suivre l'état général de la flotte et de planifier les entretiens futurs. Enfin, l'administrateur peut accéder aux informations personnelles des clients ayant loué des véhicules. (cf figure II-22)..

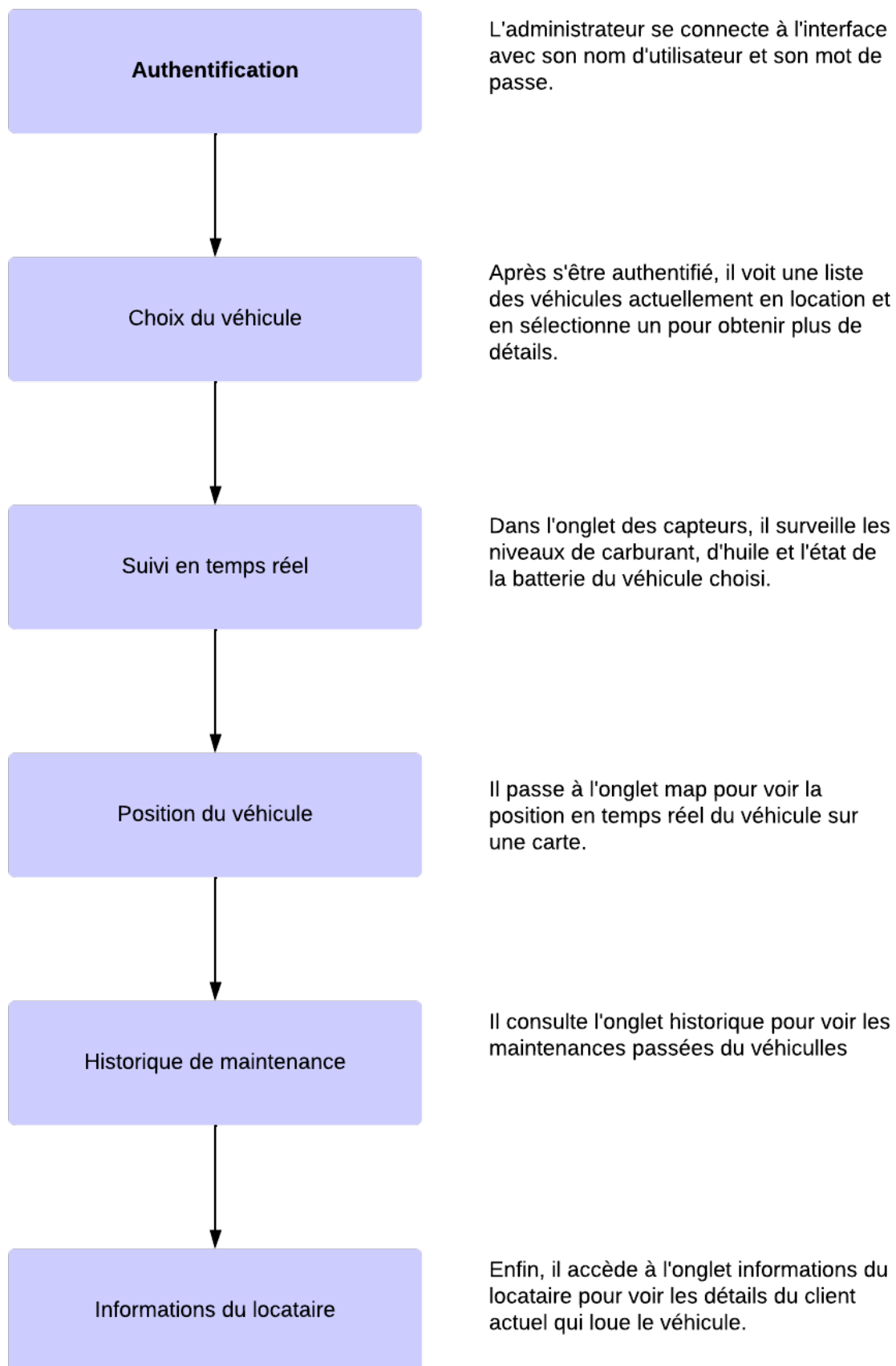


Figure II-22 : Scénario d'utilisation du mode administrateur

Source : Auteur

II.3.5.1. Authentification :

Les administrateurs doivent s'authentifier en fournissant un nom d'utilisateur et un mot de passe pour accéder à l'interface d'administration. Une fois authentifiés, les administrateurs sont redirigés vers une page qui affiche les voitures encours de location.

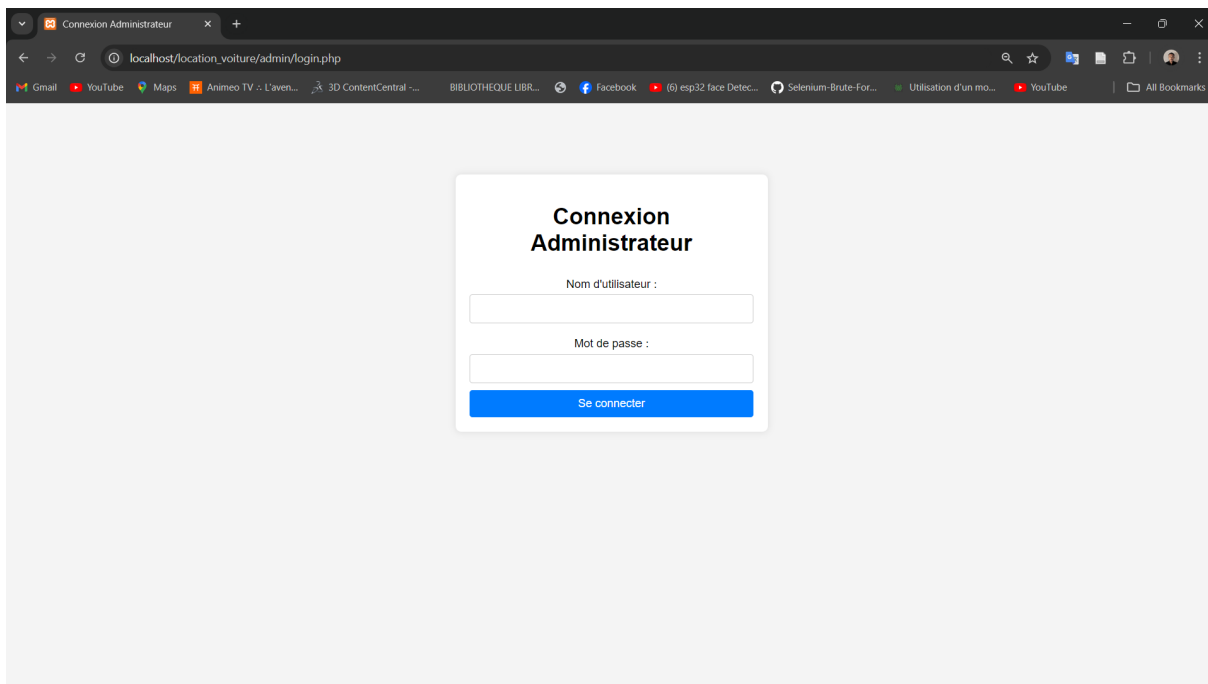
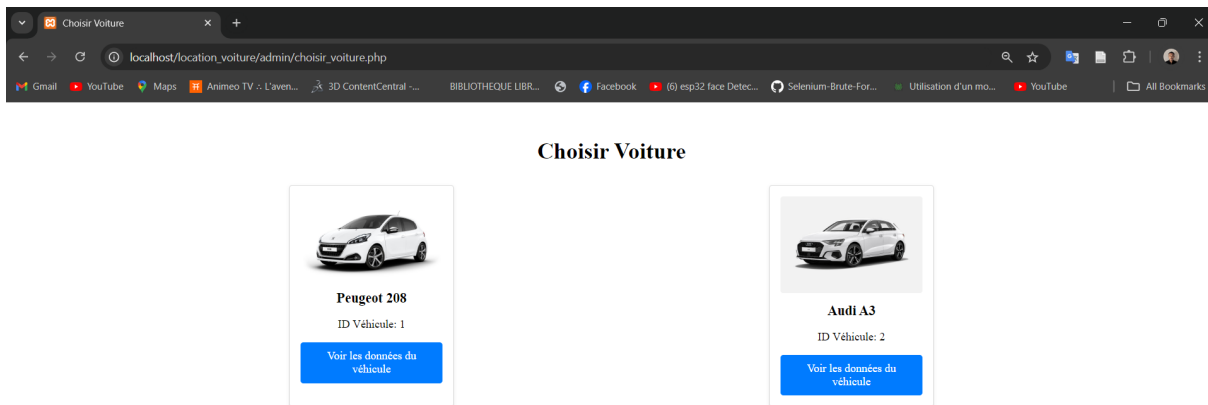


Figure II-23 : Page d'authentification

Source : Auteur

II.3.5.2. Choix du véhicule encours de location

Sur cette page de la figure suivante, seuls les véhicules actuellement en location sont affichés. L'administrateur peut sélectionner un véhicule spécifique pour visualiser l'état de ses capteurs, sa localisation, son historique de maintenance et les informations de son locataire.



Source : Auteur

II.3.5.3. Suivi en temps réel de l'état des capteurs

Dans cet onglet, la page affiche en temps réel les données des capteurs pour le véhicule sélectionné, y compris le niveau de carburant, le niveau d'huile et l'état de la batterie.

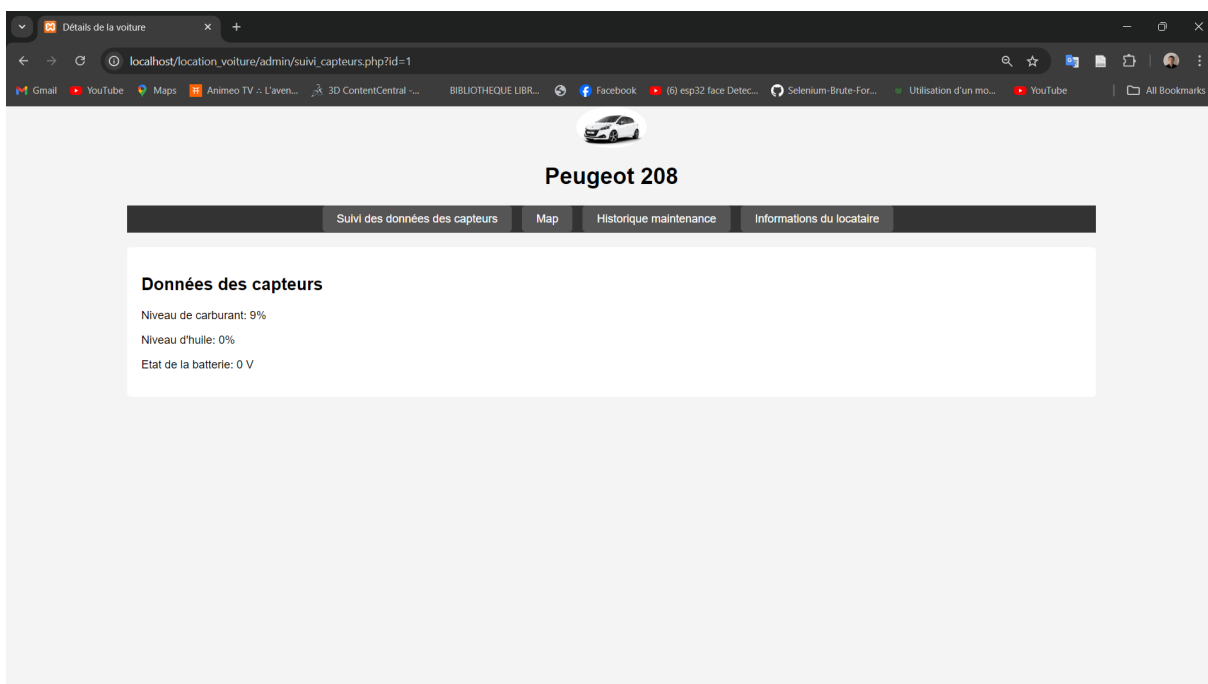


Figure II-24 : L'onglet suivi des données du capteur

Source Auteur

II.3.5.4. Position du véhicule

Dans l'onglet "Carte", l'administrateur peut voir la position actuelle du véhicule sélectionné. Il a également la possibilité de choisir le format de carte souhaité.

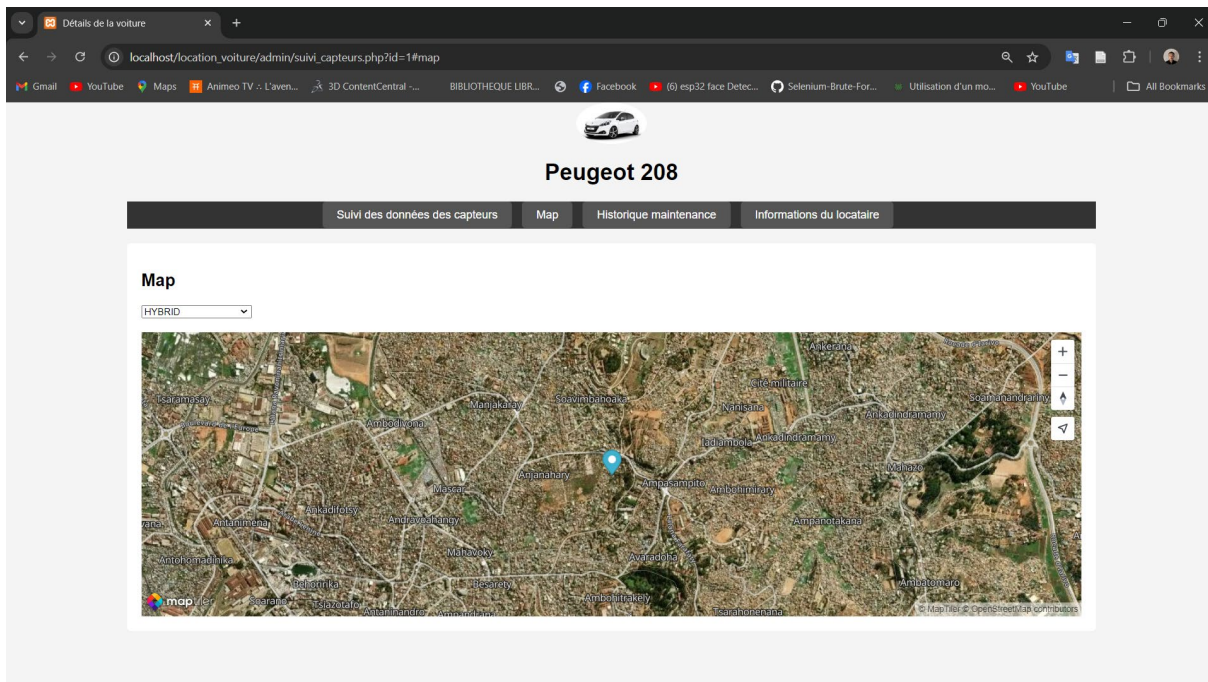


Figure II-25 : L'onglet map

Source : Auteur

II.3.5.5. Historique maintenance

Dans cet onglet, les historiques de maintenance pour le véhicule sélectionné sont affichés dans un tableau. Le tableau comporte deux colonnes : la date de maintenance et la description.

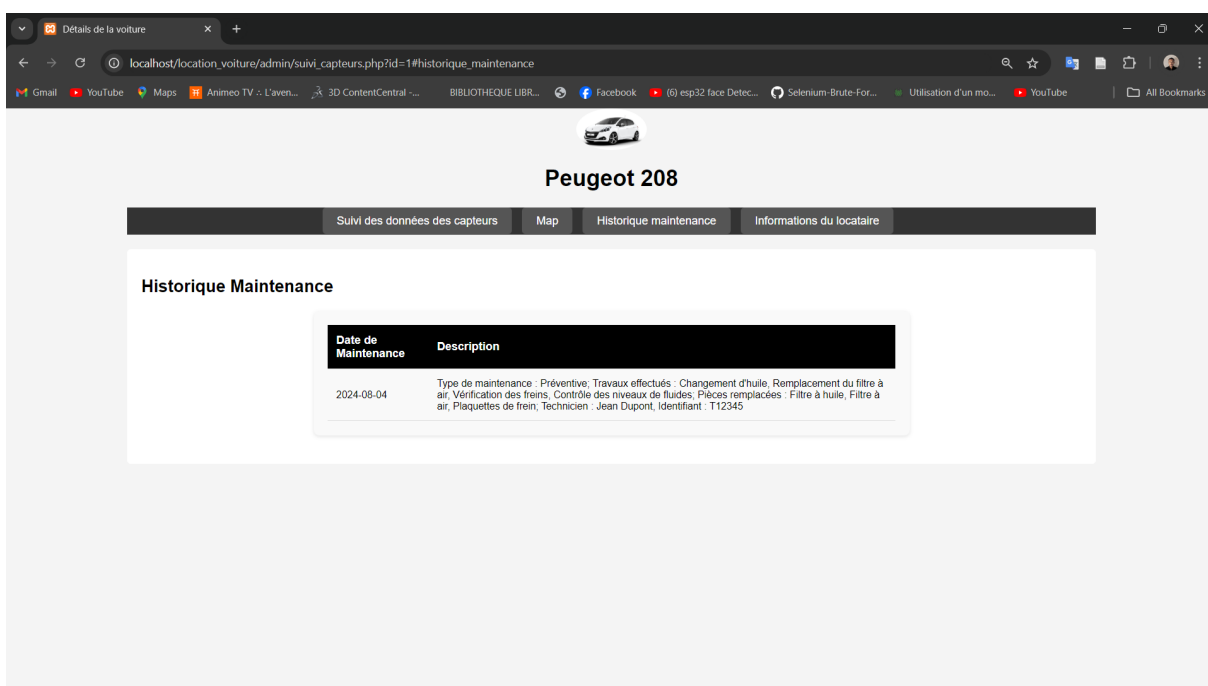


Figure II-26 : L'onglet historique maintenance

Source : Auteur

II.3.5.6. Informations du locataire

L'administrateur peut consulter les informations du locataire actuel du véhicule sélectionné en cliquant sur l'onglet "Informations du locataire".

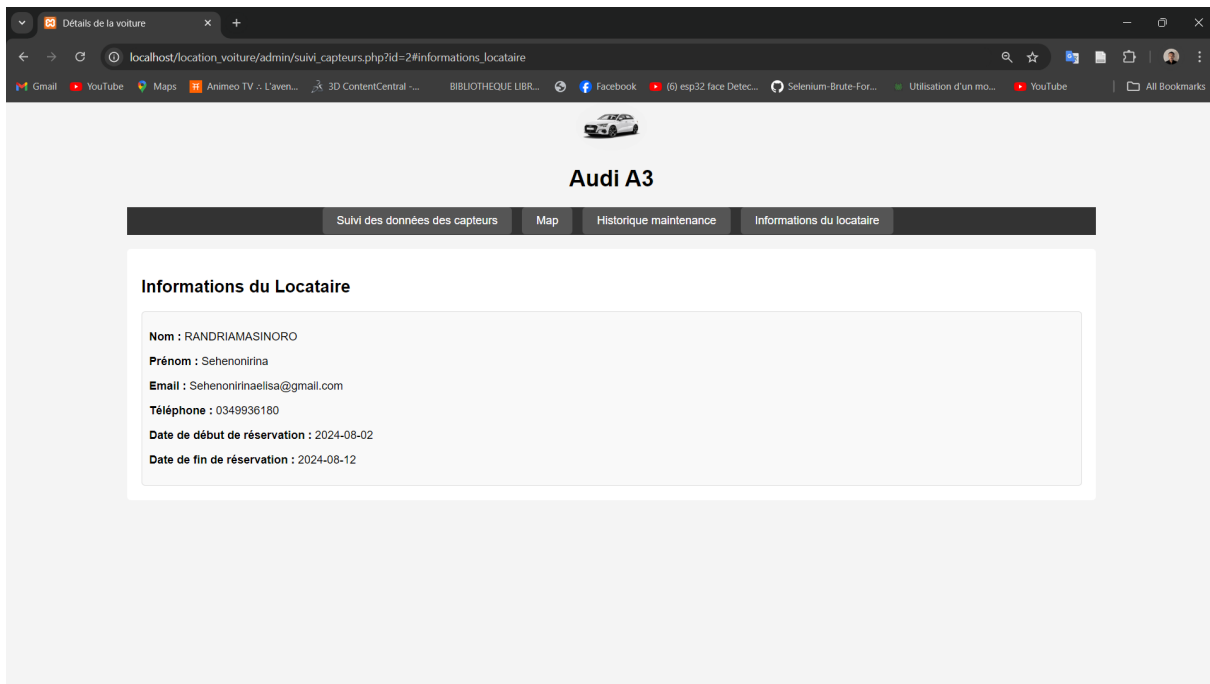


Figure II-27 : L'onglet information du locataire

Source : Auteur

Chapitre II.4 Développement et déploiement de l'application web

II.4.1 DEVELOPPEMENT FRONTEND

Les langages utilisés pour développer le frontend comprennent HTML pour la structure des pages, CSS pour la stylisation, et JavaScript pour les interactions. PHP, quant à lui, est utilisé comme langage de script côté serveur pour créer des pages web dynamiques.

II.4.1.1. Mode utilisateur

Dans l'interface du mode utilisateur, il y a quatre pages : Accueil et Formulaire, Disponibilité des véhicules, Détail du véhicule et réservation, et Paiement et Enregistrement des données

II.4.1.1.1 Accueil et Formulaire

Pour développer la page d'accueil, plusieurs lignes de code ont été nécessaires. La page est intitulée "Location de Voitures" et contient un formulaire de réservation où l'utilisateur peut entrer une date de début et une date de fin pour la location. Un bouton "Vérifier Disponibilité" permet de soumettre le formulaire (cf. Figure II-28).

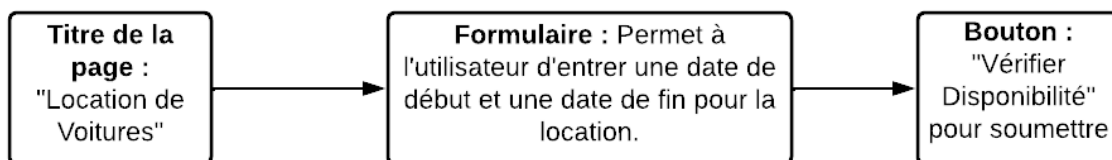


Figure II-28 : Fonctionnalité en bloc du code HTML de la page d'accueil

Source : Auteur

Lors de la soumission du formulaire, le code JavaScript vérifie si la date de début est antérieure à la date de fin. Si la date de début est postérieure ou égale à la date de fin, l'envoi du formulaire est annulé grâce à `event.preventDefault()` et un message d'erreur est affiché. Si les dates sont correctes, le message d'erreur est masqué.

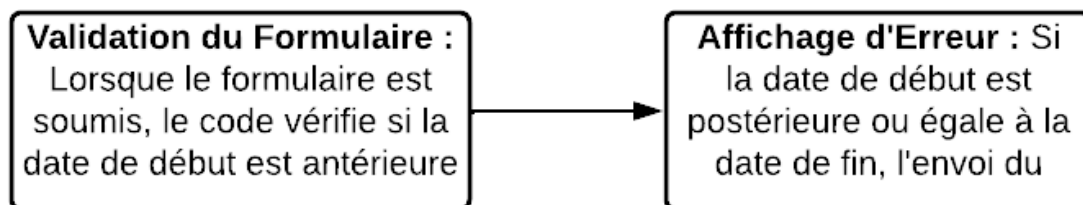


Figure II-29 : Fonctionnalité du code JavaScript de la page d'accueil

Source : Auteur

II.4.1.1.2 Disponibilité des véhicules

Pour développer la page, plusieurs lignes de code ont été nécessaires.

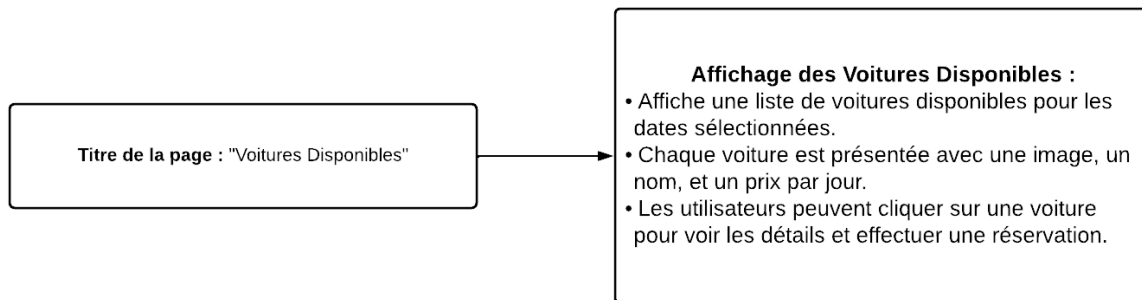


Figure II-30 : Fonctionnalité du code de la page de disponibilité des voitures

Source : Auteur

Ce code permet aux utilisateurs de voir les voitures disponibles pour une période spécifique en envoyant des dates via le formulaire précédent. Il se connecte à une base de données, récupère les voitures disponibles et les affiche sur la page. Si aucune voiture n'est disponible, un message est affiché.

II.4.1.1.3 Détail du véhicule et réservation

Pour créer la figure II-11, plusieurs lignes de code ont été nécessaires. La page est intitulée "Voitures Disponibles" et affiche une liste de voitures disponibles en fonction des dates sélectionnées par l'utilisateur. Chaque voiture est présentée avec une image, un nom, et un prix par jour. Les utilisateurs peuvent cliquer sur une voiture pour voir plus de détails et procéder à une réservation.

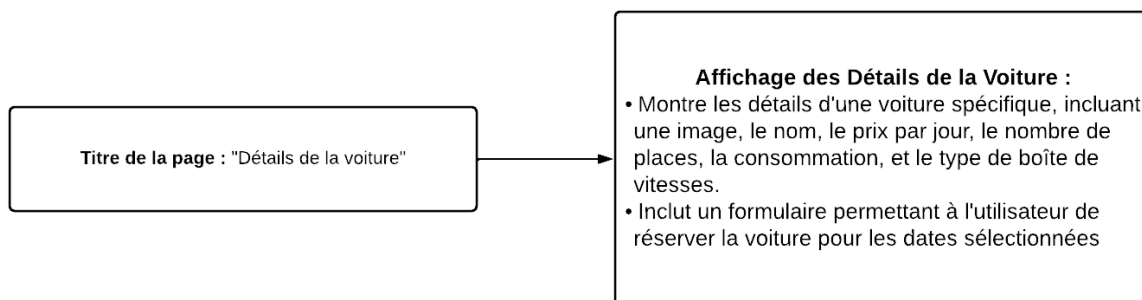


Figure II-31 : Fonctionnalité en bloc du code frontend de la page détails du véhicule

Source : Auteur

II.4.1.1.4 Paiement et Enregistrement des données

Pour créer la figure II-12, plusieurs lignes de code ont été nécessaires. La page est intitulée "Paiement" et affiche le montant total à payer, calculé en fonction du prix par jour de la voiture et de la durée de la location. Un formulaire de paiement est également présent, comprenant des champs pour les informations personnelles telles que le nom, le prénom, le téléphone, et l'adresse e-mail. Il y a aussi des champs pour les informations de carte de crédit, notamment le numéro de carte Visa, le nom du titulaire, la date d'expiration, et le CVV. Un bouton "Valider le Paiement" permet de soumettre le formulaire et de procéder au paiement.

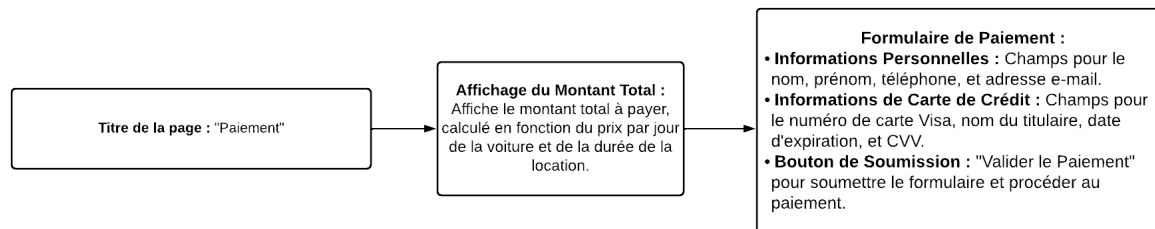


Figure II-32 : Fonctionnalité du code HTML et PHP de la page paiement

Source : Auteur

II.4.1.2. Mode administrateur

Dans l'interface du mode administrateur, il y a trois pages : l'authentification, le choix du véhicule en cours de location, et les détails de la voiture.

II.4.1.2.1 L'authentification

Pour créer l'interface de la page, plusieurs lignes de code ont été nécessaires.

La page est intitulée "Connexion Administrateur". Elle contient un formulaire de connexion avec un champ de texte pour entrer le nom d'utilisateur et un champ de mot de passe pour saisir le mot de passe. Un bouton "Se connecter" permet d'envoyer les informations de connexion. Si les informations saisies sont incorrectes, un message d'erreur est affiché. (cf figure II-33)

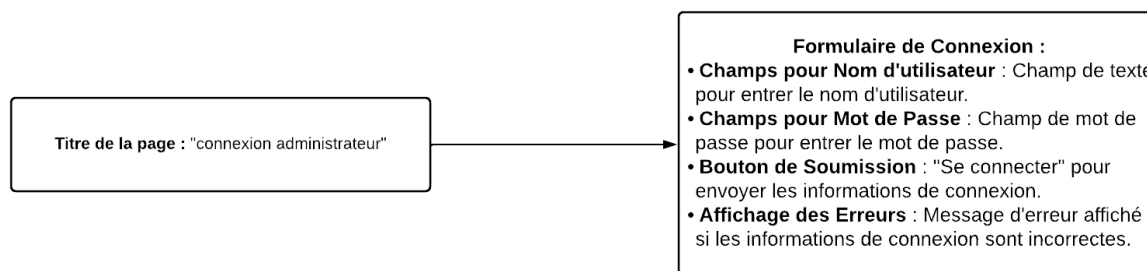


Figure II-33 : Fonctionnalité du code HTML de la page d'authentification

Source : Auteur

II.4.1.2.2 Le choix du véhicule encours de location

Afin de créer la page, plusieurs lignes de code ont été nécessaires. La page est intitulée "Choisir Voiture". Chaque voiture actuellement en cours de location est affichée sous forme de carte. En haut de chaque carte, on trouve une image de la voiture, suivie du nom de la voiture et de l'ID du véhicule affiché en dessous. Un bouton "Voir les données du véhicule" permet d'accéder aux détails du véhicule.

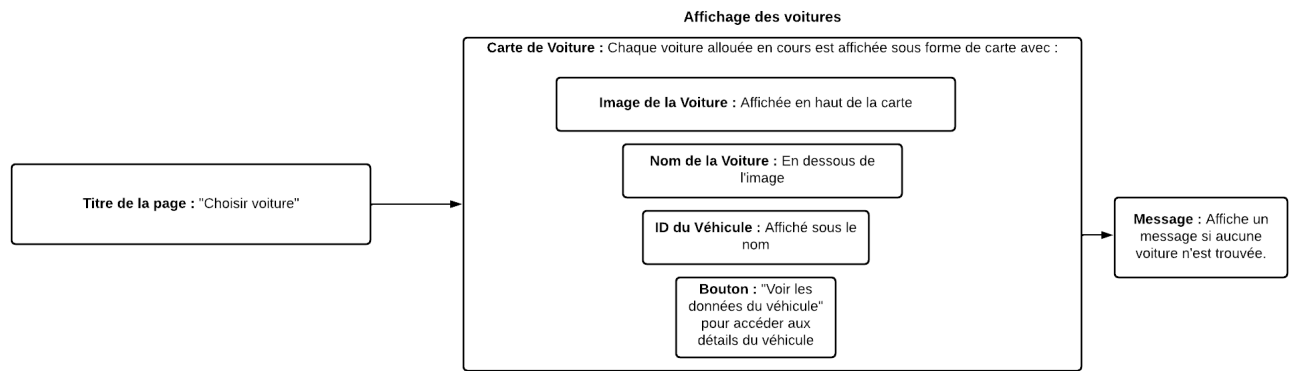


Figure II-34 : Fonctionnalité du code HTML et CSS de la page du choix du véhicule encours de location

Source : Auteur

II.4.1.2.3 Détails de la voiture

Pour créer la figure II-29, plusieurs lignes de code ont été nécessaires. La dernière page du mode administrateur est composée de trois onglets : le suivi des données des capteurs, la carte, l'historique de maintenance et les informations du locataire. Cette page est développée en HTML, CSS, JavaScript et PHP.

Le code offre une interface aux administrateurs de l'application de location de voitures, leur permettant de suivre en temps réel les données des capteurs, de visualiser la position GPS avec l'API MapTiler, de consulter l'historique de maintenance et d'accéder aux informations des locataires pour un véhicule spécifique..

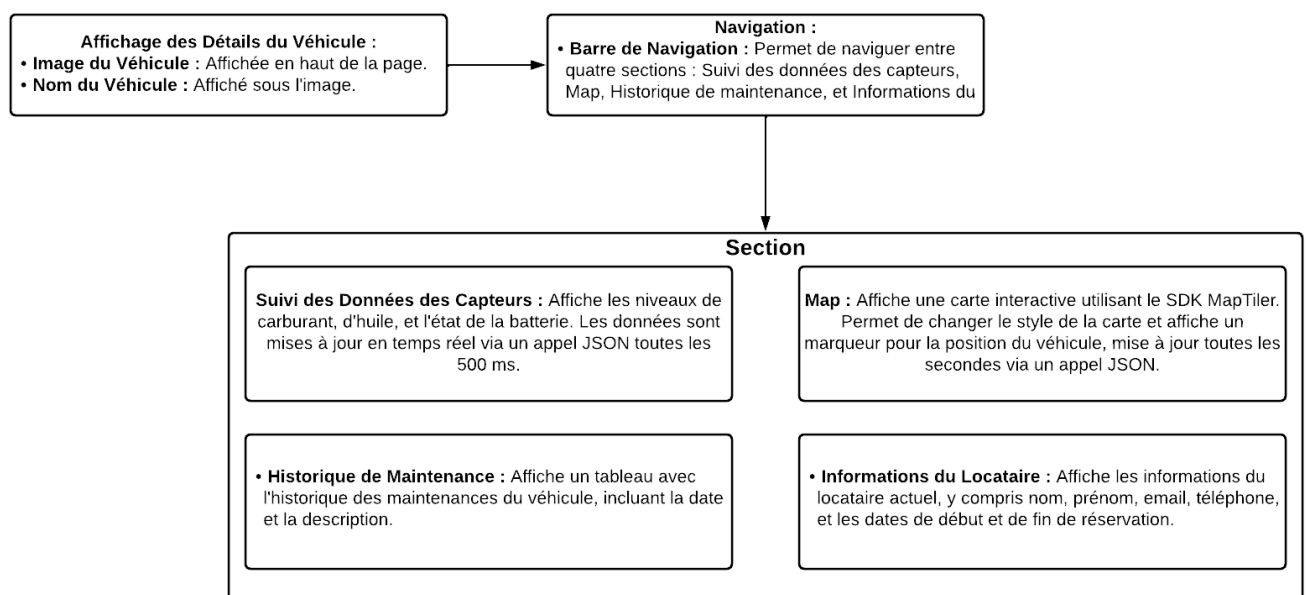


Figure II-35 : Fonctionnalité du code HTML CSS JS PHP de la page détails de la voiture

Source : Auteur

II.4.2 DEVELOPPEMENT BACKEND

Les technologies et langages utilisés pour développer le backend comprennent PHP et MySQL. PHP est un langage de script côté serveur qui permet d'interagir avec la base de données, tandis que MySQL est un système de gestion de base de données relationnelle utilisé pour stocker les informations sur les clients, les voitures, les réservations et les données des capteurs. Ensemble.

II.4.2.1. Mode utilisateur

II.4.2.1.1 Disponibilité des véhicules

Le code backend en PHP de la page récupère les dates de début et de fin de la requête POST, se connecte à la base de données MySQL, et exécute une requête pour sélectionner les voitures disponibles pendant la période spécifiée. Ensuite, il affiche les voitures disponibles ou un message si aucune voiture n'est disponible (cf. figure II-36).

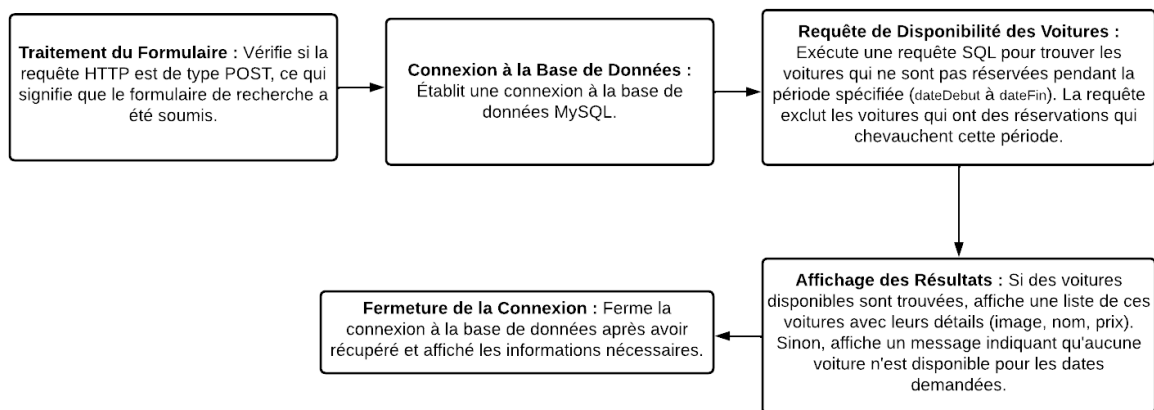


Figure II-36 : Fonctionnalité en bloc du code backend de la page disponibilité des véhicules

Source : Auteur

II.4.2.1.2 Détail du véhicule et réservation

Le code backend en PHP de la page vérifie si les paramètres de voiture (ID), de date de début, et de date de fin sont présents dans la requête GET. Il se connecte ensuite à la base de données, récupère les détails de la voiture demandée, et les affiche (cf. figure II-31).

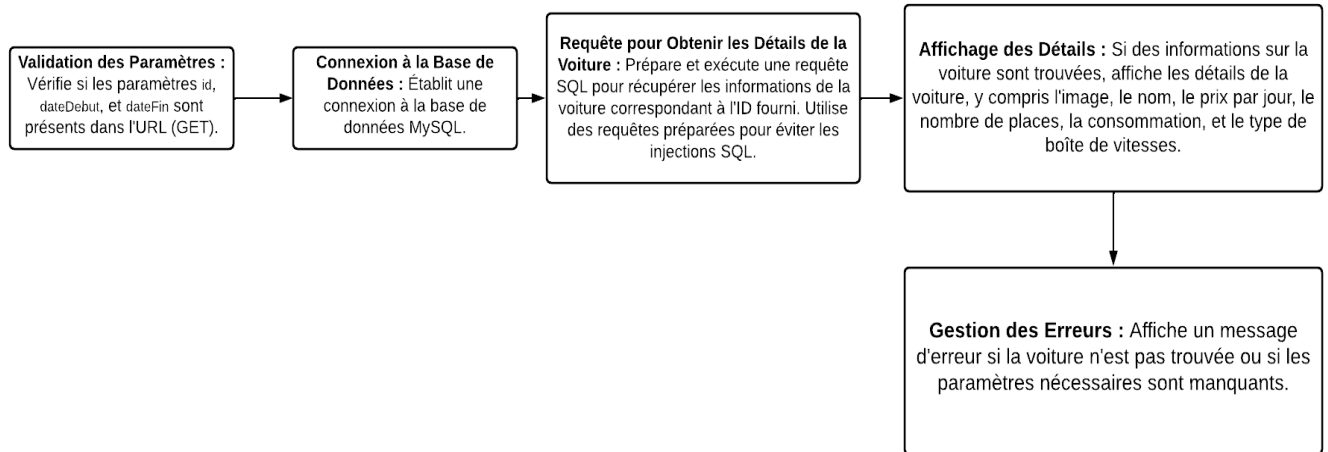


Figure II-37 : Fonctionnalité du code backend de la page détail du véhicule

Source : Auteur

II.4.2.1.3 Paiement et enregistrement des données

Le backend du code en PHP de la page de paiement vérifie que les données de réservation (ID_voiture, dateDebut, dateFin) sont présentes dans la requête POST. Il se connecte ensuite à la base de données pour obtenir le prix par jour de la voiture sélectionnée. Après cela, il calcule le montant total de la réservation en fonction de la durée et du prix par jour. Les données calculées sont ensuite affichées dans la partie HTML pour le paiement.

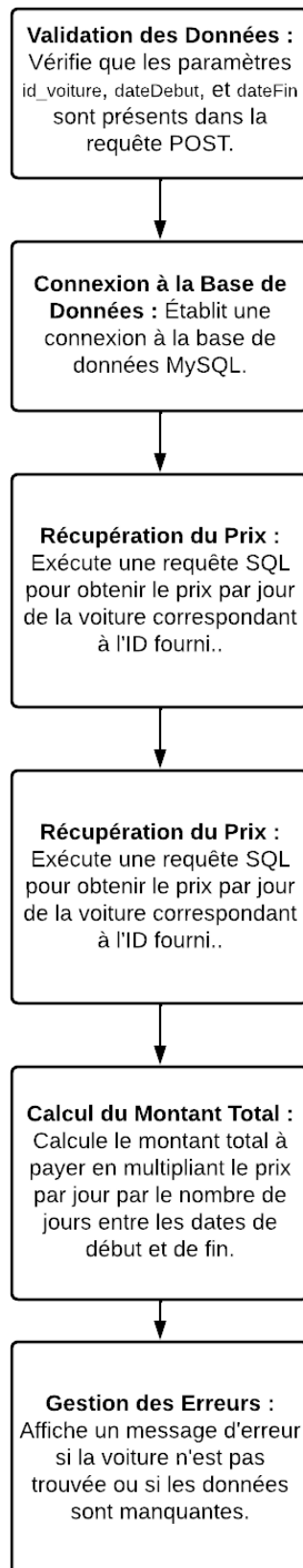


Figure II-38 : Code enregistrement des informations du client dans la base de données

Source : Auteur

Le code gère la soumission d'un formulaire de réservation de voiture en plusieurs étapes. Lorsqu'une requête POST est reçue, il récupère les informations du formulaire, telles que les détails de la voiture, les dates de réservation et les informations personnelles du client. Ensuite, il établit une connexion à la base de données "location_voiture". Le code vérifie si le client existe déjà dans la base de données en fonction de son adresse e-mail. Si le client est trouvé, son identifiant est récupéré ; sinon, un nouvel enregistrement est créé pour le client et son identifiant est stocké. Par la suite, le code ajoute une nouvelle réservation dans la base de données en utilisant les informations de la voiture, du client et des dates (cf. figure II-38).

II.4.2.2. Mode Administratif

II.4.2.2.1 L'authentification

Lorsqu'un utilisateur soumet le formulaire de connexion, le script démarre une session et récupère les informations de connexion (nom d'utilisateur et mot de passe) soumises via le formulaire. Il établit ensuite une connexion à la base de données "location_voiture" et prépare une requête SQL pour vérifier les informations d'identification en comparant le nom d'utilisateur avec ceux stockés dans la table des administrateurs. Si un utilisateur correspondant est trouvé, le mot de passe soumis est comparé avec celui stocké (cf. figure II-39).

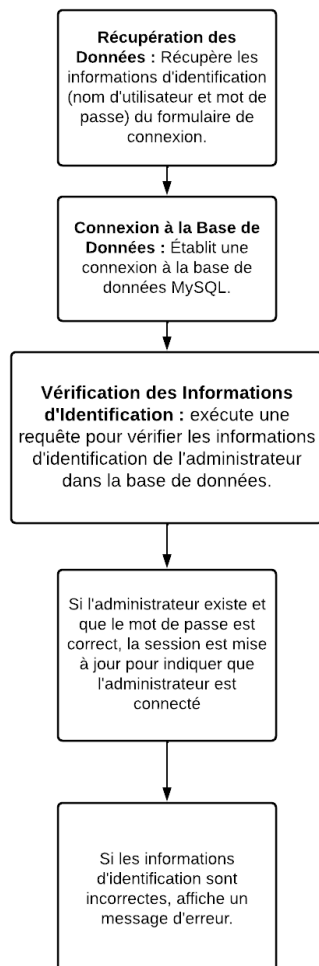


Figure II-39 : Code PHP de l'authentification

Source : Auteur

II.4.2.2.2 Le choix du véhicule encours de location

Le code backend de la page "Choix du véhicule" se connecte à la base de données et exécute une requête SQL pour récupérer les informations sur les voitures actuellement réservées, c'est-à-dire celles dont la date de fin de réservation est à venir. Les résultats de cette requête sont ensuite stockés et utilisés pour afficher les voitures réservées dans la partie front-end de la page (cf. figure II-40).

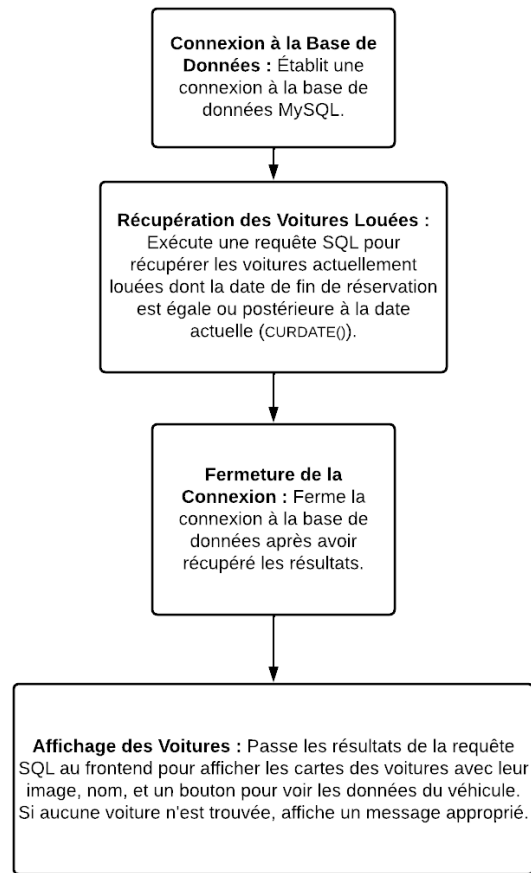


Figure II-40 : Fonctionnalité du code backend de la page choix du véhicule

Source : Auteur

II.4.2.2.3 Détails de la voiture

Le backend de cette page établit une connexion sécurisée à la base de données MySQL pour récupérer et enregistrer les informations liées aux véhicules. Après vérification des droits d'accès de l'administrateur, les détails d'un véhicule spécifique sont extraits en fonction de l'ID indiqué dans l'URL. Les données des capteurs (niveau de carburant, huile, batterie) sont mises à jour en temps réel et affichées à l'écran. L'historique de maintenance du véhicule et les informations du locataire sont également récupérés de la base de données et présentés à l'utilisateur. La position géographique du véhicule est visualisée sur la carte en temps réel à l'aide de l'API MapTiler, avec des mises à jour régulières des données de localisation. Enfin, les données des capteurs sont stockées (cf. figure II-41).

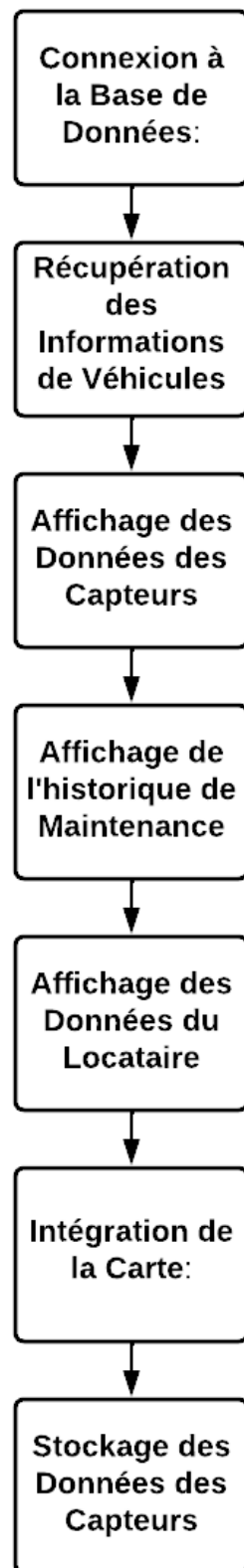


Figure II-41 : Fonctionnalité en cloc du code PHP pour la page détails du véhicule

Source : Auteur

II.4.3 DEPLOIEMENT DE L'APPLICATION WEB SUR XAAMP

Pour tester l'application, il est nécessaire de l'héberger sur un serveur. Pour cela, l'application est hébergée sur un serveur local à l'aide du logiciel XAMPP. Il faut créer un dossier 'location_voiture' dans le répertoire htdocs de XAMPP et y déplacer tous les fichiers de l'application web (cf. figure II-36 et figure II-37).

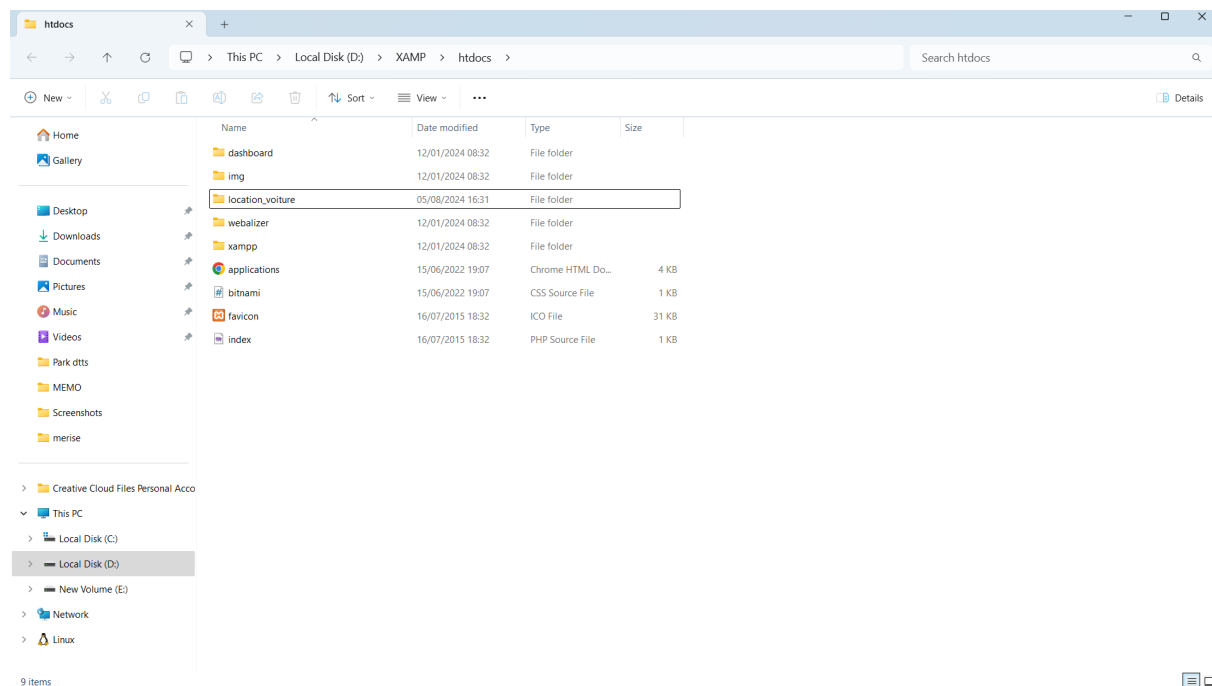


Figure II-42 : Dossier 'location_voiture' dans htdocs de XAAMP

Source : Auteur

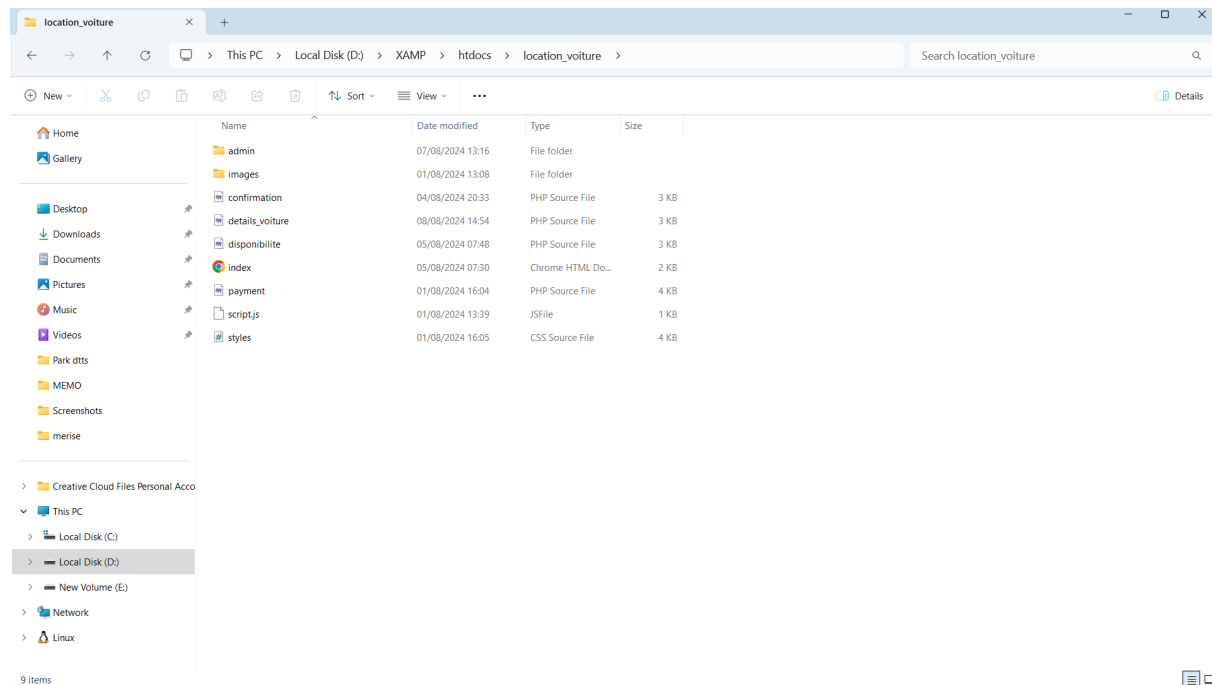


Figure II-43 : Fichiers de l'application web dans le répertoire 'location_voiture'

Source : Auteur

Chapitre II.5 Discussions

II.5.1 ANALYSE SWOT

Pour pouvoir discuter de l'application web, voici une analyse SWOT détaillée. Cette analyse met en évidence les forces, faiblesses, opportunités et menaces de l'application, permettant ainsi une évaluation globale.

Tableau II-3 : L'analyse SWOT de l'application web

Catégorie	Description
Forces	Optimisation des ressources : gestion efficace du carburant, de l'huile et de la batterie des véhicules.
	Facilité d'utilisation : Interface simple pour réserver et gérer les véhicules.
	Gestion centralisée : toutes les données (réservations, clients) sont regroupées pour une gestion facile.
	Perspective d'évolution avec la maintenance prédictive : possibilité future d'ajouter une fonctionnalité pour anticiper les pannes et améliorer la fiabilité des véhicules
Faiblesses	Dépendance technologique : dépend de la technologie comme les serveurs et les capteurs, ce qui peut avoir un impact environnemental.
	Consommation énergétique des serveurs : les serveurs consomment de l'énergie, et l'impact peut être plus élevé si l'hébergement n'est pas écologique.
	Complexité de mise en œuvre : ajouter des fonctionnalités avancées comme la maintenance prédictive peut être compliqué et coûteux.
	Maintenance et support : nécessite un entretien constant et un support technique pour fonctionner correctement.
Opportunités	Croissance du marché : la demande pour des solutions de location de voitures innovantes et écologiques est en augmentation.
	Technologies émergentes : possibilité d'intégrer de nouvelles technologies comme les véhicules électriques.
Menaces	Sécurité des données : risque de cyberattaques qui pourraient compromettre les données des clients.
	Changements technologiques : les nouvelles technologies peuvent rendre certaines fonctionnalités obsolètes ou nécessiter des mises à jour fréquentes.

Source : Auteur

II.5.2 COUT DE MISE EN ŒUVRE

Le coût de mise en œuvre du projet dépend directement du nombre de voitures à louer. Pour 20 véhicules, le calcul prend en compte le prix unitaire de chaque composant ainsi que le coût annuel de l'hébergement du site. (

Composants	Prix unitaire	Quantité	Total
Module GPS Neo6m	30 000Ar	20	600 000Ar
ESP32-cam	70 000Ar	20	1 400 000Ar
Module LTE	100 000Ar	20	370 000Ar
Hébergement du site	285 000 Ar/an	1	285 000Ar
Total			9 685 000Ar

Tableau II-4 : Coût de mise en œuvre

Source :[35] [36], [37]

Ainsi, pour 20 voitures, le coût total de mise en œuvre, incluant l'hébergement du site pour une année, s'élève à 9 685 000 Ar.

II.5.3 PERSPECTIVE D'AMELIORATION

Le projet vise à intégrer des fonctionnalités de maintenance prédictive dans la gestion des véhicules en utilisant des données collectées à partir des capteurs. Pour ce faire, il est primordial d'avoir un grand volume de données afin de concevoir un modèle prédictif basé sur la forêt aléatoire.

Afin d'évaluer la fiabilité du modèle forêt aléatoire pour notre système, un test a été réalisé avec un problème similaire. Sur Kaggle, il existe un jeu de données contenant des mesures de capteurs provenant de différentes machines, incluant diverses lectures de capteurs et les pannes enregistrées .[38]

Voici les descriptions des colonnes des données :

- **footfall** : Nombre de personnes ou d'objets passant près de la machine.
- **tempMode** : Mode de température ou réglage de la machine.
- **AQ** : Indice de qualité de l'air près de la machine.
- **USS** : Données du capteur ultrasonique, indiquant les mesures de proximité.
- **CS** : Lecture du capteur de courant, indiquant l'utilisation électrique de la machine.
- **VOC** : Niveau de composés organiques volatils détectés près de la machine.

- **RP** : Position rotative ou RPM (révolutions par minute) des pièces de la machine.
- **IP** : Pression d'entrée dans la machine.
- **Temperature** : Température de fonctionnement de la machine.
- **fail** : Indicateur binaire de panne de la machine (1 pour une panne, 0 pour aucune panne).

II.5.3.1. Séparation des données de test et d'apprentissage

942 données sont utilisées pour concevoir le modèle prédictif se basant sur l'arbre aléatoire.

```
data_csv.csv
1  footfall;tempMode;AQ;USS;CS;VOC;RP;IP;Temperature;fail
2  0;7;7;1;6;6;36;3;1;1
3  190;1;3;3;5;1;20;4;1;0
...
942 0;7;5;2;0;0;30;0;24;1
943 0;3;6;2;7;5;43;6;24;1
944 0;6;6;2;5;6;46;7;24;1
945 18;7;4;2;6;3;61;7;24;1
946
```

Figure II-44 : Nombre de données pour concevoir le modèle prédictif teste

Source : Auteur

Les données sont divisées en deux : des données d'entraînement : 70% des données et des données tests : 30% des données

```
37
38 # Diviser les données en ensembles d'entraînement et de test
39 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
40
```

Figure II-45 : Divisions des données en test et en entraînement

Source : Auteur

II.5.3.2. Création du modèle

Pour entraîner le modèle, il est d'abord nécessaire de créer un modèle vide en utilisant la méthode `RandomForestClassifier()` (cf figure II-47).

```
57 model = RandomForestClassifier(n_estimators=100, random_state=42)
```

Figure II-46 : Création du modèle forêt aléatoire

Source : Auteur

II.5.3.3. Entraînement du modèle

L'entraînement du modèle se fait avec les données d'entraînement.

```
58 model.fit(X_train_res, y_train_res)
```

Figure II-47 : Entraînement du modèle

Source : Auteur

II.5.3.4. Évaluation du modèle

L'évaluation du modèle se fait avec les données testées en utilisant la méthode `accuracy_score()` (cf figure II-49)

```
60 # Faire des prédictions
61 y_pred = model.predict(X_test_scaled)
62
63 # Évaluer le modèle
64 accuracy = accuracy_score(y_test, y_pred)
65 print(f"Accuracy: {accuracy:.4f}")
66
67 print("Confusion Matrix:")
68 print(confusion_matrix(y_test, y_pred))
69
70 print("Classification Report:")
71 print(classification_report(y_test, y_pred))
```

Figure II-48 : Évaluation du modèle

Source : Auteur

II.5.3.4.1 Matrice de confusion

Ci-après se trouve la matrice de confusion obtenue lors de l'évaluation.

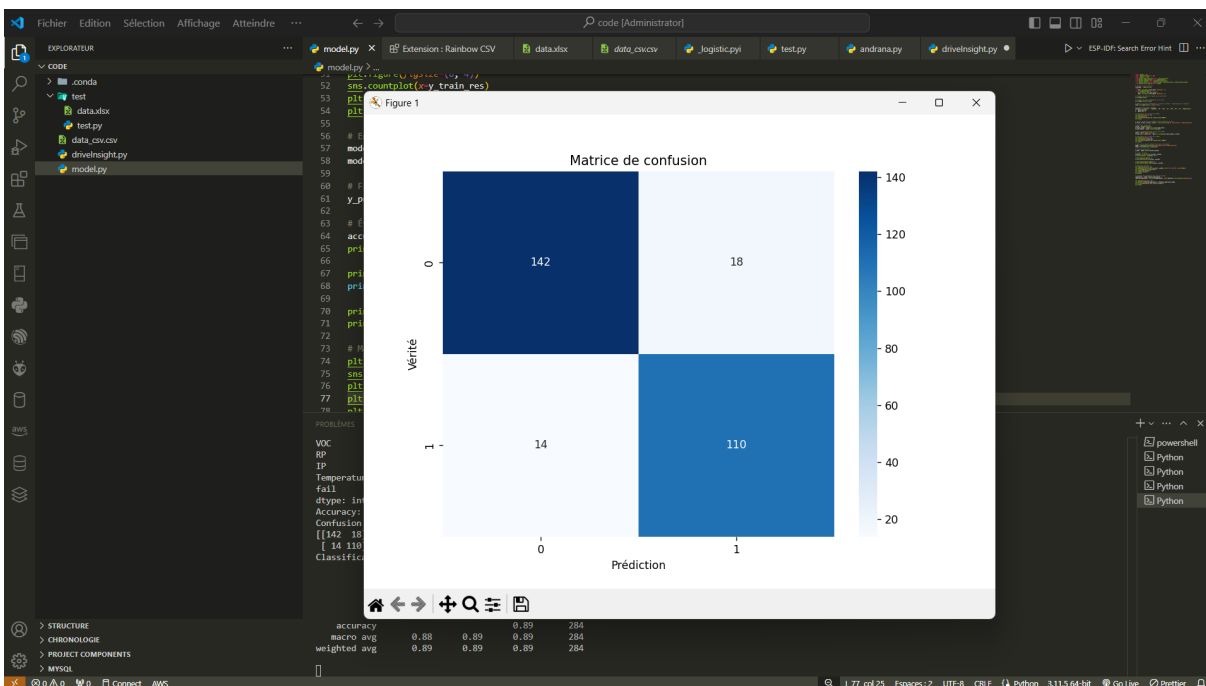


Figure II-49 : Matrice de confusion

Source : Auteur

La figure II-50 montre les nombres de vrais négatifs, faux positifs, faux négatifs et le nombre de vrais positifs dont :

- **142** : Nombre de vrais négatifs (classe 0 correctement prédite comme 0).
- **18** : Nombre de faux positifs (classe 0 incorrectement prédite comme 1).
- **14** : Nombre de faux négatifs (classe 1 incorrectement prédite comme 0).
- **110** : Nombre de vrais positifs (classe 1 correctement prédite comme 1).

II.5.3.4.2 Précision, recall et support

La figure II-43 montre le rapport de classification de l'évaluation :

Classification Report:				
	precision	recall	f1-score	support
0	0.91	0.89	0.90	160
1	0.86	0.89	0.87	124

Figure II-50 : Rapport de classification de l'évaluation

Source : Auteur

- Précision :

La précision est le pourcentage de prédictions positives correctes sur toutes les prédictions positives faites par le modèle.

- Précision pour la classe **0** : **0.91** (91%)
- Précision pour la classe **1** : **0.86** (86%)

- Rappel

Le rappel est le pourcentage d'instances positives qui ont été correctement identifiées par le modèle.

- Rappel pour la classe **0** : **0.89** (89%)
- Rappel pour la classe **1** : **0.89** (89%)

- F1-Score

Le F1-score est la moyenne harmonique de la précision et du rappel, fournissant une mesure unique qui prend en compte à la fois la précision et le rappel.

- F1-Score pour la classe **0** : **0.90**
- F1-Score pour la classe **1** : **0.87**

En résumé, votre modèle est fiable, avec une précision globale de 88.73 %, et des performances solides en termes de précision, rappel et F1-score pour les deux classes.

CONCLUSION GENERALE

En conclusion, cette étude a conduit à la création d'une application web pour la gestion et la réservation de véhicules de location, combinant l'exploitation de l'informatique et des technologies de systèmes automatisés. En utilisant des technologies modernes telles que le GPS et les capteurs intégrés, ainsi qu'une carte ESP32-CAM pour collecter des données cruciales comme le niveau de carburant, l'application permet de suivre les véhicules en temps réel. Elle propose deux modes d'utilisation, à savoir un mode administrateur pour la gestion des véhicules et un mode utilisateur pour faciliter les réservations et les paiements en ligne. Cette intégration technologique permet de simplifier la gestion des véhicules de location et d'améliorer la qualité du service.

L'objectif principal de l'étude était d'explorer comment améliorer le service client et optimiser la gestion des flottes en exploitant les capteurs des véhicules. L'application web développée répond à cette problématique en permettant une surveillance en temps réel, une gestion centralisée des informations, et une interface conviviale pour les clients et les gestionnaires.

Cependant, des limites subsistent. La précision et la fiabilité des capteurs peuvent varier en fonction des conditions d'utilisation et des modèles de véhicules. De plus, l'intégration des capteurs et du module 3G/LTE peut entraîner des coûts supplémentaires. La gestion de grandes flottes avec des besoins d'entretien variés reste un défi, et l'application pourrait être améliorée pour mieux anticiper les besoins de maintenance.

Pour surmonter ces limites, plusieurs perspectives d'amélioration peuvent être envisagées. L'intégration de capteurs avancés pour la maintenance prédictive permettrait d'anticiper les pannes potentielles en analysant les données collectées. L'utilisation de modèles de machine learning pourrait également améliorer la précision des prédictions et réduire les coûts de maintenance. De plus, l'extension de l'application pour inclure d'autres paramètres de performance des véhicules, comme la température du moteur et la pression des pneus, offrirait une vue plus complète de l'état des véhicules. Enfin, la mise en place d'un système de notifications et d'alertes pour les administrateurs et les utilisateurs améliorerait encore la gestion des véhicules de location.

REFERENCES BIBLIOGRAPHIQUES

- [1] « archi._des_systemes_automatisees.pdf ». Consulté le: 14 septembre 2023. [En ligne]. Disponible sur: http://sc-st.univ-batna2.dz/sites/default/files/sc_st/files/archi._des_systemes_automatisees.pdf
- [2] « Qu'est-ce que l'IoT et comment fonctionne-t-il ? », SAP. Consulté le: 30 juillet 2024. [En ligne]. Disponible sur: <https://www.sap.com/suisse/products/artificial-intelligence/what-is-iot.html>
- [3] « ESP32-DevKitC V4 Getting Started Guide - ESP32 - — ESP-IDF Programming Guide latest documentation ». Consulté le: 16 septembre 2023. [En ligne]. Disponible sur: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/hw-reference/esp32/get-started-devkitc.html>
- [4] « ESP32 Camera with USB Connectivity », Cytron Technologies Malaysia. Consulté le: 17 septembre 2023. [En ligne]. Disponible sur: <https://my.cytron.io/p-esp32-cam-wireless-iot-vision-development-board>
- [5] « Guide de brochage ESP32-CAM AI-Thinker : explication de l'utilisation des GPIO ». Consulté le: 18 septembre 2023. [En ligne]. Disponible sur: <https://www.raspberryme.com/guide-de-brochage-esp32-cam-ai-thinker-explication-de-lutilisation-des-gpio/>
- [6] « Guide to NEO-6M GPS Module Arduino | Random Nerd Tutorials ». Consulté le: 5 août 2024. [En ligne]. Disponible sur: <https://randomnerdtutorials.com/guide-to-neo-6m-gps-module-with-arduino/>
- [7] « Des puces capables de calculer et stocker sur un même SoC - Le Monde Informatique », LeMondelInformatique. Consulté le: 5 août 2024. [En ligne]. Disponible sur: <https://www.lemondeinformatique.fr/actualites/lire-des-puces-capables-de-calculer-et-stocker-sur-un-meme-soc-78303.html>
- [8] « ZED-F9P module », u-blox. Consulté le: 5 août 2024. [En ligne]. Disponible sur: <https://www.u-blox.com/en/product/zed-f9p-module>
- [9] « LEA-6 series », u-blox. Consulté le: 5 août 2024. [En ligne]. Disponible sur: <https://www.u-blox.com/en/product/lea-6-series>
- [10] « SIM800L V2.0 5V MODULE GSM GPRS sans fil quadri-bande | YoupiLab Components ». Consulté le: 5 août 2024. [En ligne]. Disponible sur: <https://youpilab.com/components/product/sim800l-v20-5v-module-gsm-gprs-sans-fil-quadri-bande>
- [11] « keystudio SIM5320E 3G Module GSM GPRS GPS Modules for Arduino 51 AVR MCU ». Consulté le: 5 août 2024. [En ligne]. Disponible sur: <https://www.keystudio.com/products/keystudio-sim5320e-3g-module-gsm-gprs-gps-modules-for-arduino-51-avr-mcu>
- [12] « Particle Boron LTE IoT Development Board - WRL-15069 - SparkFun Electronics ». Consulté le: 5 août 2024. [En ligne]. Disponible sur: <https://www.sparkfun.com/products/retired/15069>
- [13] « Qu'est-ce qu'un environnement de développement intégré (IDE) ? » Consulté le: 20 septembre 2023. [En ligne]. Disponible sur: <https://www.redhat.com/fr/topics/middleware/what-is-ide>
- [14] « Définition du Développement Web : Le Futur du Numérique ». Consulté le: 5 août 2024. [En ligne]. Disponible sur: <https://www.digitalschool.paris/guides/developpement-web/definition/>
- [15] P.-A. Marendat, « Single Page Application - HTML 5, CSS3, JavaScript », SoftFluent. Consulté le: 5 août 2024. [En ligne]. Disponible sur: <https://www.softfluent.fr/blog/single-page-application-html-5-css3-javascript/>
- [16] « CSS logo and symbol, meaning, history, PNG ». Consulté le: 5 août 2024. [En ligne]. Disponible sur: <https://1000logos.net/css-logo/>

- [17] SMI, « js-logo », The International Conference On Missions. Consulté le: 5 août 2024. [En ligne]. Disponible sur: <https://theicom.org/js-logo/>
- [18] « React, programing icon - Download on Iconfinder ». Consulté le: 5 août 2024. [En ligne]. Disponible sur: https://www.iconfinder.com/icons/7526871/react_programing_icon
- [19] Nica, « AngularJS logo », Dwglogo. Consulté le: 5 août 2024. [En ligne]. Disponible sur: <https://dwglogo.com/angularjs/>
- [20] « Vue.js full logo transparent PNG - StickPNG ». Consulté le: 5 août 2024. [En ligne]. Disponible sur: <https://www.stickpng.com/img/icons-logos-emojis/tech-companies/vuejs-full-logo>
- [21] « Memoire Online - Développement d'une application web pour l'optimisation du processus d'archivage et d'accès aux données d'une entreprise. Cas de Bell Equipement. - Freddy ILUNGA KADIATA », Memoire Online. Consulté le: 6 août 2024. [En ligne]. Disponible sur: https://www.memoireonline.com/09/18/10304/m_Developpement-d-une-application-web-pour-l-optimisation-du-processus-d-archivage-et-dacces-aux25.html
- [22] C.-A. Azencott, « Introduction au Machine Learning ».
- [23] Machine Learnia, *FORMATION DEEP LEARNING COMPLETE (2021)*, (25 avril 2021). Consulté le: 27 septembre 2023. [En ligne Vidéo]. Disponible sur: <https://www.youtube.com/watch?v=XUFLq6dKQok>
- [24] B. Roy, « Monte Carlo Learning », Medium. Consulté le: 19 octobre 2023. [En ligne]. Disponible sur: <https://towardsdatascience.com/monte-carlo-learning-b83f75233f92>
- [25] P. Sharma, « The Ultimate Guide to K-Means Clustering: Definition, Methods and Applications », Analytics Vidhya. Consulté le: 19 octobre 2023. [En ligne]. Disponible sur: <https://www.analyticsvidhya.com/blog/2019/08/comprehensive-guide-k-means-clustering/>
- [26] D. Team, « Algorithme de descente de gradient - Machine Learning », Formation Data Science | DataScientest.com. Consulté le: 19 octobre 2023. [En ligne]. Disponible sur: <https://datascientest.com/descente-de-gradient>
- [27] Pitpitt, « Optimisation Adam — DataFranca ». Consulté le: 19 octobre 2023. [En ligne]. Disponible sur: https://datafranca.org/wiki/Optimisation_Adam
- [28] S. Saha, « A Comprehensive Guide to Convolutional Neural Networks — the ELI5 way », Medium. Consulté le: 20 octobre 2023. [En ligne]. Disponible sur: <https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53>
- [29] « Qu'est ce qu'un réseau de neurones convolutif (ou CNN) ? », OpenClassrooms. Consulté le: 20 octobre 2023. [En ligne]. Disponible sur: <https://openclassrooms.com/fr/courses/4470531-classez-et-segmentez-des-donnees-visuelles/5082166-quest-ce-quun-reseau-de-neurones-convolutif-ou-cnn>
- [30] « (56) Random Forest Algorithm Clearly Explained! - YouTube ». Consulté le: 6 août 2024. [En ligne]. Disponible sur: <https://www.youtube.com/>
- [31] « RandomForestRegressor », scikit-learn. Consulté le: 6 août 2024. [En ligne]. Disponible sur: <https://scikit-learn/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>
- [32] « Définir les besoins avec l'outil "Bête à cornes" en gestion de projet ». Consulté le: 16 octobre 2023. [En ligne]. Disponible sur: <https://www.manager-go.com/gestion-de-projet/dossiers-methodes/bete-a-cornes>
- [33] « Diagramme FAST ▷ définition, exemples et modèle à télécharger », appvizer.fr. Consulté le: 21 octobre 2023. [En ligne]. Disponible sur: <https://www.appvizer.fr/magazine/operations/gestion-de-projet/diagramme-fast>
- [34] S. CUBE, « SMART CUBE », SMART CUBE. Consulté le: 5 août 2024. [En ligne]. Disponible sur: <https://www.smart-cube.biz/produit/module-neo-6m-gps/>

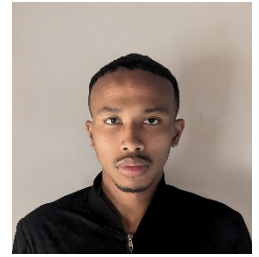
- [35] « esp32 cam - Achat en ligne | Aliexpress ». Consulté le: 9 août 2024. [En ligne]. Disponible sur: <https://fr.aliexpress.com/w/wholesale-esp32-cam.html>
- [36] « Gps Neo 6m - Circuits Intégrés - AliExpress ». Consulté le: 9 août 2024. [En ligne]. Disponible sur: <https://fr.aliexpress.com/w/wholesale-gps-neo-6m.html>
- [37] « Hébergement web professionnel - Créateur de Domaine - .MG ». Consulté le: 9 août 2024. [En ligne]. Disponible sur: <https://www.madagascar-internet.mg/>
- [38] « Machine Failure Prediction using Sensor data ». Consulté le: 6 août 2024. [En ligne]. Disponible sur: <https://www.kaggle.com/datasets/umerrtx/machine-failure-prediction-using-sensor-data>

Nom : RANDRIAMASINORO

Prénom : Sehenonirina Elisa

Tél : +261 34 99 361 80

Email : randriamasnrelisa@gmail.com



Nombre de pages : **76** Nombre de Tableaux : **5** Nombre de figures : **83**

Thème : **ÉTUDE ET DEVELOPPEMENT DE DRIVE INSIGHT, UNE APPLICATION WEB POUR LA GESTION DE LOCATION DE VEHICULES.**

Résumé :

Cette étude présente la création d'une application web pour la gestion et la réservation de véhicules de location en ligne. L'objectif est d'améliorer la satisfaction des clients tout en optimisant la gestion des véhicules. L'application utilise des technologies modernes, incluant un GPS pour le suivi en temps réel des véhicules et une carte ESP32-CAM pour collecter des données cruciales, telles que le niveau de carburant, à partir des capteurs des véhicules. Ces données sont transmises à un serveur via un module 3G/LTE, permettant ainsi une surveillance et un contrôle à distance. L'application web comporte deux modes, administrateur et utilisateur. Le mode administrateur permet de suivre la position et l'état des véhicules, de consulter l'historique des maintenances, et d'accéder aux informations des clients. Le mode utilisateur permet aux clients de réserver des véhicules, de choisir les dates de location, de consulter les détails des véhicules disponibles et d'effectuer des paiements en ligne. Grâce à ces technologies, le système offre une gestion plus efficace des locations de véhicules, avec une surveillance en temps réel, une gestion centralisée des informations, et une interface utilisateur conviviale. Cette application simplifie la gestion des véhicules de location et améliore la qualité du service. Pour les perspectives d'amélioration, l'intégration de la maintenance prédictive et l'utilisation de modèles de machine learning pourraient encore perfectionner le système, en anticipant les pannes et en optimisant la disponibilité des véhicules. De plus, l'extension des capacités de surveillance avec des capteurs supplémentaires pour divers paramètres des véhicules pourrait offrir une vue encore plus complète et détaillée de l'état de la flotte, augmentant ainsi la sécurité et la fiabilité du service.

Mots-clés : gestion des véhicules, application web, capteurs, système IoT.

Rapporteur : Madame Mirantsoa Volana ANDRIAMANDIMBISOA